

AI Agent x MCP 應用實作體驗營

Local LLM Application First

#JD
jeddy.hwang@gmail.com

自我介紹

- PLM/PDM 客製開發(研發)
- WiMAX 晶片設計(研發)
- 工業基礎建設專案(研發)
- 工業4.0推廣(工具機)(研發)
- DigiTalent (子)計劃(主持)
- 大型語言模型應用開發(研發)



- 經濟部產業人才能力鑑定iPAS AI應用規劃師初級 (2026)
- 經濟部產業人才能力鑑定iPAS AI應用規劃師中級 (2026)

今天的重點

今天的材料及大綱

- <https://github.com/jeddyclass/AIAGENT202609.git>
- 認識現在的大型語言模型
- 認識現在的代理模式
- 認識必要的部件(圖、環、驗及工具等)
- 認識框架
- 實例展示



最近的大事

2026-09-15



最近的大事

2026-09-15

- 駁斥 AI 末日論：黃仁勳直言有關 AI 導致人類滅絕的恐懼與預測「不科學且全是編造」，美國總統川普隨後在直播連線中也附和此觀點。
- 點出能源與電力瓶頸：他強調 AI 下一階段的限制不在於模型本身，而是電力與資料中心的容量，凸顯基礎建設落後於運算需求。
- 推動推理市場（Inference）：AI 的價值正從訓練展示轉向大規模的推理經濟，推升了加速運算與節能基建的需求。
- 政治與科技的交匯：川普突發電話連線，將這場對談從單純的企業技術分享，提升為美國國家級 AI 企圖心與戰略資源的全球對話。

現在的AI 及Agent

語言模型的今天

好生意!!

OpenAI Developers

Home API Codex ChatGPT Resources

API Dashboard

Get started

Overview

Get started

Access and CLI

Latest: GPT-5.5

Prompt guidance

Core concepts

Text generation

Code generation

Images and vision

Audio and speech

Structured output

Function calling

Responses API

Using tools

Agents SDK

Overview

Quickstart

Agent definitions

Pricing

Copy Page

Flagship models

Our latest models

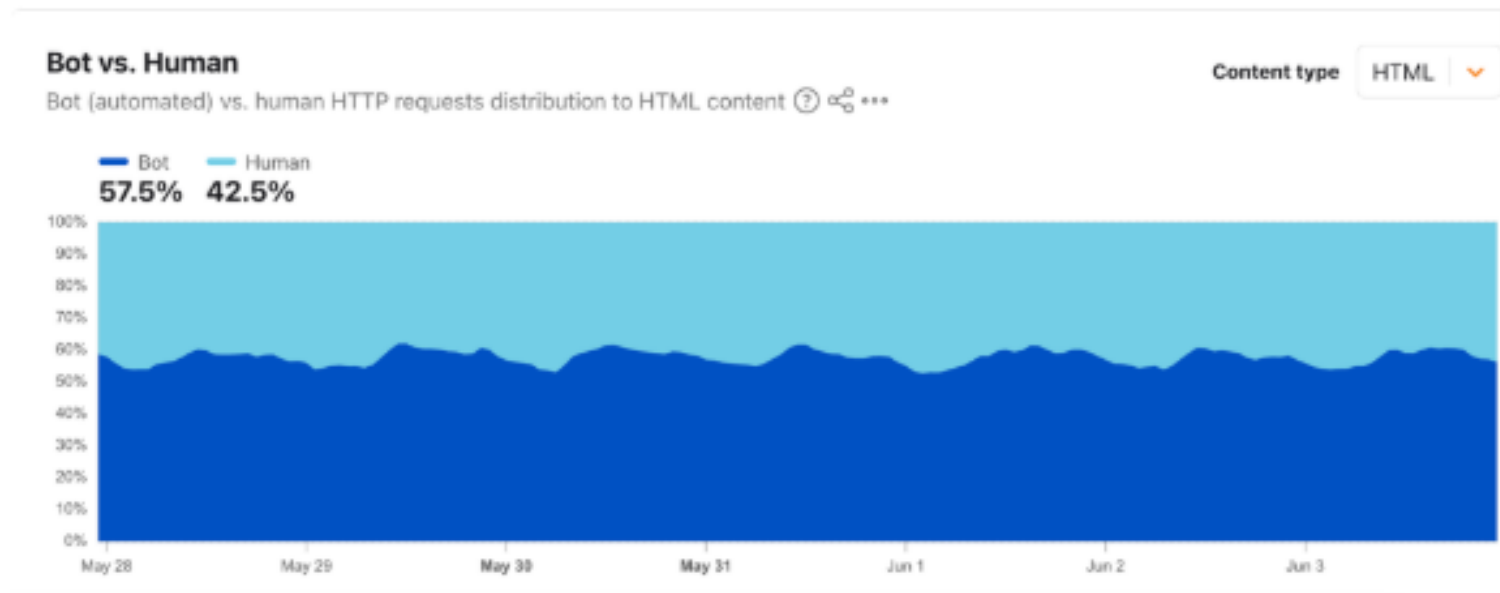
Prices per 1M tokens.

Standard Batch Flex Priority

Model	Short context			Long context		
	Input	Cached input	Output	Input	Cached input	Output
gpt-5.5	\$5.00	\$0.50	\$30.00	\$10.00	\$1.00	\$45.00
gpt-5.5-pro	\$30.00	-	\$180.00	\$60.00	-	\$270.00
gpt-5.4	\$2.50	\$0.25	\$15.00	\$5.00	\$0.50	\$22.50
gpt-5.4-mini	\$0.75	\$0.075	\$4.50	-	-	-
gpt-5.4-nano	\$0.20	\$0.02	\$1.25	-	-	-
gpt-5.4-pro	\$30.00	-	\$180.00	\$60.00	-	\$270.00

Regional processing (data residency) endpoints are charged a 10% uplift for models released on or after March 5, 2026, that are eligible for data residency. See our [Your data](#) guide for supported regions and processing details. [OpenAI models in Amazon Bedrock](#) are billed through AWS and may differ from direct OpenAI pricing.

語言模型的今天



Bit Vs. Human traffic for 2026. Source: <https://radar.cloudflare.com/traffic#bot-vs-human>

Cloudflare

語言模型的今天

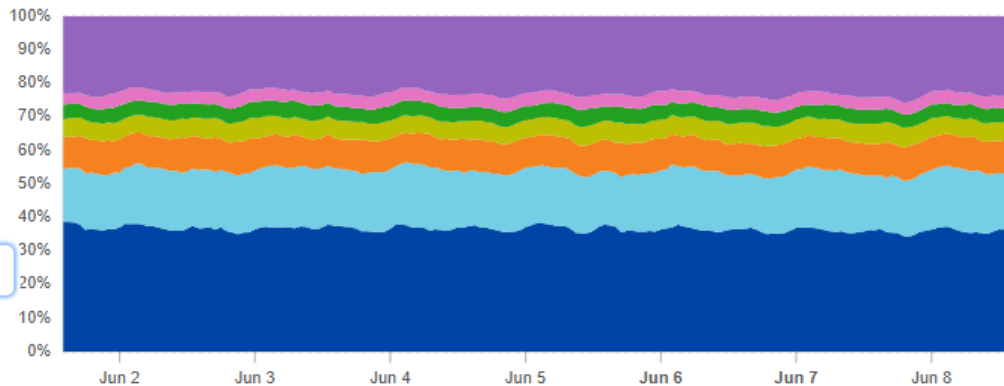
Content type

Distribution of HTTP responses, grouped by MIME type category ? 🔗 ...

Traffic type

Bot traffic

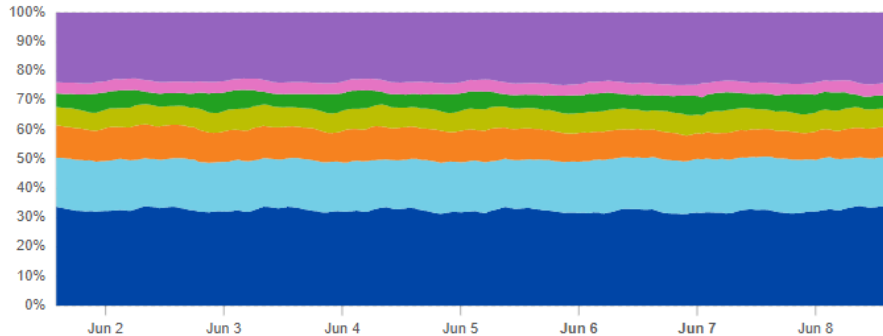
JSON 36.6% HTML 17.3% Plain text 9.5% Images 5.5% JavaScript 4.3% XML 3.7% Other 23.1%



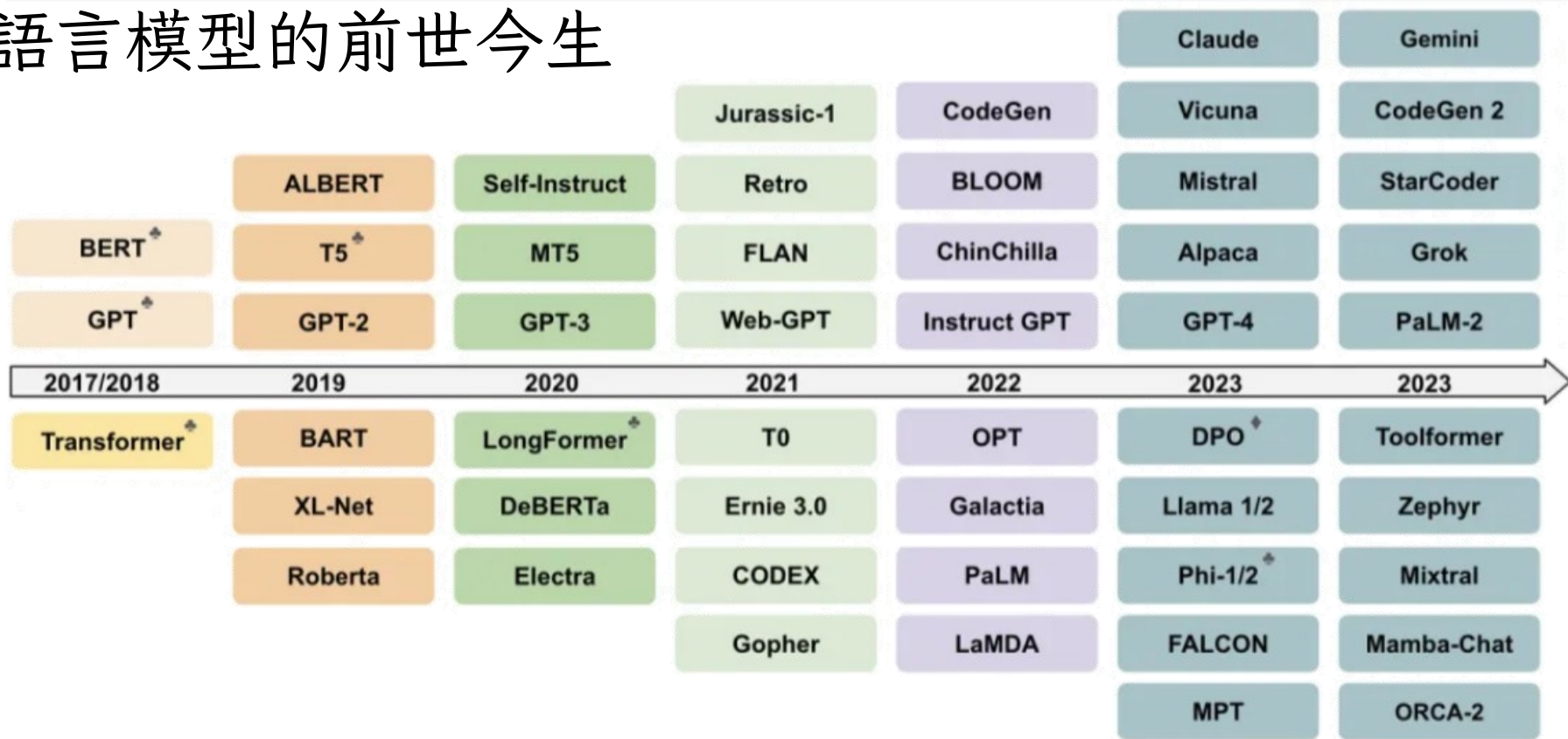
Traffic type

Human traffic

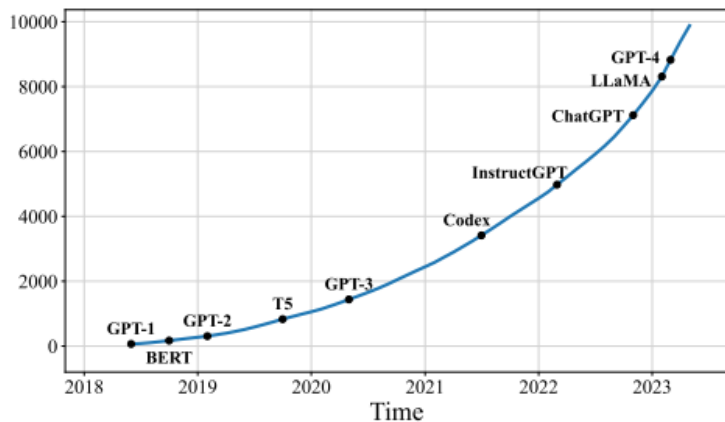
JSON 32.6% Images 17.3% JavaScript 10.3% HTML 6.8% Video 5.4% Plain text 3.9% Other 23.7%



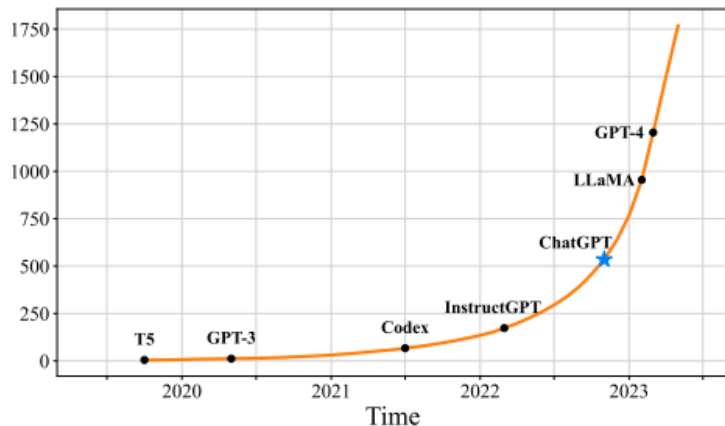
語言模型的前世今生



語言模型的前世今生



(a) Query="Language Model"



(b) Query="Large Language Model"

Fig. 1: The trends of the cumulative numbers of arXiv papers that contain the keyphrases "*language model*" (since June 2018) and "*large language model*" (since October 2019), respectively. The statistics are calculated using exact match by querying the keyphrases in title or abstract by months. We set different x-axis ranges for the two keyphrases, because "*language models*" have been explored at an earlier time. We label the points corresponding to important landmarks in the research progress of LLMs. A sharp increase occurs after the release of ChatGPT: the average number of published arXiv papers that contain "*large language model*" in title or abstract goes from 0.40 per day to 8.58 per day (Figure 1(b)).

語言模型的前世今生

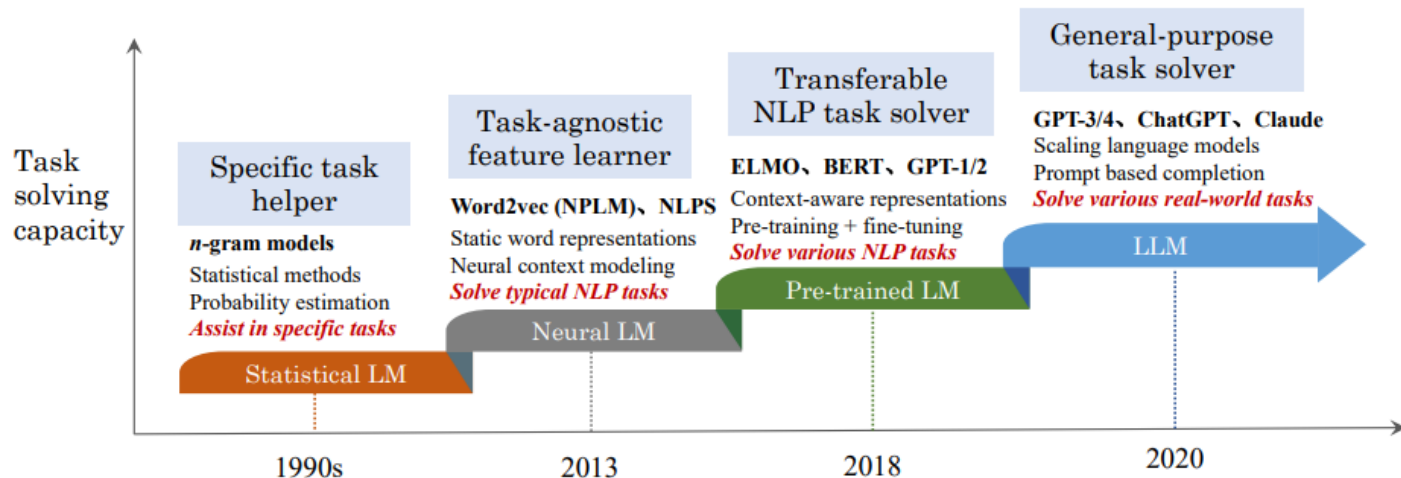


Fig. 2: An evolution process of the four generations of language models (LM) from the perspective of task solving capacity. Note that the time period for each stage may not be very accurate, and we set the time mainly according to the publish date of the most representative studies at each stage. For neural language models, we abbreviate the paper titles of two representative studies to name the two approaches: NPLM [1] (“A neural probabilistic language model”) and NLPS [2] (“Natural language processing (almost) from scratch”). Due to the space limitation, we don’t list all representative studies in this figure.

語言模型的前世今生

	LLM（大型語言模型）	Agent LLM（自主代理人）
定位與架構	單一的模型組件（例如 GPT-4、Claude 3）	圍繞目標構建的複雜系統（模型+工具+記憶+循環）
運作機制	無狀態（Stateless）：給予 Prompt 則回應，對話結束即遺忘	有狀態（Stateful）：擁有控制循環，會追蹤進度與狀態
記憶能力	僅依賴當前對話的上下文視窗（Context Window）	具備短期與長期記憶機制，能儲存歷史經驗
外部聯動	無法獨立行動，受限於靜態的訓練數據	能呼叫 API、資料庫、網頁瀏覽等外部工具
自主程度	被動。一問一答，必須由人類驅動下一步	主動。給予核心目標後，能自主拆解步驟並循環修正
主要輸出	文字內容、程式碼、文本摘要	已完成的任務結果、自動化操作流程

語言模型的前世今生

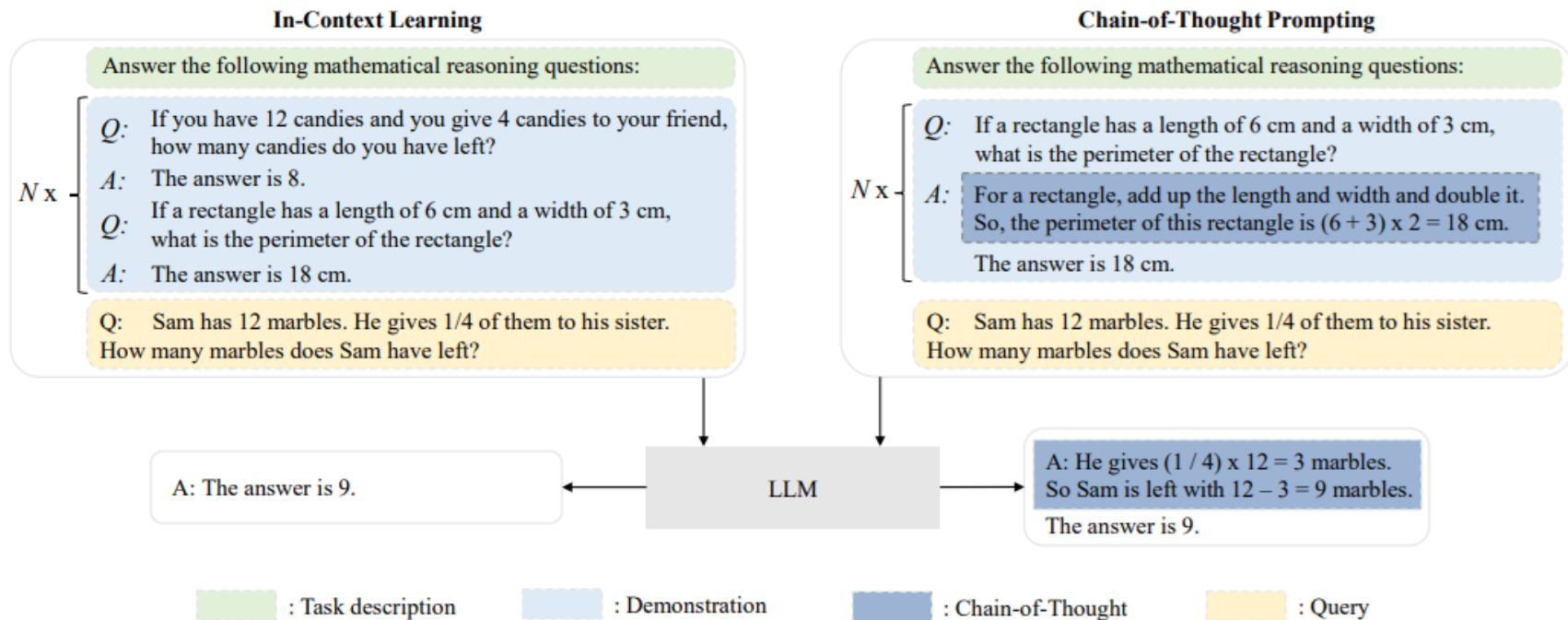


Fig. 14: A comparative illustration of in-context learning (ICL) and chain-of-thought (CoT) prompting. ICL prompts LLMs with a natural language description, several demonstrations, and a test query, while CoT prompting involves a series of intermediate reasoning steps in prompts.

LLM的新方向

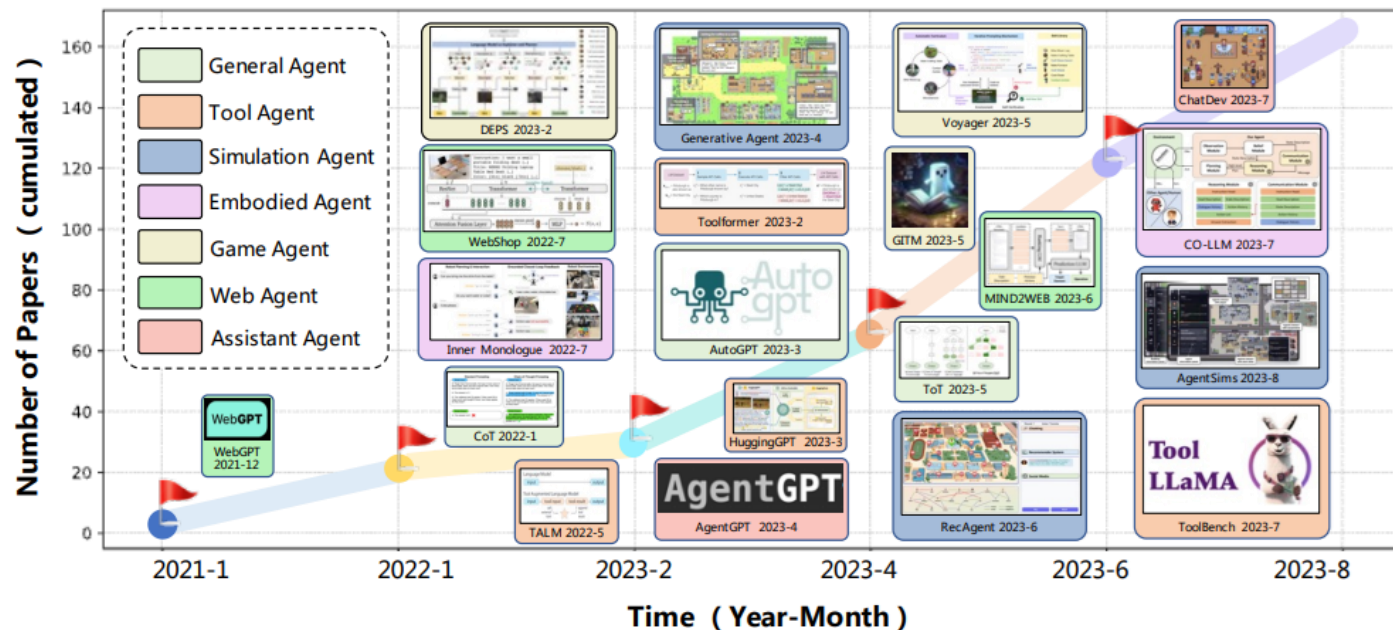


Fig. 1 Illustration of the growth trend in the field of LLM-based autonomous agents. We present the cumulative number of papers published from January 2021 to August 2023. We assign different colors to represent various agent categories. For example, a game agent aims to simulate a game-player, while a tool agent mainly focuses on tool using. For each time period, we provide a curated list of studies with diverse agent categories.

LLM的新方向

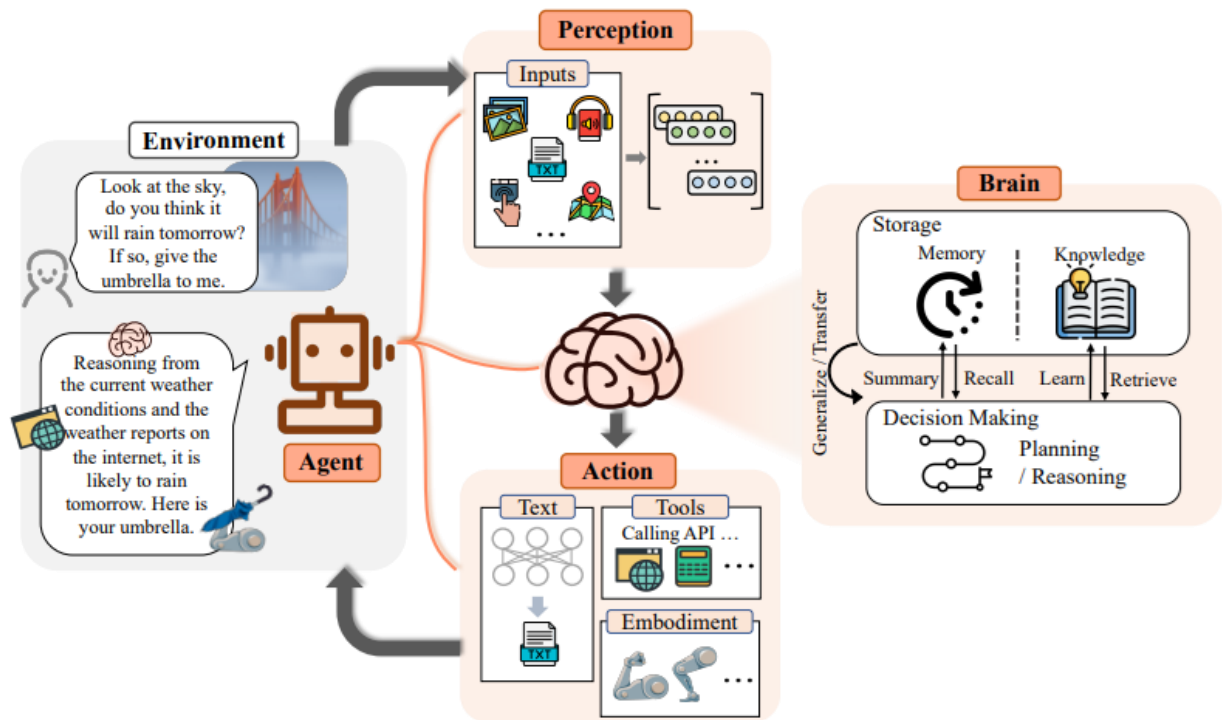


Figure 2: Conceptual framework of LLM-based agent with three components: brain, perception, and action. Serving as the controller, the brain module undertakes basic tasks like memorizing, thinking, and decision-making. The perception module perceives and processes multimodal information

LLM的新方向

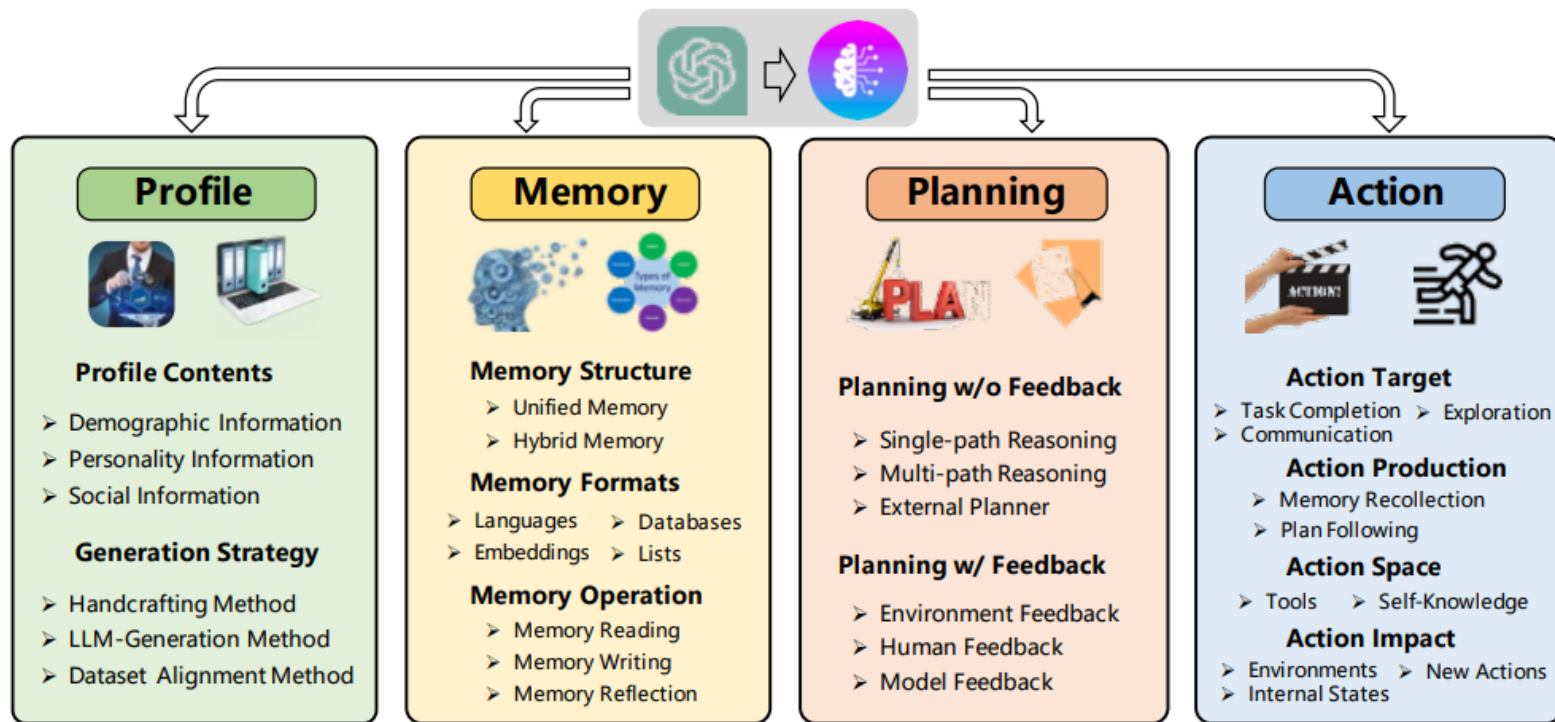


Fig. 2 A unified framework for the architecture design of LLM-based autonomous agent.

LLM的新方向

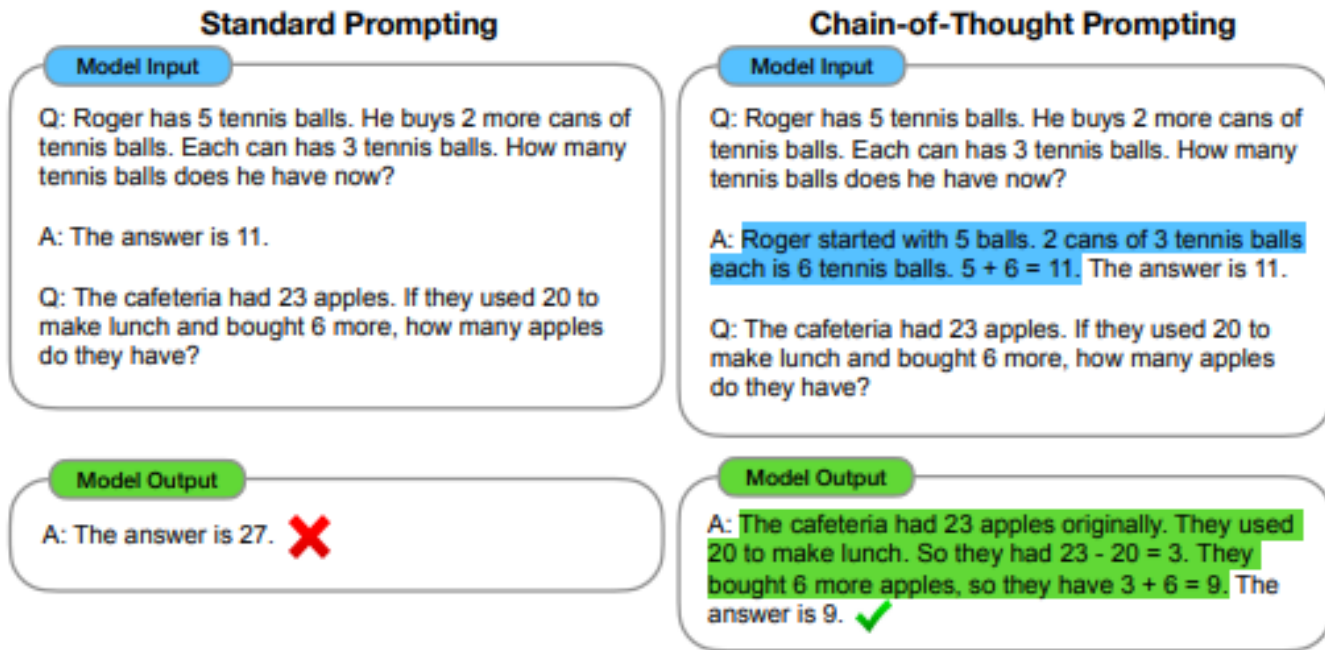


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

LLM的新方向

規劃 (Planning)

記憶 (Memory)

工具呼叫 (Tools)

執行行動 (Action)

迴圈 (Loop)

圖 (Graph)

LLM的新方向

史丹佛與 Google 在 2023 年發表、啟發開源社群《虛擬小鎮 Smallville》

<https://rikiphukon.medium.com/stanford-smallville-is-officially-open-source-9882e3fbc981>

https://reverie.herokuapp.com/arXiv_Demo/

https://github.com/joonspk-research/generative_agents



25 個基於
ChatGPT 的 AI
代理人

獨立的人格與背
景故事

複雜的湧現行為

多智能體技術

Figure 1: Generative agents are believable simulacra of human behavior for interactive applications. In this work, we demonstrate generative agents by populating a sandbox environment, reminiscent of The Sims, with twenty-five agents. Users can observe and intervene as agents plan their days, share news, form relationships, and coordinate group activities.

LLM的新方向

More Agents Is All You Need

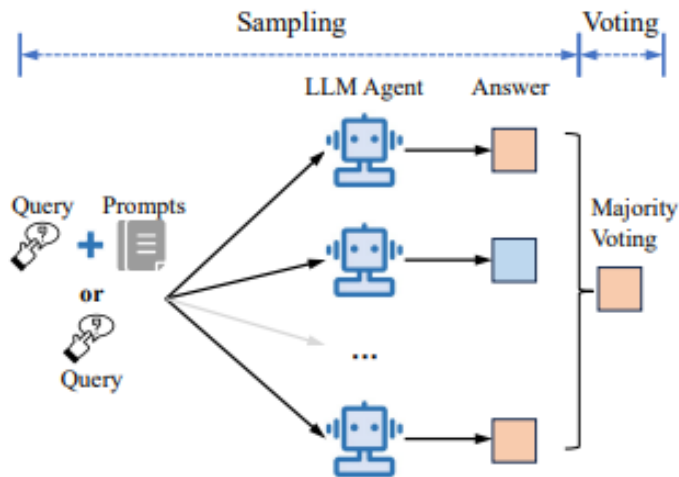


Figure 2: Illustration of Agent Forest. The two-phase process begins by feeding the task query, either alone or combined with prompt engineering methods, into LLM agents to generate answers. Subsequently, majority voting is applied to these answers to determine the final answer. Specifically, an LLM agent refers to a single LLM or a multiple LLM-Agents collaboration framework.

「採樣與投票
(sampling-and-voting)」方法

隨著 Agent 數量
增加會出現「邊
際遞減效應」

性能提升的程度與任務的
難易度正相關，且該方法
與目前其他複雜的 LLM
增強手段是正交的

此方法用於生成訓練數據，
透過多個衍生版本的投票
結果來進行 DPO

震撼的結論：三個臭皮匠，
勝過一個諸葛亮

LLM的新方向

ReAct: Synergizing Reasoning and Acting in Language Models

ReAct 是一種將「推理」(Reasoning) 與「行動」(Acting) 相結合的 LLM 提示方法

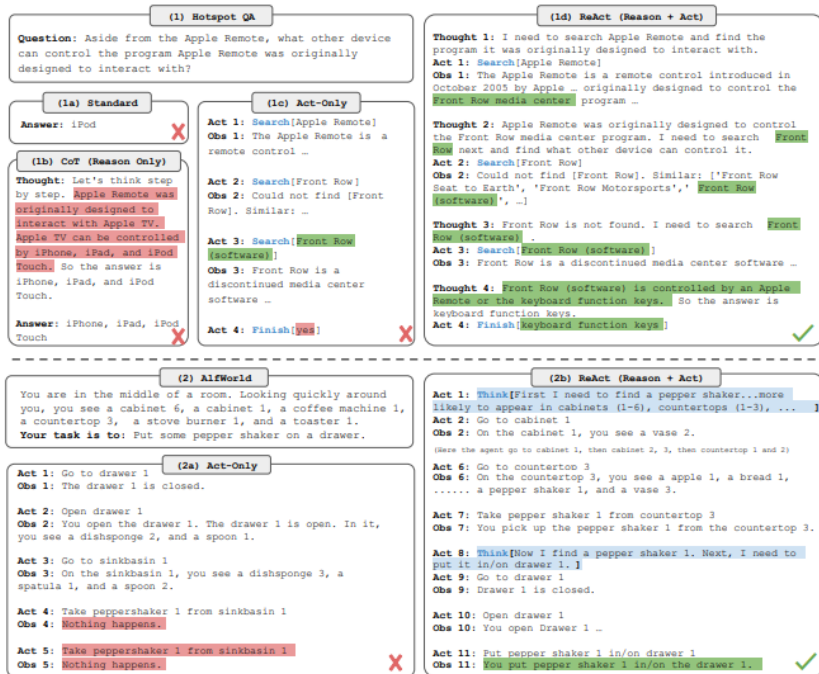


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotspotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

傳統的思維鏈 (Chain-of-Thought) 容易出現幻覺 (Hallucination) 和錯誤傳播，ReAct 透過與外部環境 (如 Wikipedia API) 互動來獲取即時資訊以克服此問題

單純的操作計畫缺乏靈活性，ReAct 的推理過程能幫助模型誘導、追蹤並更新行動計畫，且能更有效地處理例外狀況

ReAct 會生成可讀的推理軌跡，其解決問題的過程比沒有推理步驟的方法更具備「人類可解釋性」(Interpretability) 與「可信賴度」(Trustworthiness)

ReAct 的核心貢獻，就是將「推理 (Reason)」與「行動 (Act)」交織 (Interleave) 在一起，創造出完美的 Synergistic Loop (協同循環)

LLM的新方向

ReAct: Synergizing Reasoning and Acting in Language Models

Prompt Method ^a	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC → ReAct	34.2	64.6
ReAct → CoT-SC	35.1	62.0
Supervised SoTA ^b	67.5	89.5

Table 1: PaLM-540B prompting results on HotpotQA and Fever.

^a HotpotQA EM is 27.1, 28.9, 33.8 for Standard, CoT, CoT-SC in Wang et al. (2022b).

^b (Zhu et al., 2021; Lewis et al., 2020)

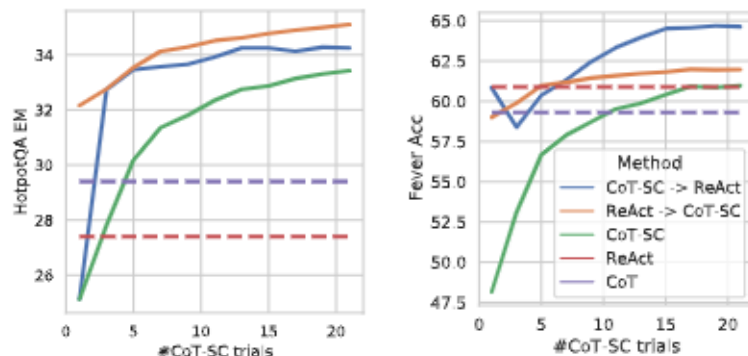


Figure 2: PaLM-540B prompting results with respect to number of CoT-SC samples used.

LLM的新方向

ReAct: Synergizing Reasoning and Acting in Language Models

- ReAct 的想法非常直覺和簡單：我們不是只讓智能體做出「實際行動」，而是擴充它的「行動空間」。新的行動空間會包含原本的實體行動 A 加上一個全新的組件 L，也就是語言空間（Language Space）。我們在這個語言空間 L 裡增加的一種行動，我們稱之為「思維」（Thought）或「推理軌跡」（Reasoning Trace）。
- 「思考」的特性：不影響環境：當智能體產生一個「思考」(Thought) 時，它只是在內部進行計算和想法構思，不會對外部環境造成任何實質改變，因此它也不會收到來自環境的回饋觀察（Observation Feedback）。
 - 目的：這個「思考」的主要目的，是讓智能體能夠根據當前的上下文，主動地組合出一些有用的資訊。然後，它會用這個新資訊來更新下一個上下文，為後續的思考或行動做準備。

LLM的新方向

ReAct: Synergizing Reasoning and Acting in Language Models

- 透過「思考」步驟，智能體可以做很多有幫助的事情：
 - 分解任務目標和制定計畫：把一個大任務拆解成小步驟
 - 注入常識知識：根據實務經驗補充背景知識，幫助理解任務
 - 從觀察結果中提取重點資訊：篩選掉雜訊，只抓住關鍵點
 - 追蹤進度並調整計畫：了解自己目前執行到哪一步了
 - 處理意外情況和修正行動：當遇到問題時，能主動停下來反思並修改計畫

LLM的新方向

ReAct: Synergizing Reasoning and Acting in Language Models

- ReAct 帶來的好處
 - 因為將決策制定 (Decision Making) 的能力和推理能力整合到一個 LLM 中，ReAct 體現了以下幾個優點：
 - A) 直觀易設計 (Intuitive and Easy to Design)：設計 ReAct 的提示詞 (Prompt) 非常簡單。人類的資料標註者只需要像寫自白稿一樣，在實際採取的行動之上，寫下他們「正在思考什麼」。本文沒有使用任何固定的、僵硬的格式要求或特殊的範例選擇方式。
 - B) 通用且靈活 (General and Flexible)：由於它有彈性的「思考」空間和可變化的「思維-行動發生時機」，ReAct 可以應用到很多不同的任務場景，例如：問答 (QA)、事實查證、文字遊戲、網頁導航等，只要這些任務的「行動空間」和推理需求不同，它就能適用。
 - C) 性能強大且穩定 (Performant and Robust)：僅從少量（一到六個）的範例學習，ReAct 在新的任務情境中展現出很强的泛化能力。在不同的領域測試時，它的表現一直優於那些只學推理或只學行動的基線模型。
 - D) 人為對齊且可控 (Human Aligned and Controllable)：ReAct 最大的賣點之一是它提供了一個「可解釋的、循序漸進的決策與推理過程」。人類可以清晰地看到智能體是怎麼想的，以及它的判斷是否符合事實。更進一步的是，使用者甚至可以在思考過程中進行干預或修正（例如：用「修改思維」的方式來糾正它），這極大地增強了系統的可控性。

基礎環境(地端)

地端環境

- Anaconda or Miniconda
- Ollama
- Open Webui
 - python 3.11
- Models
 - Gemma4, GPT-OSS, BGE, e5(Embedded)
- `conda create -n gptcode python=3.11 -y`

地端環境

Item	vLLM	Ollama
核心定位	高性能 LLM 推理引擎	本地模型管理與服務工具
主要優勢	極致的推理速度、高吞吐量（基於 PagedAttention）	極致的易用性（一條命令搞定下載、運行、API）
運行機制	需要用戶指定模型和參數，程式碼控制複雜	抽象化層，用戶只需輸入模型名即可運行
性能表現	極佳（業界頂尖），特別適合高並發、批次處理	良好，適合個人桌面使用，但針對極致性能的優化不如 vLLM
部署複雜度	中等到高。需要了解 GPU 資源和運行模式（腳本或 API 伺服器）	極低。通常只需要一行命令 <code>ollama run <model></code>
模型生態	需使用 Hugging Face 結構的模型（如 meta-llama/...） PyTorch / HF 模型 (FP16/BF16)	建立了自己的模型庫，易於管理不同來源的模型
適用場景	企業級/生產環境、需要最大化 GPU 利用率的 API 服務	個人開發、測試、本地原型驗證，追求快速啟動
核心技術	PagedAttention, Continuous Batching	Docker/容器化、簡單的 API 封裝

地端環境

Local API

編輯預設權限

🔧 權限

允許語言轉文字



允許文字轉語言



允許通話



允許在對話中使用多個模型



允許臨時對話



強制使用臨時對話



功能權限

API 金鑰



筆記



頻道



分組



直連工具伺服器



網頁搜尋



圖片生成



程式碼直譯器



記憶



...



儲存

地端環境

使用者 分析 評估 函式

設定

Q 搜尋

⚙️ 一般

🔌 連線

📄 模型

📄 評估

🔧 外接功能

📄 文件

🌐 網頁搜尋

💻 程式碼執行

🖥️ 介面

🔊 音訊

🖼️ 圖片

📶 管線

🗄️ 資料庫

一般

版本

v0.9.6 (最新版)

[檢視新功能](#)

說明

探索如何使用 Open WebUI 並從社群尋求支援。

[Discord](#) [Open WebUI](#)

[Follow @OpenWebUI](#)

[Star us on Github](#)

Unable to select next GitHub token from pool

授權

升級至授權方案以獲得更強大功能，包括客製化主題與品牌，和專屬支援。

驗證

預設使用者角色

待處理 ▾

預設群組

None ▾

允許新使用者註冊

☐

在帳號待客覆蓋層中顯示管理員詳細資訊

☒

管理員聯絡信箱

留空以使用第一個管理員使用者

待處理的使用者訊息覆蓋層標題

為待處理的使用者訊息覆蓋層輸入標題。留空以使用預設值。

待處理的使用者訊息覆蓋層內容

為待處理的使用者訊息覆蓋層輸入內容。留空以使用預設值。

啟用 API 金鑰

☒

API 金鑰端點限制

☒

允許的端點

/api/v1,/api/chat/completions

[若要進一步了解可用的端點，請參閱我們的說明文件。](#)

JWT 過期時間

4w

Local API

[說明檔案](#)

地端環境

Local API

設定

Q 搜尋

⚙️ 一般

🗨️ 介面

🔌 外掛功能

😊 個人化

🔊 音訊

📁 資料

👤 帳號

① 關於

您的帳號

管理您的帳號資訊。

J

名稱

jd

個人簡介

分享您的背景與興趣

性別

不想透露

生日

年 / 月 / 日

修改密碼

顯示

API 金鑰

隱藏

JWT Token

..... 🔍 📄

API 金鑰

+ 建立新的金鑰

Ex.地端環境

- 測試一下
 - 透過 Openwebui 取用ollama管理的model

小結一下不同的呼叫方式

	requests 直連 Ollama	openai SDK 直連 Ollama	openai SDK + Open WebUI 轉發
網路路徑 (URL)	http://127.0.0.1:11434/v1/chat/completions	http://127.0.0.1:11434/v1	http://172.10.0.22:8080/api
API 金鑰 (API Key)	不需要	不需要 (程式隨便填即可)	必須提供 (需在 Open WebUI 產生的真實 sk-...)
連線情境	本機開發 (或同台伺服器環境)	本機開發 (業界標準寫法)	跨網路/企業環境 (Python 在外網, LLM 在 Docker 內)
依賴套件	僅需 requests	需安裝 openai	需安裝 openai
資料結構解析	手動解析字典： result["choices"][0]...	物件化操作： response.choices[0]...	物件化操作： response.choices[0]...
管理功能	無 (直接由 Ollama 處理)	無 (直接由 Ollama 處理)	有 (對話會被記錄在 Open WebUI 介面上, 受權限控管)

小結一下不同的呼叫方式

- 我該選哪一個？
 - 【寫法一 (requests)】：適合極度追求輕量化、不想安裝任何大型第三方套件，或是正在寫簡單自動化 Shell 腳本的時刻。
 - 【寫法二 (openai 直連)】：適合本機純粹的開發與測試。你想快速驗證某個 Prompt 或 AI 功能，且 Ollama 就裝在你自己的實體電腦裡。
 - 【寫法三 (openai + Open WebUI)】：這是最接近「企業生產環境 (Production)」的寫法。當 AI 模型架設在遠端伺服器（或 Docker 叢集內），而你從外部開發機要連接，並需要透過 WebUI 來監控 Token 消耗量、聊天歷史紀錄、或做使用者權限管理時，【寫法三】是無庸置疑的最佳解。

AI Agent 幾個小專案

Ex. 1.2.1

lab / minimal-agent /

**xixiaofinland** and **claude** feat: add write_file tool and post-run validation check

Name

 ..

 .er

 AC

 RE

 agent.ts

 package-lock.json

 package.json

 tsconfig.json



xixiaofinland Xi Xiao

Salesforce developer exploring FP world

 Finland

 Owns this repository

 Committed to this repository

Follow

Last commit message

feat: minimal agent loop, no tools, openai-compatible

feat: load AGENTS.md as system message for persistent memory

feat: minimal agent loop, no tools, openai-compatible

feat: add write_file tool and post-run validation check

feat: minimal agent loop, no tools, openai-compatible

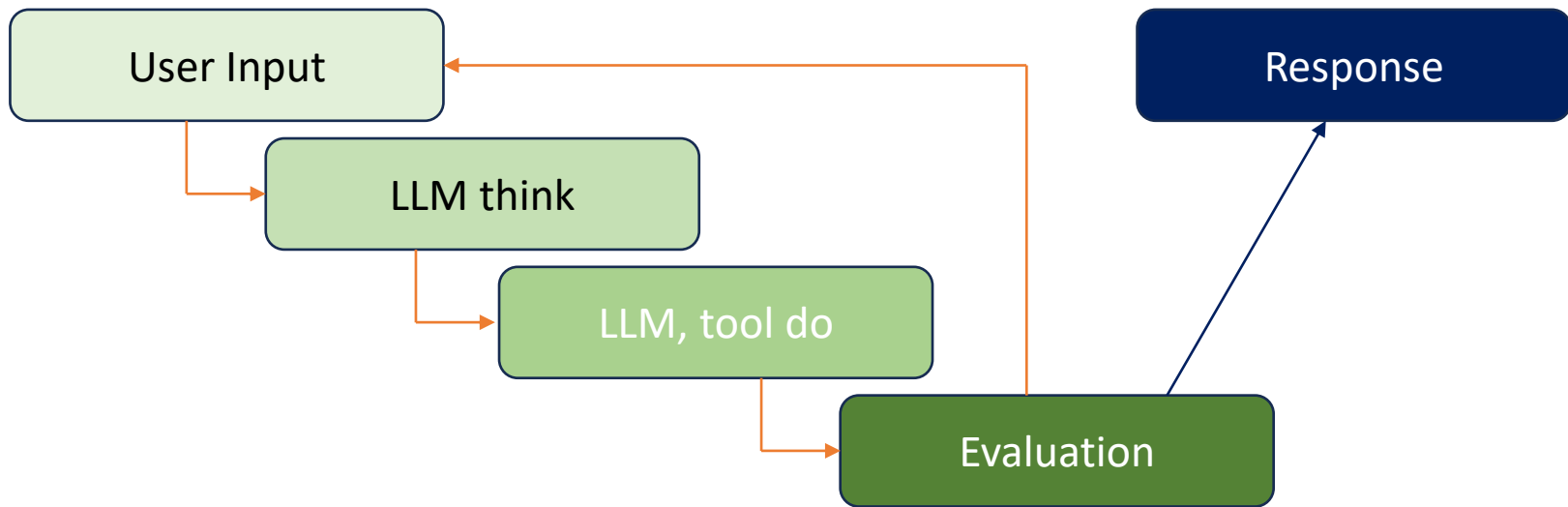
feat: minimal agent loop, no tools, openai-compatible

feat: minimal agent loop, no tools, openai-compatible

README.md

minimal-agent

Ex. 1.2.1



Ex. 1.2.1

- https://github.com/meta-llama/llama-cookbook/blob/main/end-to-end-use-cases/agents/Agents_Tutorial/Tool_Calling_101.ipynb

這支程式展示了如何不依賴 LangChain 等複雜框架，僅用純 Python、正規表達式 (Regex) 與本地 Ollama / Open WebUI (API: sk-abc123)，無縫實現 AI Agent 的工具閉環。

[使用者提問] → (正確的 System Prompt 啟動) → [本地 Llama 模型]
| (決策：輸出特定格式)
[最終精準解答] ← [後端 Python 函式] (執行/撈DB) ← v [Regex 攔截器]

Ex. 1.2.1

Install and setup groq dependencies

- Install `groq` api to access Llama model(s)
- Configure our client and authenticate with API Key(s), Note: PLEASE UPDATE YOUR KEY BELOW

```
In [ ]: #!/pip3 install groq  
%set_env GROQ_API_KEY=''
```

```
In [ ]: import os  
from groq import Groq  
# Create the Groq client  
client = Groq(api_key='')
```

Ex. 1.2.1

System Prompt

Chat History

```
In [5]: SYSTEM_PROMPT = """
Cutting Knowledge Date: December 2023
Today Date: 20 August 2024

You are a helpful assistant
"""
```

```
In [6]: system_prompt = {}
chat_history = []

def model_chat(user_input: str, sys_prompt = SYSTEM_PROMPT, temperature: int = 0.7, max_tokens=2048):

    chat_history = [
        {
            "role": "system",
            "content": sys_prompt
        }
    ]

    chat_history.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(model="llama-3.3-70b-versatile",
                                              messages=chat_history,
                                              max_tokens=max_tokens,
                                              temperature=temperature)

    chat_history.append({
        "role": "assistant",
        "content": response.choices[0].message.content
    })

    #print("Assistant:", response.choices[0].message.content)

    return response.choices[0].message.content
```

Ex. 1.2.1

User Prompt

model_chat()

In []:

```
user_input = """
When is the next Elden Ring game coming out?
"""

print("Assistant:", model_chat(user_input, sys_prompt=SYSTEM_PROMPT))
```

Assistant: As of my knowledge cutoff in December 2023, there has been no official announcement from FromSoftware, the developers of the Elden Ring series, regarding a release date for a new Elden Ring game.

However, it's worth noting that FromSoftware has mentioned that they are working on new projects, and there have been rumors and speculation about a potential Elden Ring sequel or DLC. But until an official announcement is made, we can't confirm any details about a new Elden Ring game.

If you're eager for more Elden Ring content, you can keep an eye on the official Elden Ring website, social media channels, and gaming news outlets for any updates or announcements. I'll be happy to help you stay informed if any new information becomes available!

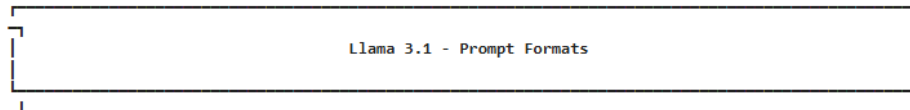
Ex. 1.2.1

Special Token

|python_tag|

In []:

```
SYST
Envir
Tool
Cutt
Toda
"""
user
When
"""
prio
Assist
```



Toc

Tokens

Here is a list of special tokens that are supported by Llama 3.1:

With

If even

follow

This

Time

- `<|begin_of_text|>`: Specifies the start of the prompt
- `<|end_of_text|>`: Model will cease to generate more tokens. This token is generated only by the base models.
- `<|finetune_right_pad_id|>`: This token is used for padding text sequences to the same length in a batch.
- `<|start_header_id|>` and `<|end_header_id|>`: These tokens enclose the role for a particular message. The possible roles are: [system, user, assistant and ipython]
- `<|eom_id|>`: End of message. A message represents a possible stopping point for execution where the model can inform the executor that a tool call needs to be made. This is used for multi-step interactions between the model and any available tools. This token is emitted by the model when the Environment: ipython instruction is used in the system prompt, or if the model calls for a built-in tool.
- `<|eot_id|>`: End of turn. Represents when the model has determined that it has finished interacting with the user message that initiated its response. This is used in two scenarios:
 - at the end of a direct interaction between the model and the user
 - at the end of multiple interactions between the model and any available tools This token signals to the executor that the model has finished generating a response.
- `<|python_tag|>`: Is a special tag used in the model's response to signify a tool call.

There are 4 different roles that are supported by Llama 3.1

- `system`: Sets the context in which to interact with the AI model. It typically includes rules, guidelines, or necessary information that helps the model respond effectively.
- `user`: Represents the human interacting with the model. It includes the inputs, commands, and questions to the model.
- `ipython`: A new role introduced in Llama 3.1. Semantically, this role means "tool". This role is used to mark messages with the output of a tool call when sent back to the model from the executor.
- `assistant`: Represents the response generated by the AI model based on the context provided in the `system`, `ipython` and `user` prompts.

Ex. 1.2.1

- Meta 官方文檔 Special Tokens
 - 給架構師、推論引擎開發者、或是要做微調（Fine-tuning）的人使用
 - 一般的應用開發者
 - 不需要手動拼接 `<|python_tag|>`
 - 不需要手動判斷 `<|eom_idl|>`
 - 只需要向主流引擎傳遞 `tools` 參數，後台就會自動幫您翻譯成 Llama 看得懂的格式
- 現在開發 Agent 已經非常方便了

Ex. API Calls

- 透過 Ollama，讓本機的 Python 程式利用標準的 API 格式呼叫它執行 Agent 的工具
- `ex_1_2_calling_ollama_v1.py`
- `ex_1_2_calling_api_v1.py`

Ex. 1.2.2

- <https://medium.com/repro-repo/local-llm-agents-autogen-and-llama-3-ff6588857b76>



您好，請問你是誰？



我是一個友善的人工智慧系統，但為了這次的對話情境設定，我們可以假裝我是您的同學或是輔導員！因為我們處於一個虛擬教室的模擬環境中，您可以稱呼我為「教授」或直接叫「老師」。請問您今天希望討論哪個主題呢？關於目前學科內容，您是否有任何疑問需要我協助解答，或者有什麼特別想了解的地方嗎？

太好了！我們的日常課堂時間到囉！老師，關於我們昨天上課的那些概念，其實我…

那我們就開始…

```
from autogen import AssistantAgent

# Setup API key. Add your own API key to config file or environment variable
config_list = [
    {
        "model": "llama3",
        "base_url": "http://localhost:11434/v1",
        'api_key': 'ollama',
    },
]

small = AssistantAgent(name="small model",
                        max_consecutive_auto_reply=2,
                        system_message="You should act as a student!",
                        llm_config={
                            "config_list": config_list,
                            "temperature": 1,
                        })

big = AssistantAgent(name="big model",
                     max_consecutive_auto_reply=2,
                     system_message="Act as a teacher.",
                     llm_config={
                         "config_list": config_list,
                         "temperature": 1,
                     })
```

Ex. Novel Creator

- 實作 Local LLM Agents with AutoGen and Gemma4 程式碼展示如何初始化一個 AssistantAgent（如學生角色）與另一個評審 Agent，讓兩個本機小模型互相對話、糾錯、反思並完成特定程式任務
- `ex_novel_multi_agent_sys_topic_finer.py`

Ex. 1.2.3

- <https://docs.langchain.com/oss/python/langchain/sql-agent>
- LangChain 官方提供完全基於 Ollama 本地模型實作 SQL Agent

```
from langchain_ollama import ChatOllama
from langchain_community.utilities import SQLDatabase
from langchain_community.agent_toolkits import create_sql_agent

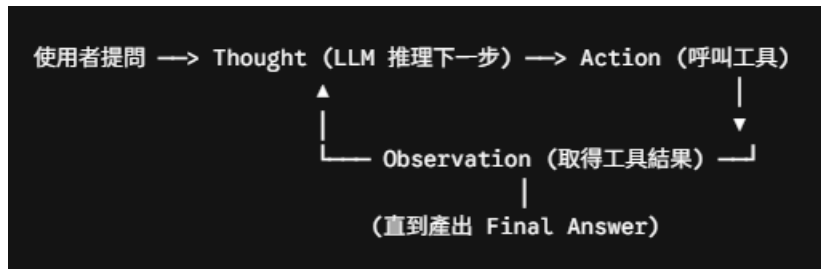
# 1. 連結本地的資料庫 (以一個 SQLite 檔案為例)
db = SQLDatabase.from_uri("sqlite:///company_sales.db")

# 2. 宣告大腦：使用 Ollama 本地運行的 Llama 3.1 模型
llm = ChatOllama(model="llama3.1", temperature=0)

# 3. 建立 SQL Agent (LangChain 會自動把資料庫結構當作「工具」餵給模型)
agent_executor = create_sql_agent(llm, db=db, verbose=True)

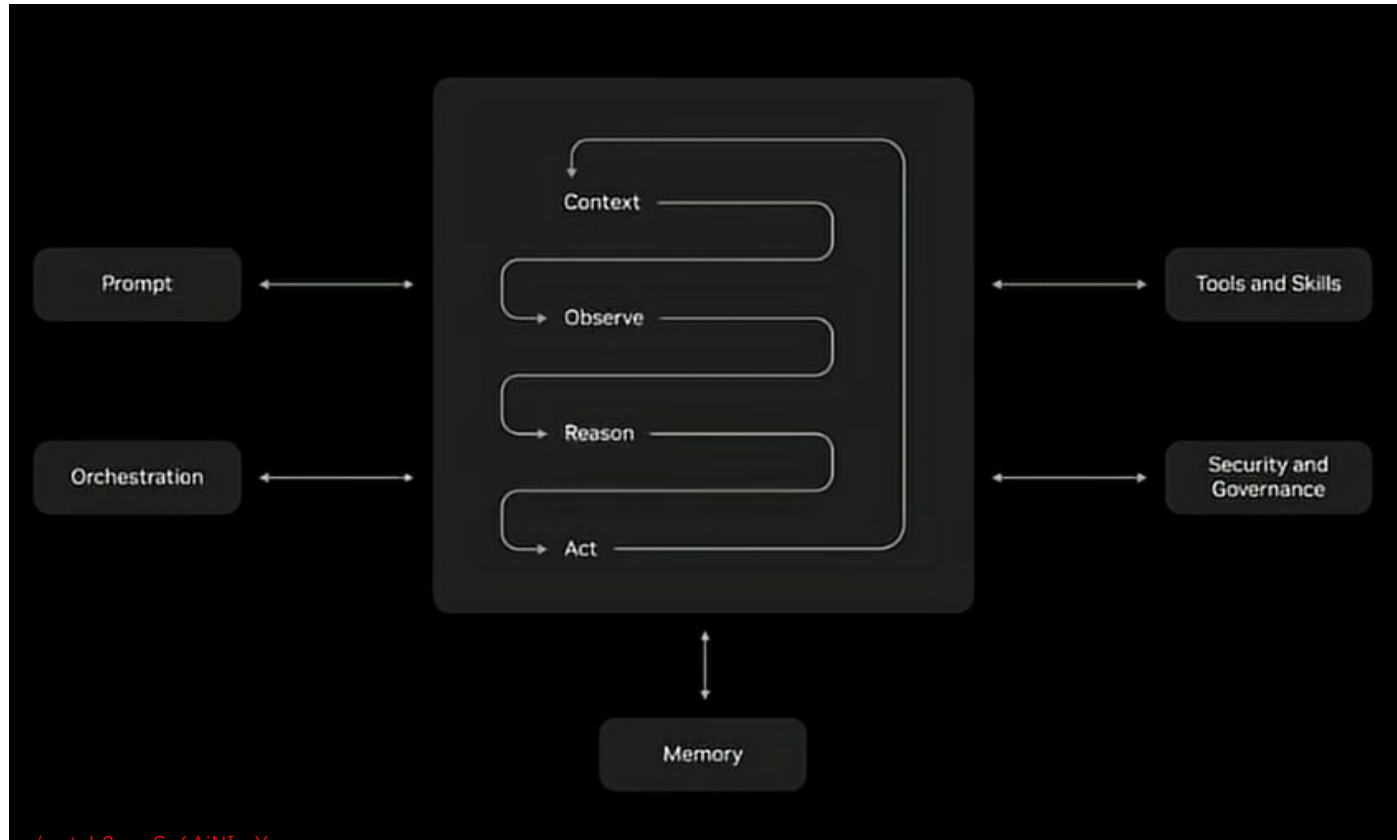
# 4. 提出問題，讓 Agent 開始自主思考並查表
agent_executor.invoke({"input": "幫我找出去年在台北市消費最高的前三名客戶是誰?"})
```

Ex. SQL call example



- 不背答案：小模型（如 Gemma4）的訓練資料裡絕對不可能有自己的私有資料。它是靠現場學習資料庫結構、現場推理來解決問題
- 具備容錯與修正能力：在實際運作中，如果 LLM 第一遍寫的 SQL 語法有語法錯誤（Syntax Error），本地資料庫會報錯回傳給它。Agent 看到錯誤訊息後，會進行自我反思（Self-Reflection），重新修改 SQL 再執行一次，直到成功或達安全上限為止
- `ex_sql_call_example.py`

What agent sounds like...



What agent sounds like...

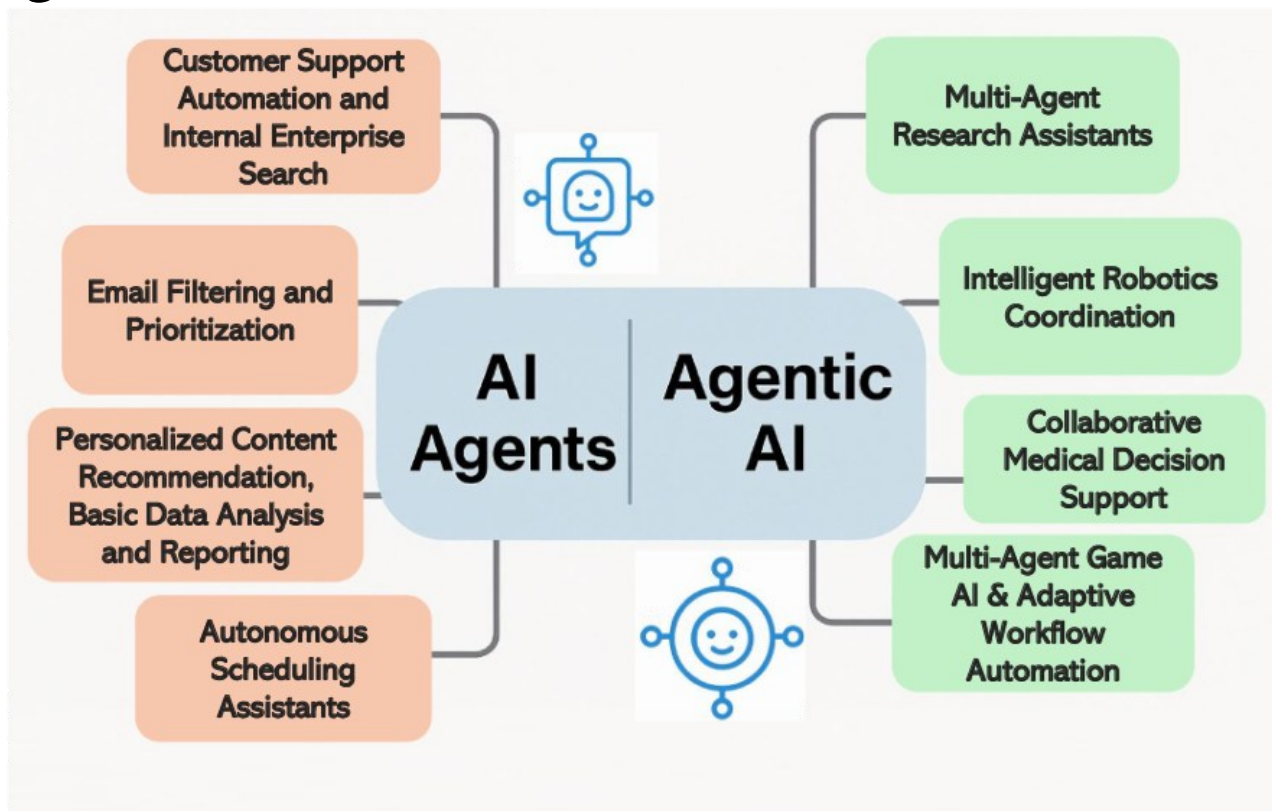
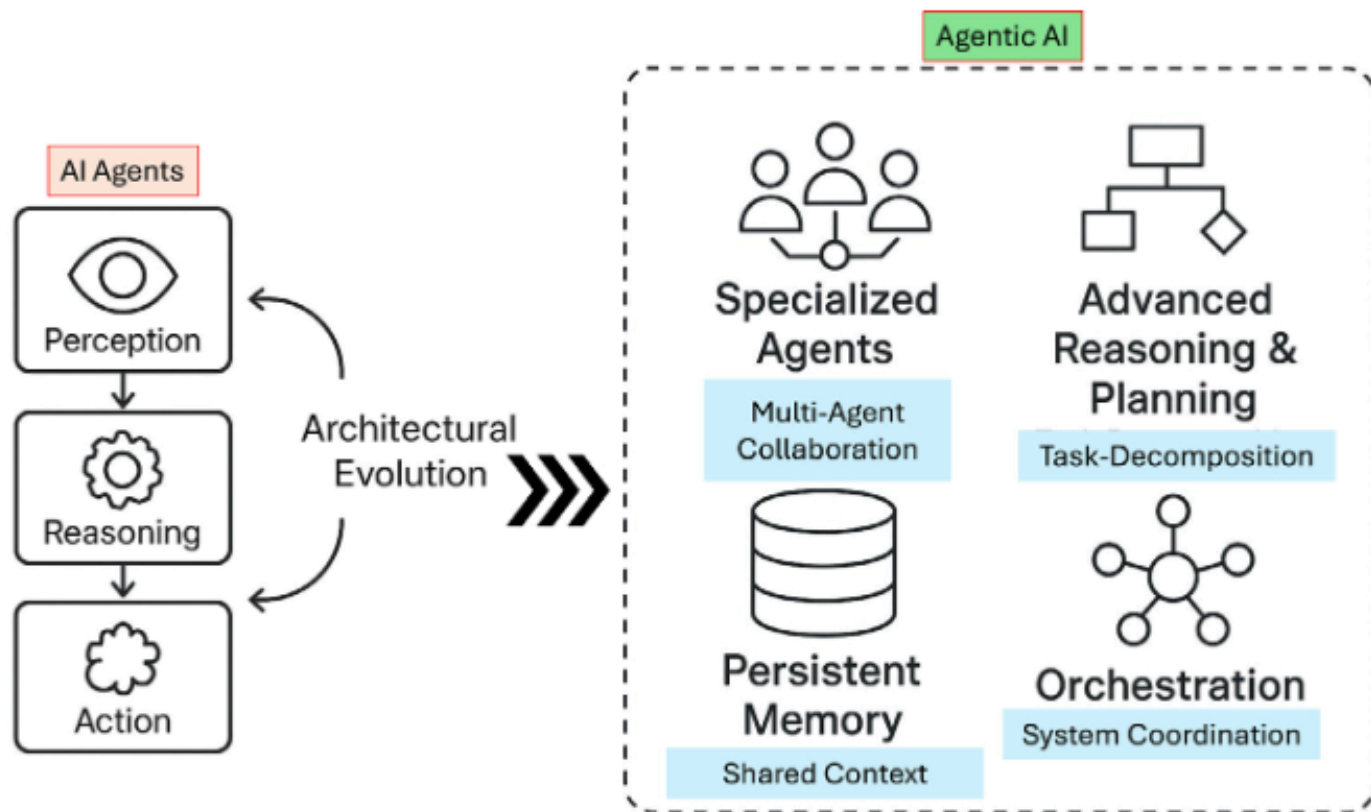


Fig. 9. Categorized applications of AI Agents and Agentic AI across eight core functional domains.

What agent sounds like...



What agent sounds like...

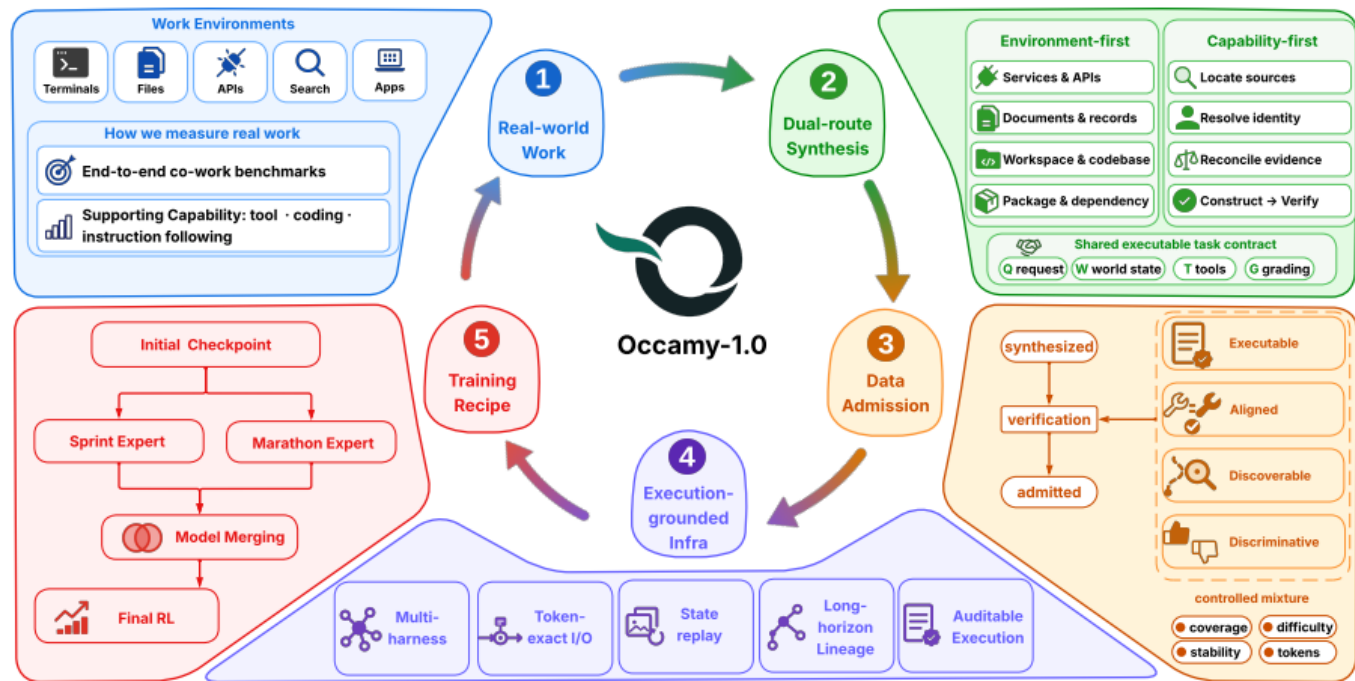


Figure 1: Overview of Occamy-1.0. The system connects real-work task construction, verified data admission, replayable multi-harness execution, and staged post-training.

What agent sounds like...

- Occamy-1.0 的高效能 AI 模型

- 專為處理複雜的「協作工作流」（如資訊收集、工具調用、寫程式及檔案操作）而設計。
- 研究團隊發現，許多日常工作其實不需要每次都動用最強大的推理能力，而是更依賴狀態追蹤與執行力。
- 基於 Qwen3.6-35B 進行後訓練，透過構建真實的執行環境與長路徑數據來強化其能力。
- 結果顯示，Occamy-1.0 在維持低成本與低延遲的同時，性能能與規模大得多的頂尖模型競爭，達到了「成本與性能」的最佳平衡點（帕累托前沿），在不犧牲通用能力的條件下，極大地提升了 AI 代理人在實際工作中的實用價值。

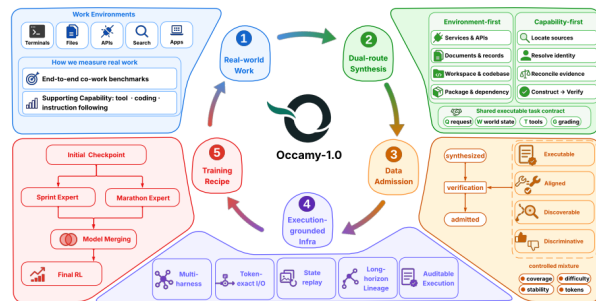
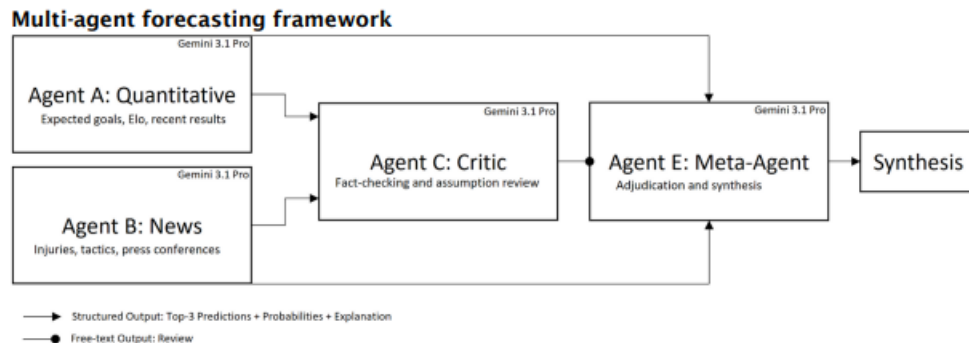


Figure 1: Overview of Occamy-1.0. The system connects real-world task construction, verified data admission, replayable multi-harness execution, and staged post-training.

What agent sounds like...

- 在多代理人系統中，讓 AI 分擔「不同專長」是否能提升預測準確率。
- 針對 2026 年世界盃的比賽，設計了兩種專家 AI：一個專攻「數據統計」，另一個專攻「最新新聞（如傷病、戰術）」，最後由評論員與整合代理人彙整結果。
- 在多代理人系統中，讓 AI 分擔「不同專長」是否能提升預測準確率。研究團隊針對 2026 年世界盃的比賽，設計了兩種專家 AI：一個專攻「數據統計」，另一個專攻「最新新聞（如傷病、戰術）」，最後由評論員與整合代理人彙整結果。
- 結果發現：
 - 1. 追蹤即時新聞的 AI 表現最出色，甚至能與專業博弈市場的預測相媲美。
 - 2. 儘管分工明確，但兩位專家 AI 的看法其實高度一致。
 - 3. 增加後續的「評論」與「整合」階段並沒有顯著提升預測品質，反而僅是重複強勢專家的觀點。
- 簡言之：在處理高變動資訊時，即時非結構化數據（新聞）比單純的統計數據更有價值，但過多層級的 AI 協作並不一定能產生更好的結果。



What agent sounds like...

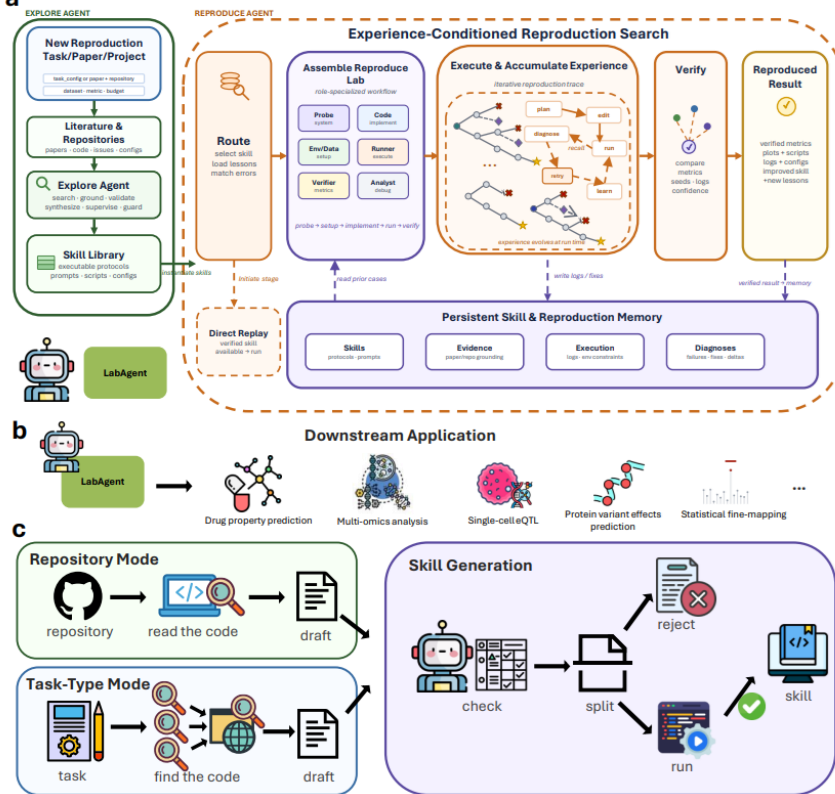


Figure 1: **The LabAgent framework.** (a) The framework of LabAgent. A target reaches the explore agent as a task configuration or as a paper with its repository, dataset, metric and budget. That agent searches the literature and code and deposits an executable skill in the library. The reproduce agent

What agent sounds like...

- 針對科學研究中「人員流動導致技術斷層」的問題
 - 例如學生畢業後，辛苦開發的實驗方法沒人能接手
 - 提出了一套名為 LabAgent 的 AI 系統
- 知識傳承與複製：讓 AI 學習並執行實驗室既有的操作技能，確保研究能被正確複現。
- 經驗記錄與修正：自動記錄錯誤及其修正過程，讓後續研究能直接避坑或快速修復。
- 實戰成效：在藥物屬性預測、生物醫學分析等生命科學領域中，LabAgent 的表現超越了市面上的通用 AI 代理，甚至能精準還原已發表的科研數據圖表。
- LabAgent 就像是一個「永不遺忘且能持續進化」的數位研究助理，將實驗室的碎片化知識系統化，讓科學探索能無縫接軌。

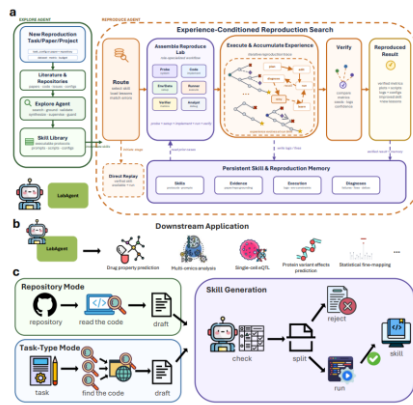


Figure 1: The LabAgent framework. (a) The framework of LabAgent. A target reaches the explore agent as a task configuration or as a paper with its repository, dataset, metric and budget. That agent reaches the Reproduction and Verify and deposits an executable skill in the library. The repository agent

What agent sounds like...

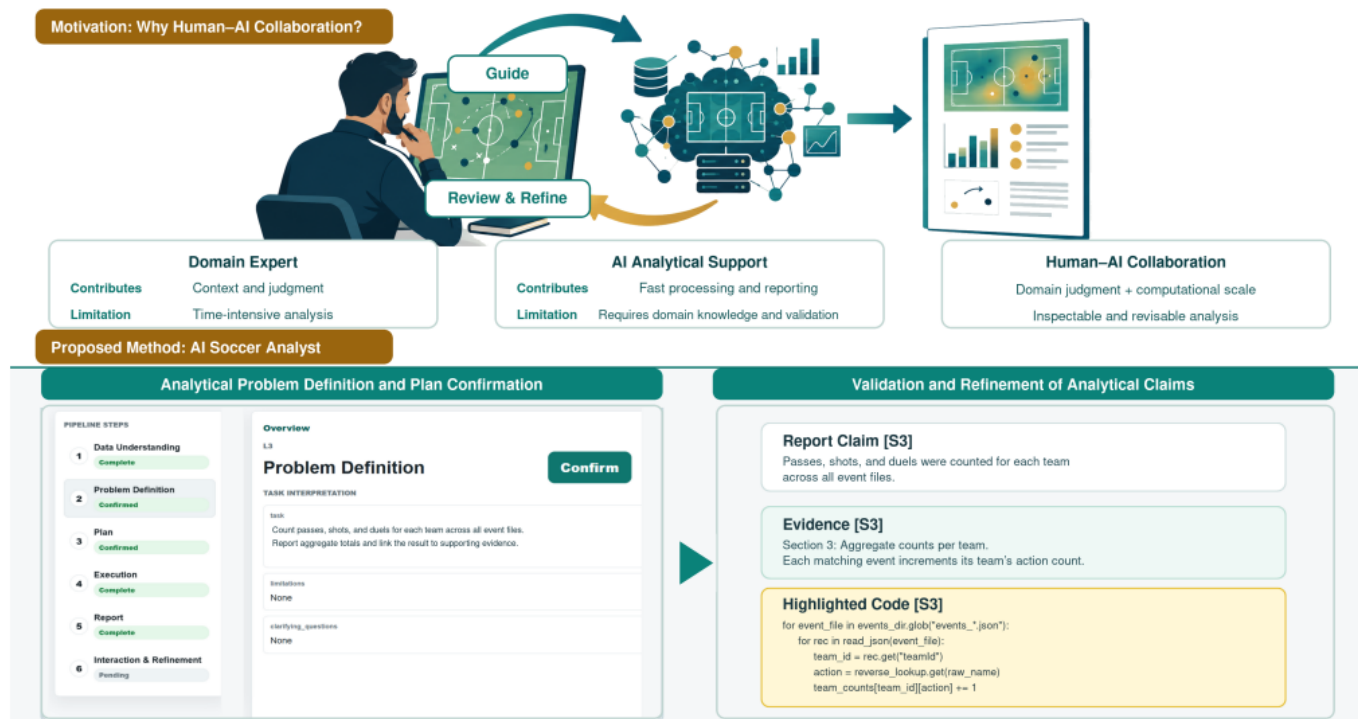


Figure 1: AI Soccer Analyst supports stage-aware human-AI collaboration in soccer data analysis. Domain experts contribute context and judgment, while AI provides scalable processing and reporting through an iterative guidance-and-review loop (top). The interface exposes two key control points (bottom): confirming the analytical problem and plan before execution, and inspecting linked evidence and relevant code to support claim validation and refinement.

What agent sounds like...

- 「AI Soccer Analyst」的協作系統
 - 旨在解決傳統 AI 分析過程中（如直接從指令跳到報告）缺乏透明度與證據的問題
- 該系統將分析流程拆解為六個可修正的階段：
 - 數據理解 → 問題定義 → 結構化規劃 → 執行 → 基於證據的報告 → 互動與精煉
- 透過讓人類專家在每個階段都能介入審查並調整，確保最終的分析結果不僅能自動化產出，且具備高度的可驗證性與可靠性，讓領域專家能掌控關鍵決策而非盲目信任 AI。

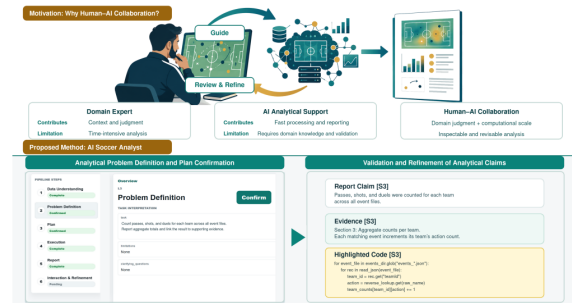


Figure 1: AI Soccer Analyst supports stage-aware human-AI collaboration in soccer data analysis. Domain experts contribute context and judgment, while AI provides scalable processing and reporting through an iterative guidance-and-review loop (top). The interface exposes two key control points (bottom): confirming the analytical problem and plan before execution, and inspecting linked evidence and relevant code to support claim validation and refinement.

What agent sounds like...

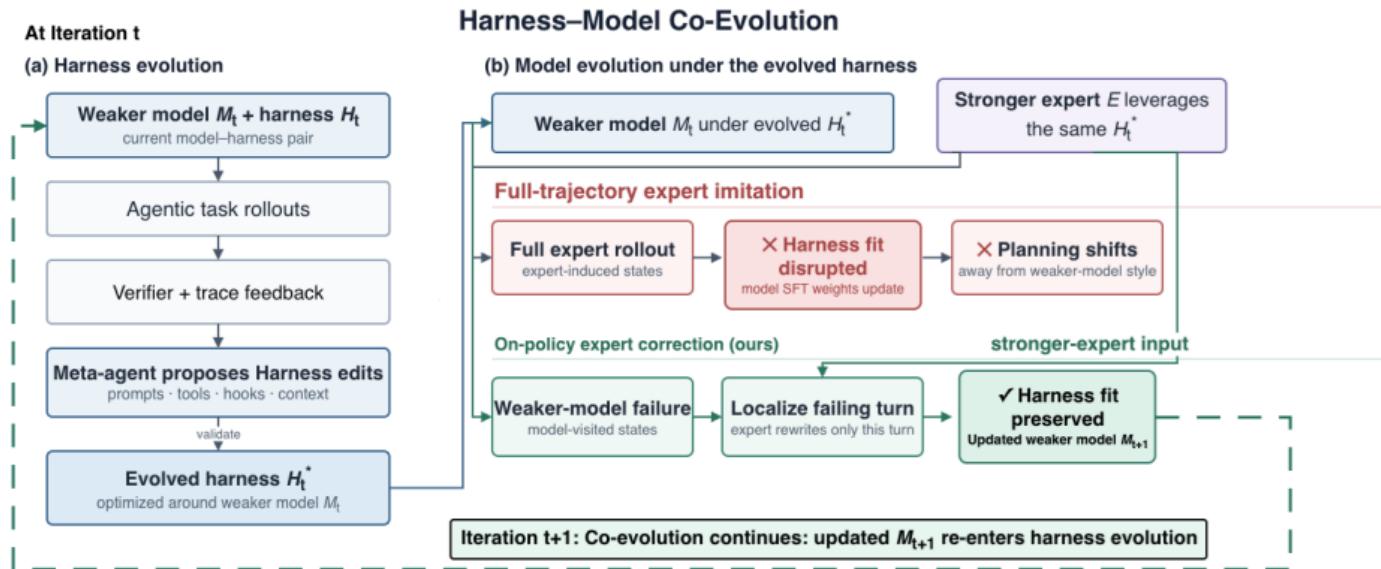


Figure 1: **Harness–model co-evolution.** Each round evolves a harness H_t^* around the current weaker model M_t and then updates the model under that harness. Full-trajectory expert imitation can disrupt the resulting model–harness fit, whereas on-policy expert correction uses the weaker model’s visited states together with localized corrections from the stronger expert, producing an updated model M_{t+1} while preserving the fit. The updated model can then re-enter harness evolution in the next round.

What agent sounds like...

- 探討如何讓小型 AI 模型（弱模型）在特定企業任務中達到與大模型（強模型）的表現。
- 僅靠「優化系統設定（Harness）」或單純「模仿強模型的完整路徑」效果不佳
 - 因為弱模型若盲目模仿強模型的規劃方式，反而會與原本適配它的系統設定產生衝突，導致性能下降
- 提出一種「在線策略修正」機制
 - 讓弱模型先嘗試執行，僅針對出錯的步驟請強模型進行局部修改
 - 既保留了弱模型原有的運作風格，又吸納了專家的精準修正
 - 成功實現了系統設定與模型能力的同步進化，讓經濟實惠的小模型也能高效完成複雜任務。

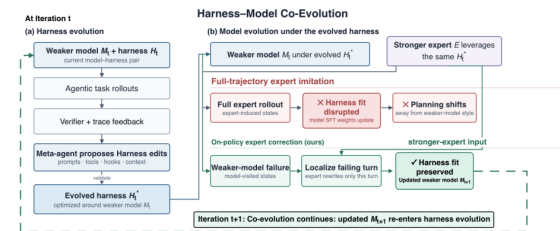


Figure 1: **Harness-model co-evolution.** Each round evolves a harness H_t^* around the current weaker model M_t , and then updates the model under that harness. Full-trajectory expert imitation can directly replace the model harness, but on-policy expert correction uses the weaker model's existing harness to generate localized expert patches. Localized expert patches produce an updated model M_{t+1} while preserving the fit. The updated model can then re-enter harness evolution in the next round.

工具應用與重構實踐

工具

Ex. 2.1.1

ex_2_function_calling

```
>python app.py 20200123
>>> current time_seed: [47]
```

=====

根據您提供的生日 ****2020年1月23日****，以及我查詢當下的時間點（假設為今天），您的命重計算結果如下：

```
* **您的出生日期 (Date of Birth):** 2020/01/23
* **目前的計算參考時間:** 2026/06/09 (請注意，這個「今天」的時間是系統執行查詢時的預設值)
* **命重數值 (Fate Weight):** 5.1
* **命格級別 (Level):** 昌盛之命
```

****【簡單解讀】****

您的命重數值為 ****5.1****，被判定為 ****「昌盛之命」****。

在不同的命理學體系中，「命重」的確切解讀方式會有所不同。但從「昌盛之命」這個描述來看，通常代表著：

1. ****發展順暢****：生活和運勢的基礎較好，容易給人留下積極、穩定的印象。
2. ****潛力優異****：具有向上發展的動力和能力。
3. ****建議方向****：適合透過學習、專業技能或穩定積累來發揮個人潛能。

****⚠ 重要提醒 ****

命理計算僅為一種文化娛樂和參考工具，不能作為人生的絕對指南。它提供的是一個從特定曆法角度出發的「趨勢分析」。請務必將此結果視為一種****參考建議****，實際的人生發展仍取決於您的努力、選擇與後天的積累！

如果您希望了解更深入的命理分析（例如針對性格、適合的行業等），建議您諮詢專業且信譽良好的命理師，並告知他們計算時應採用的具體系統或流派。

=====

Ex. 2.1.2

ex_2_function_calling

```
>python app2.py 20200123 台北
>>> msg: [ChatCompletionMessage(content='', refusal=None, role='assistant', annotations=None, audio=None, function_call=None, tool_calls=[ChatCompletionMessageToolCall(id='call_3ito85sf', function=Function(arguments='{"location": "\u53f0\u5317"}', name='get_weather'), type='function', index=0), ChatCompletionMessageToolCall(id='call_o9p9mu0v', function=Function(arguments='{"date_of_birth": "20200123"}', name='calculate_fate_weight'), type='function', index=1)]]
>>> 正在執行工具: get_weather, 參數: {'location': '台北'}
>>> 正在執行工具: calculate_fate_weight, 參數: {'date_of_birth': '20200123'}
>>> current time_seed: [45]
```

=====
您好，我為您查詢了台北的天氣和您的「命重」。

****🌧️ 今天台北天氣預報: ****

今天台北天氣不太好喔，可能會下陣雨，最高溫會達到攝氏 ****36 度****。建議您攜帶雨具出門。

****👤 您的命理資訊 (根據出生日期 2020/01/23): ****

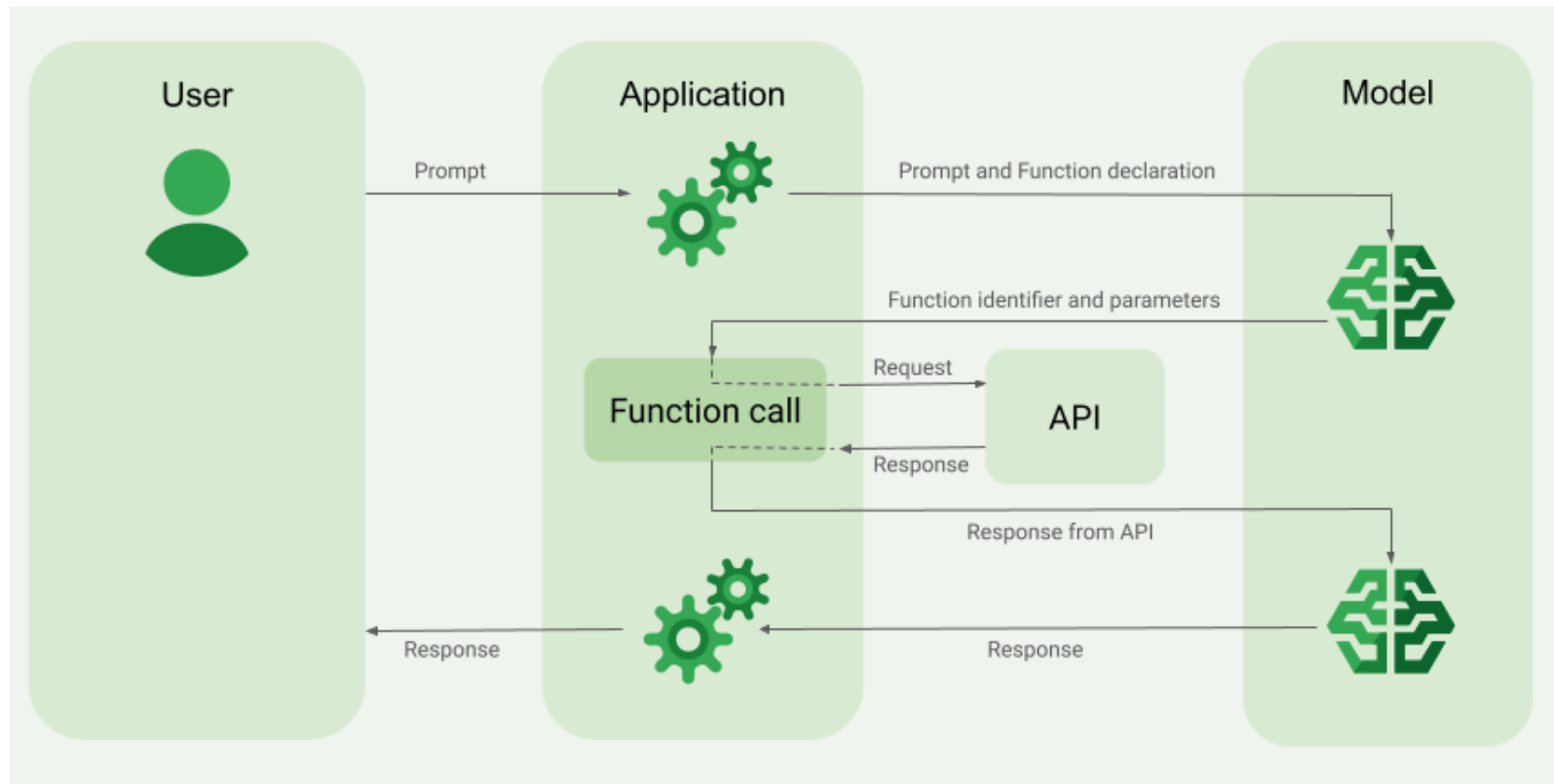
* ****您的命重: ** 2.5**

* ****等級判斷: ** 輕羽之命**

請注意，天氣預報是即時的參考資料，建議您多留意當地氣象局的最新公告。至於「命重」屬於一種傳統或個人的解讀方式，這僅根據您提供的資訊為模型計算的一個數值和名稱，不代表學術或科學結論喔！😊

=====

Ex. 2.1.2



工具

Model	API args	OpenAI相容	Notes
OpenAI (ex: GPT-4o)	tools	相容	基準;支援度最完美，可穩定同時執行 10 個以上的平行工具。
Google (ex: Gemini)	tools	是 (透過其相容 API)	原生 SDK 架構極為相似，但它是用 FunctionDeclaration 物件包裝。
Anthropic (ex: Claude)	tools	否 (原生欄位相同，但傳入結構微異)	雖然也叫 tools，但 Claude 3.5 的參數結構和 tool_choice 的寫法與 OpenAI 有細微出入。
開源模型 (Gemma / Llama)	依賴推論引擎	是 (透過 Ollama/vLLM)	模型本身不具備 API，必須靠 Ollama / vLLM 這類後端將 OpenAI 格式轉成模型看得懂的提示詞。

工具

- 除了標準的 `client.chat.completions.create` 之外，實務上還有以下幾種常見的呼叫手段：

1. Streaming (序列串流輸出)

如果你希望模型的字是一個一個「吐」出來（像 ChatGPT 網頁那樣），而不是等全部生完才顯示，會加上 `stream=True`：

python

```
response = client.chat.completions.create(
    model="...",
    messages=[...],
    stream=True # 啟用串流
)
for chunk in response:
    print(chunk.choices[0].delta.content, end="")
```

串流模式下處理 Function Calling 會變得很複雜，因為工具參數也是一塊一塊片段吐出來的，需要自行拼接。

工具

- 除了標準的 `client.chat.completions.create` 之外，實務上還有以下幾種常見的呼叫手段：

2. 結構化 JSON 輸出 (`response_format`)

如果你不需要模型自己決定何時用工具，只是單純想要模型「乖乖吐出格式正確的 JSON 格式」，你可以不用 `tools`，改用 `response_format`：

```
python
response = client.chat.completions.create(
    model="...",
    messages=[...],
    response_format={ "type": "json_object" } # 強制輸出 JSON
)
```

工具

- 除了標準的 `client.chat.completions.create` 之外，實務上還有以下幾種常見的呼叫手段：

3. 透過高階框架（如 LangChain / LlamaIndex / AutoGen）

在真正的企業級開發中，大家很少從頭手寫 `client.chat.completions` 和 `for tool_call in msg.tool_calls` 的迴圈（也就是你上一題遇到的苦工）。

大家會用框架把工具包裝起來，讓框架自動去跑迴圈：

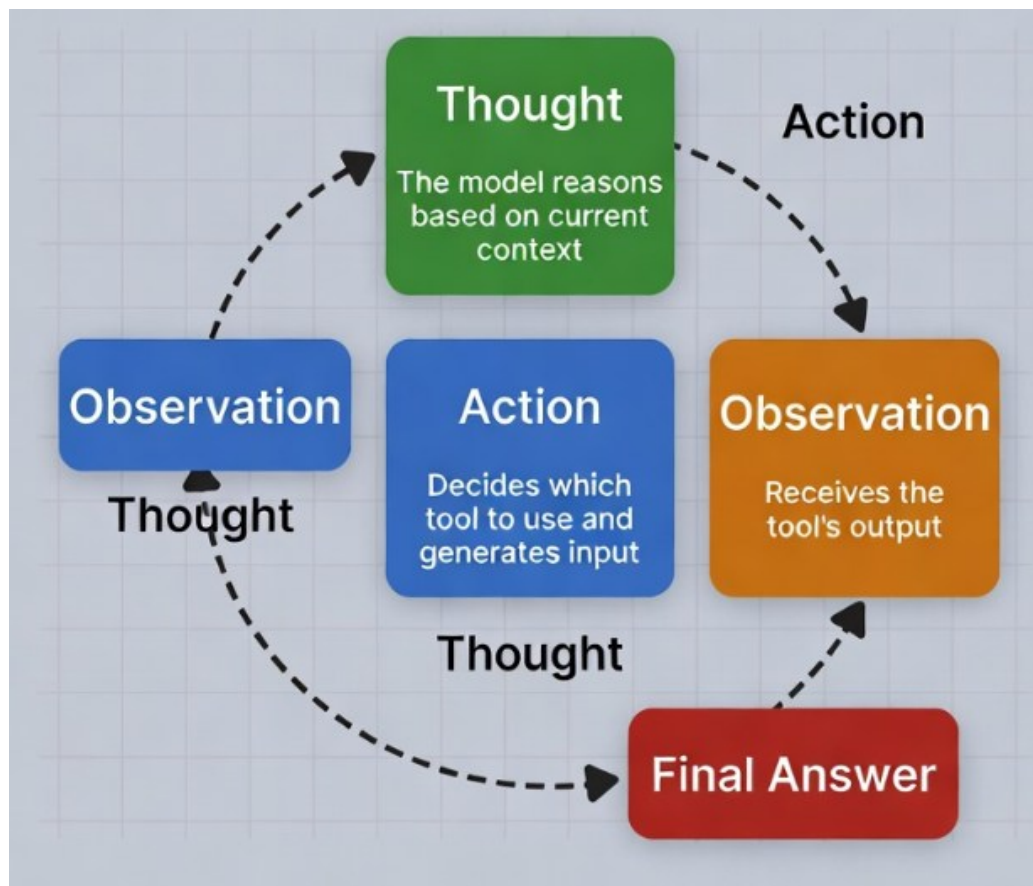
python

```
# LangChain 偽程式碼範例：直接把 Python 函式綁定給模型，後面它自己會跑完所有流程
model_with_tools = langchain_gemma.bind_tools([get_weather, calculate_fate_weight])
response = model_with_tools.invoke("幫我查高雄天氣和生日命重")
```

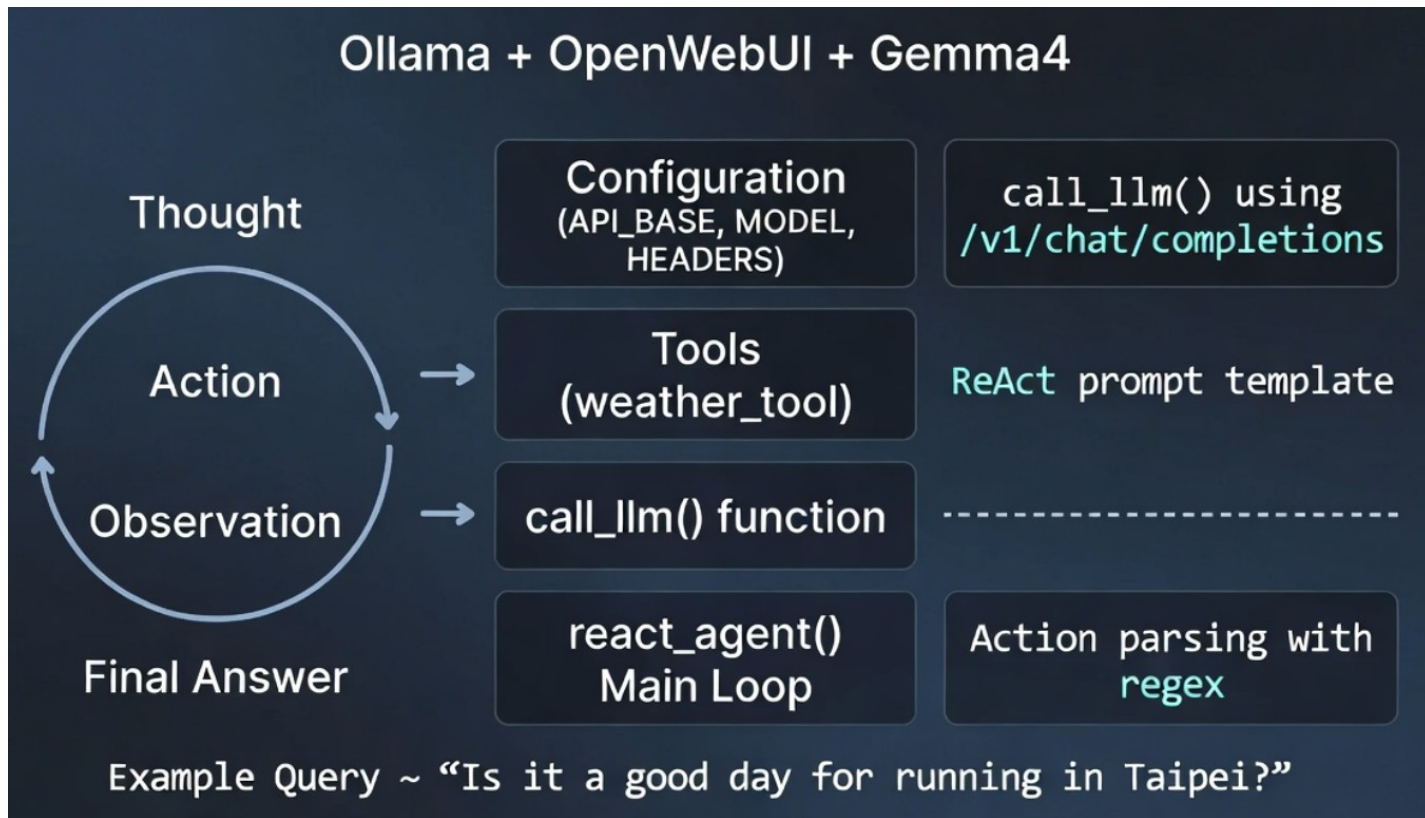
工具及協議

- ReAct
- 範例：「今天在台北跑步適合嗎？」
- 1. 思維過程 (Thought)：
我需要查詢當天的天氣狀況。
- 2. 執行動作 (Action)：
我會調用「天氣工具」，並輸入地點「台北」。
- 3. 觀察結果 (Observation)：
天氣資料顯示：氣溫 30 ° C，多雲，下午有雷陣雨。
- 4. 判斷 (Thought)：
根據這些資訊判斷，這樣的天氣不適合戶外跑步。
- 5. 最終建議 (Final Answer)：

由於預計有雨，今天不建議進行戶外跑步活動。



Ex. 2.1.3



Ex. 2.1.4

- 多輪
- 常見的停止機制有以下幾種：
 - 檢測 “Final Answer”（如果LLM具備這類輸出，則最推薦）
 - LLM 輸出 "No action needed" 或 "None"
 - 達到最大步數（安全網）
 - 連續多次沒有 Action

協議

Ex. 2.2.1

ex_2_function_calling

```
>python app_mcp.py 19761026 台北.
>>> [MCP] 正在請求 Server 執行工具: get_weather, 參數: {'location': '台北'}
>>> [MCP] 正在請求 Server 執行工具: calculate_fate_weight, 參數: {'date_of_birth': '19761026'}

=====
您好！根據您的要求，我為您查詢了今日的台北天氣以及您的命重：

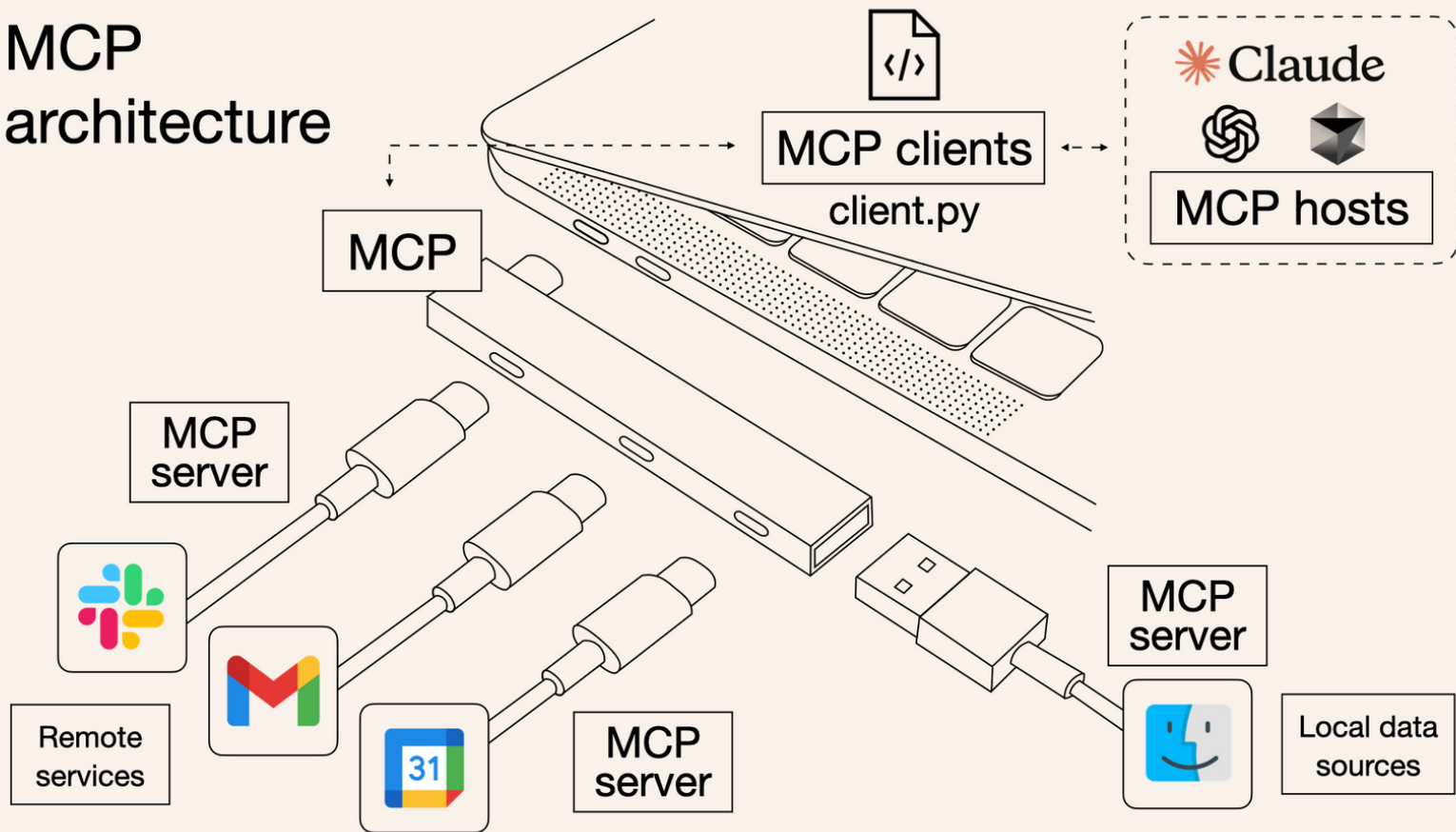
**🌞 今日台北天氣預報：**
* **天氣狀況：** 晴天
* **氣溫：** 約 26°C

**💫 您的命理分析（生日：1976年10月26日）：**
* **命重：** 四兩二錢
* **整體評語：** 中上之命

希望這些資訊能為您帶來一些參考！😊
=====
```

MCP

MCP architecture

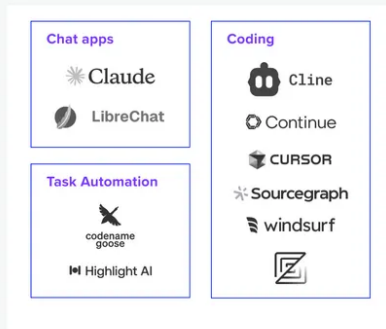


MCP

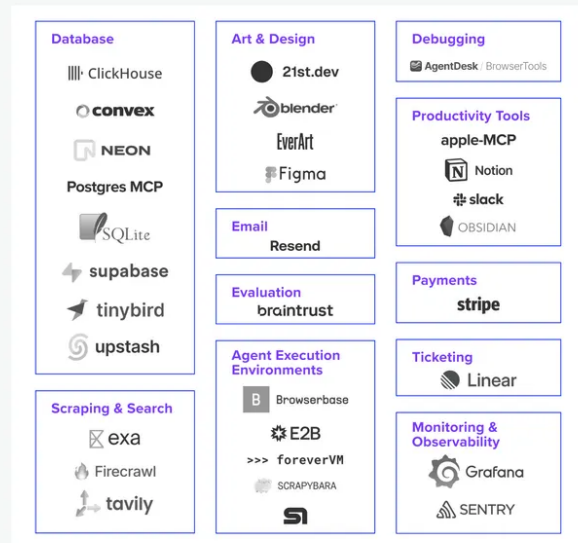
MCP Market Map

A work in progress.

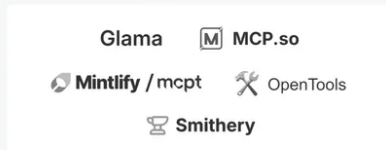
Top MCP Clients



Top MCP Servers



MCP Marketplace



Server Generation & Curation



Server Hosting

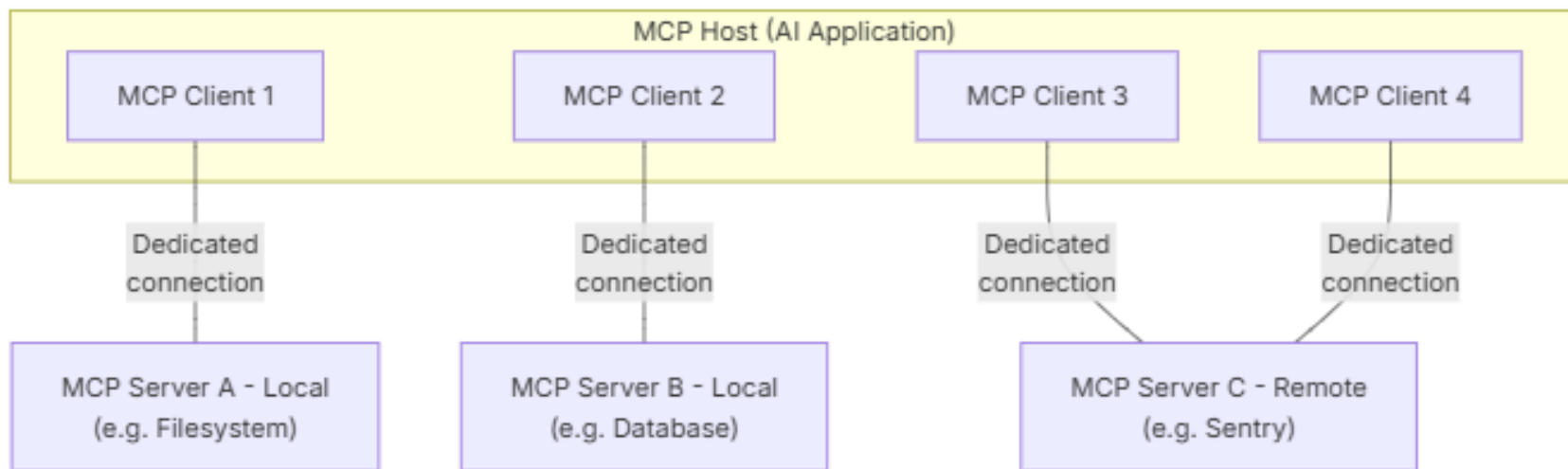


Connection Management



Charts provided herein are for informational purposes only and should not be relied upon when making any investment decision. Past performance is not indicative of future results. None of the above should be taken as investment advice; please see a16z.com/disclosures for more information.

MCP



MCP

Core Server Features

Servers provide functionality through three building blocks:

Feature	Explanation	Examples	Who controls it
Tools	Functions that your LLM can actively call, and decides when to use them based on user requests. Tools can write to databases, call external APIs, modify files, or trigger other logic.	Search flights Send messages Create calendar events	Model
Resources	Passive data sources that provide read-only access to information for context, such as file contents, database schemas, or API documentation.	Retrieve documents Access knowledge bases Read calendars	Application
Prompts	Pre-built instruction templates that tell the model to work with specific tools and resources.	Plan a vacation Summarize my meetings Draft an email	User

MCP

Core Server Features

功能	一句話比喻	核心用途與運作機制	教學案例
Tools	AI 的「手」(動態呼叫)	模型可主動執行的外部函式，通常具備副作用（如建立紀錄、改寫資料、發送請求）。支援傳遞參數並獲得執行結果。	查詢即時氣象 API、在 GitHub 上開 PR、在資料庫執行 SQL。
Resources	AI 的「書櫃」(靜態或半動態資料)	被動讀取的資料來源，以 URI 識別（如 <code>postgres://db/table</code> 、 <code>file:///project/readme.md</code> ）。主要提供上下文資訊，不執行操作。	讀取專案日誌檔、讀取 API 文件、同步系統設定檔。
Prompts	AI 的「預製範本」(引導工作流程)	Server 預先封裝好的標準提示詞樣板。讓使用者或 Client 一鍵觸發特定的任務流程，並可帶入動態參數。	「debug-code」樣板（要求注入錯誤日誌）或「generate-pr-summary」樣板。

MCP

Core Server Features

工具
(Tools)

Functionality through three building blocks:

	Explanation	Examples	Who controls it
Tools	Functions that your LLM can actively call, and decides when to use them based on user requests. Tools can write to databases, call external APIs, modify files, or trigger other logic.	Search flights Send messages Create calendar events	Model

- Server 開放給 AI 執行的「動態功能/函式」AI 可以傳入參數來執行這段 Python 程式碼，且通常會改變外部世界的狀態（例如：寫入檔案、重啟地端服務、發送 API 請求）
- AI 必須決定「要不要用」以及「參數要帶什麼」
- Server 提供 tools/list（宣告工具功能與參數格式）和 tools/call（真正執行 Python 程式並回傳結果）

MCP

Core Server Features

資源 (Resources)

functionality through three building blocks:

Explanation

Examples

Who controls it

- Server 開放給 AI 讀取的「唯讀資料數據」
- Server 提供 resources/list (有哪些資料可看) 和 resources/read (讀取特定資料內容)

Resources

Passive data sources that provide read-only access to information for context, such as file contents, database schemas, or API documentation.

Retrieve documents
Access knowledge bases
Read calendars

Application

Prompts

Pre-built instruction templates that tell the model to work with specific tools and resources.

Plan a vacation
Summarize my meetings
Draft an email

User

MCP

Core Server Features

提示詞模板 (Prompts)

functionality through three building blocks:

Explanation

Examples

Who controls it

- Server 預先寫好的「最佳實踐提示詞範本 (Templates)」。AI 或是 Client 應用程式可以載入這些 Prompts，讓 AI 快速進入特定工作角色或 workflow
- 例如："你現在是審查 {language} 程式碼的專家"
- Server 提供 prompts/list 和 prompts/get (動態帶入變數並組裝出最終的 Prompt 給 AI)

Prompts

Pre-built instruction templates that tell the model to work with specific tools and resources.

Plan a vacation
Summarize my meetings
Draft an email

User

MCP

Core Client Features

In addition to making use of context provided by servers, clients may provide several features to servers. These client features allow server authors to build richer interactions.

Feature	Explanation	Example
Elicitation	Elicitation enables servers to request specific information from users during interactions, providing a structured way for servers to gather information on demand.	A server booking travel may ask for the user's preferences on airplane seats, room type or their contact number to finalise a booking.
Roots	Roots allow clients to specify which directories servers should focus on, communicating intended scope through a coordination mechanism.	A server for booking travel may be given access to a specific directory, from which it can read a user's calendar.
Sampling	Sampling allows servers to request LLM completions through the client, enabling an agentic workflow. This approach puts the client in complete control of user permissions and security measures.	A server for booking travel may send a list of flights to an LLM and request that the LLM pick the best flight for the user.

功能	一句話比喻	核心用途與運作機制	教學案例
Roots	AI 的「作業邊界」	Client 告訴 Server 哪些目錄或工作區是合法的操作範圍。 Server 只能在這些 Roots 底下存取資源，防止安全穿透。	本地 VS Code 開啟 <code>/workspace/app</code> ，IDE 告知 Server 僅能讀寫該資料夾。
Sampling	Server 呼叫「大腦外援」	允許 Server 反向請求 Client 執行 LLM 推論 。Server 不需要自建 API Key 或模型介面，直接借用 Client 現成的 LLM 來解析資料。	Server 抓到 10 萬字未結構化日誌，反向請 Client 的 LLM 萃取前 3 大關鍵錯誤後再回傳。
Elicitation	主動「向人發問」	當 Server 執行 Tool 或流程中發現資訊不足、需要決策確認或需要敏感憑證時，由 Client 彈出介面主動向使用者索取答案。	執行「部署至生產環境」Tool 時，觸發確認視窗要求工程師輸入部署授權碼。

MCP

Core Client Features

In addition to making use of context provided by servers, clients may provide several features to servers. These client features allow server authors to build richer interactions.

根目錄宣告 (Roots)

Exp

Elic
info
prov
info

- 由 Client 告訴 Server
- Client 會主動發送一個清單給 Server，說：
「你這個地端 Server 只能讀寫我指定的這兩個資料夾 (Roots)，其他地方你都不准碰。」
- 保護地端隱私

Roots

Roots allow clients to specify which directories servers should focus on, communicating intended scope through a coordination mechanism.

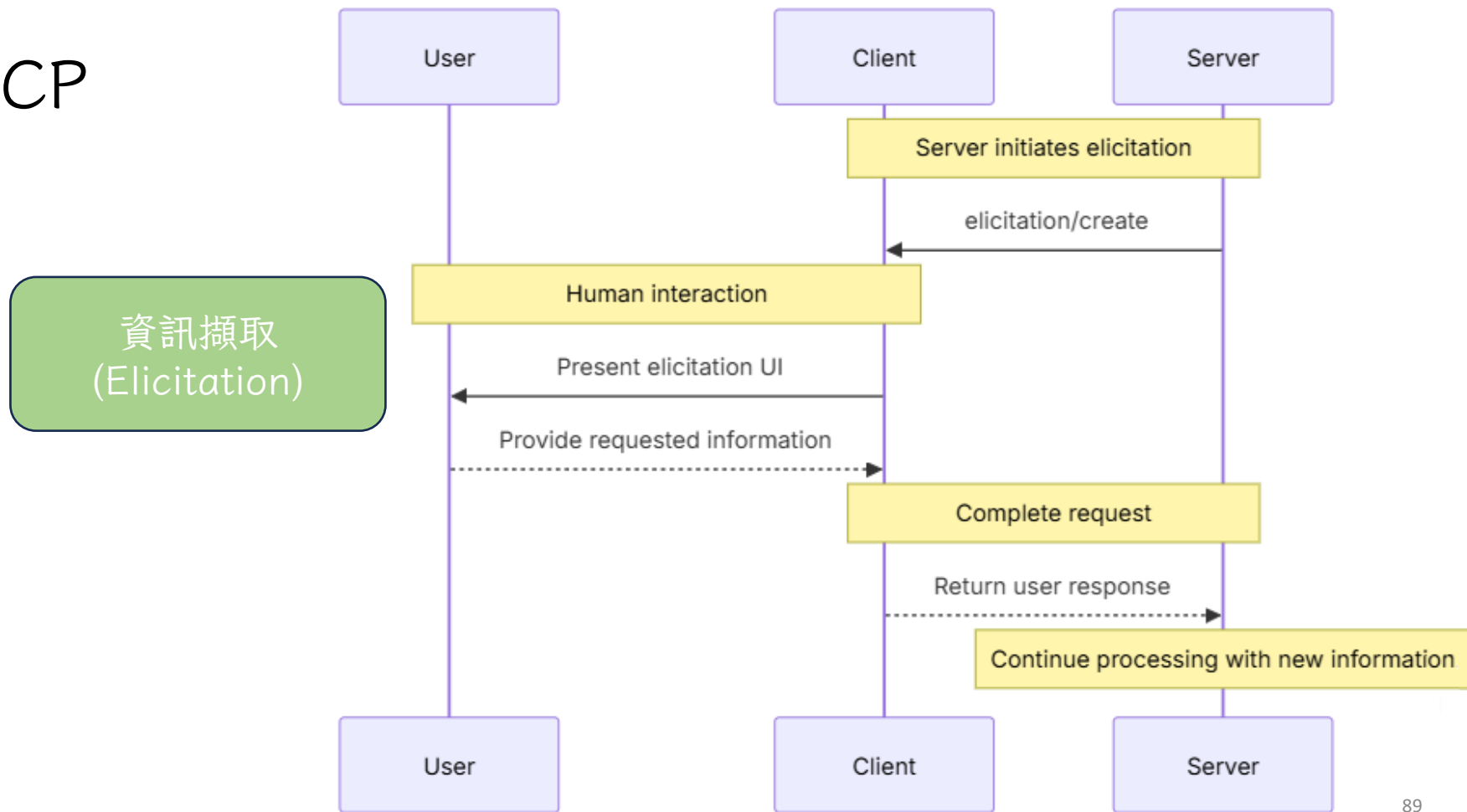
A server for booking travel may be given access to a specific directory, from which it can read a user's calendar.

Sampling

Sampling allows servers to request LLM completions through the client, enabling an agentic workflow. This approach puts the client in complete control of user permissions and security measures.

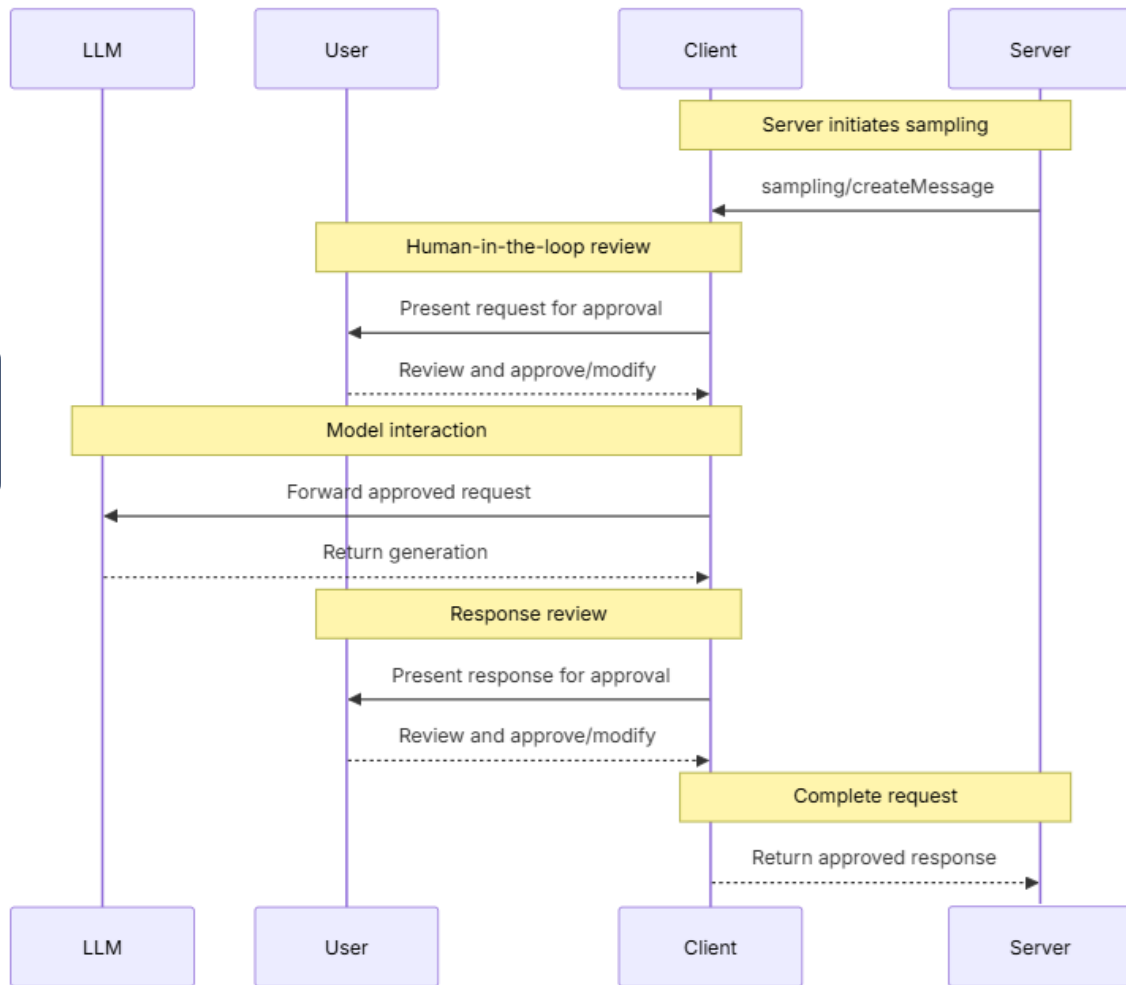
A server for booking travel may send a list of flights to an LLM and request that the LLM pick the best flight for the user.

MCP



MCP

採樣 (Sampling)



MCP

- 「AI 秘書 (Client) 與 後勤辦公室 (Server)」
 - Server (AI 能用什麼？)
 - 「秘書要完成任務，辦公室給了他三樣東西：Resources (查閱檔案卷宗)、Prompts (標準作業流程 SOP)、以及 Tools (蓋章、寄信等執行動作)。」
 - Client 核心 (辦公室需要什麼保護與支援？)
 - 「辦公室不能為所欲為，秘書也要掌控全域：Roots (劃分禁區，只能翻特定辦公室的檔案)、Elicitation (遇重大決策停下來問主管『確定要刪嗎？』)、Sampling (辦公室遇到看不懂的文件，交給秘書本人的大腦幫忙消化)。」

MCP

- MCP 採用 Client-Server 架構，包含：
 - MCP Host：AI 應用程式
 - MCP Client：Host 內部的連接器，負責與 MCP Server 溝通
 - MCP Server：連接外部資料來源或工具並回傳結果
- MCP 採用 JSON-RPC 2.0 作為通信協議，並定義三大BASE：
 - Resources：上下文資料（ex: 文件、資料庫記錄等）
 - Prompts：Template指令，指導AI模型行為
 - Tools：可供AI 呼叫的函數(API)
- 過去: N 個 LLM $\times$ M 個工具 = $N \times M$ 次客製;
- MCP 推出後: Client $\leftrightarrow$ Host $\leftrightarrow$ Server 統一架構
 - MCP Server：負責暴露工具（檔案系統、資料庫、API、Git 等）
 - MCP Host：負責與 LLM 溝通（Claude Desktop、Cursor、OpenWebUI 自訂客戶端等）
 - LLM：只需要認識 MCP 協定就好
- MCP 的比喻：就像 USB-C 一樣，一個接口走天下。寫一次 Server，任何支援 MCP 的 LLM 都能用。

Ex. 2.2.2

- 示範重點：
 - Tool Definition (JSON Schema)
 - 簡單 MCP Client 實作讓本地 LLM (Gemma) 能讀取本地檔案

功能 (Feature)	步驟一：讓 AI 看菜單 (宣告)	步驟二：AI 決定點菜 (執行)	傳遞的本質
Resources (資源)	@server_app.list_resources()	@server_app.read_resource()	靜態資料 / 唯讀 (Read)
Tools (工具)	@server_app.list_tools()	@server_app.call_tool()	動態動作 / 讀寫 (Execute)
Prompts (提示詞)	@server_app.list_prompts()	@server_app.get_prompt()	角色樣板 / 注入 (Inject)

MCP

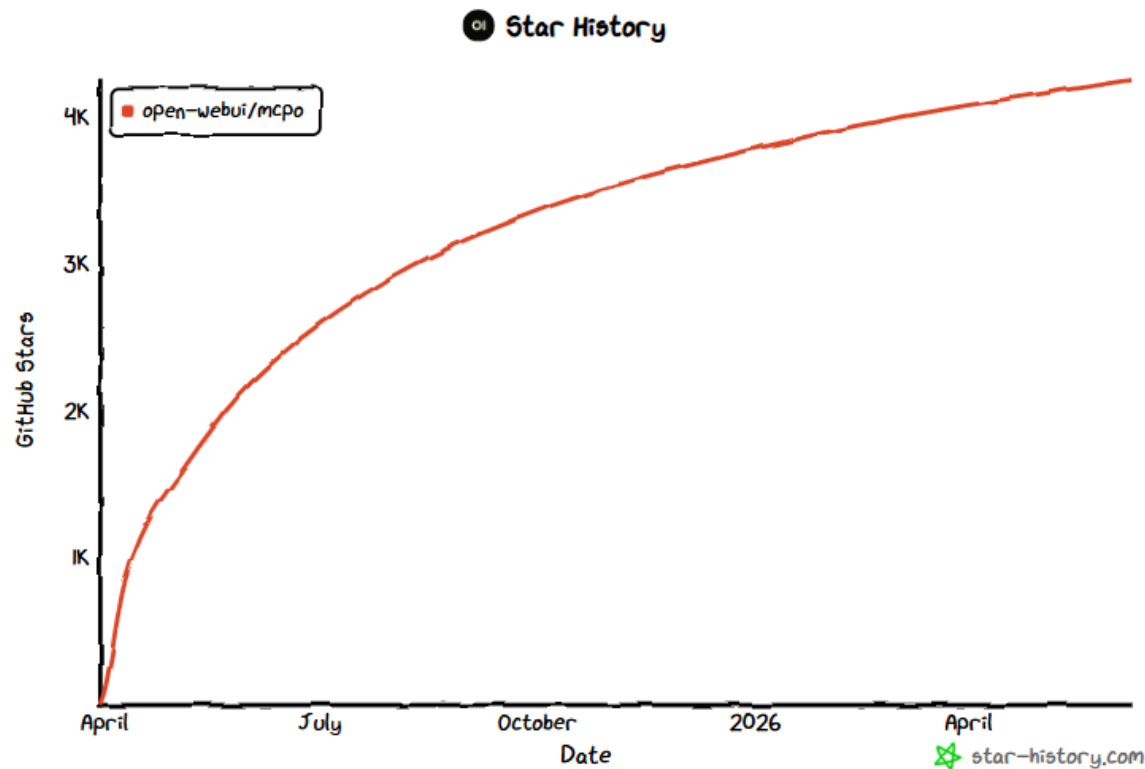
- MCP 通訊模式
- stdio 傳輸：是一種地端通訊方式。MCP 伺服器在本地機器上運行，並由宿主應用程式自動管理。客戶端和伺服器之間透過標準輸入（stdin）和標準輸出（stdout）進行直接通訊，這種方式僅限於本地存取。宿主應用程式會自動執行預設的 shell 命令來啟動和管理伺服器。
- SSE 傳輸：則提供了更大的彈性。MCP 伺服器可以運行在本地，也可以部署在遠端伺服器上，完全由使用者進行管理。客戶端和伺服器之間透過網路進行通訊，因此可以在不同的機器之間共享和使用。客戶端需要提供外部 MCP 伺服器上 /sse 端點的 URL 來建立連接。

MCP

- mcpo
- Open WebUI 官方推出的一個專案（全名是 MCP-to-OpenAPI Proxy）
- 來解決一個痛點：
 - 許多開源的 AI 客戶端（例如早期的 Open WebUI、LangChain、或者是各式各樣的傳統 API 網關）只認得標準的 HTTP RESTful API (OpenAPI 規範)，但不支援 Anthropic 官方的 MCP 通訊協定。
- 原生 MCP SSE 版本本質：它是遵照 Anthropic 官方標準的 MCP 網路通訊協議。通訊流程：它是一個標準的 MCP 伺服器，對外公開一個 /sse 端點（透過 HTTP POST 與 Server-Sent Events 流式傳輸進行雙向或單向通訊）。誰來接它：必須是原生支援 MCP 協定的客戶端才能連它。例如 Claude Desktop、Cursor、Windsurf，或者像我們剛剛用官方 mcp SDK 自行寫出來的 ClientSession 程式碼。
- mcpo 版本（Open WebUI 體系）本質：它是一個「翻譯官 / 代理伺服器 (Proxy)」。通訊流程：mcpo 通常在背景吃掉一個地端的 stdio MCP 伺服器（或是連上某個 MCP SSE）。然後，mcpo 自己會啟動一個標準的 Web 服務，把這些 MCP 工具包裝、翻譯成傳統網頁那種萬用的 OpenAPI (openapi.json / Swagger UI) 端點。誰來接它：不支援 MCP 的老系統或傳統 LLM 框架（如不裝外掛的 Open WebUI）。它們只要透過傳統的 HTTP POST 呼叫 mcpo 的網址，就能間接調用到 MCP 工具。

MCP

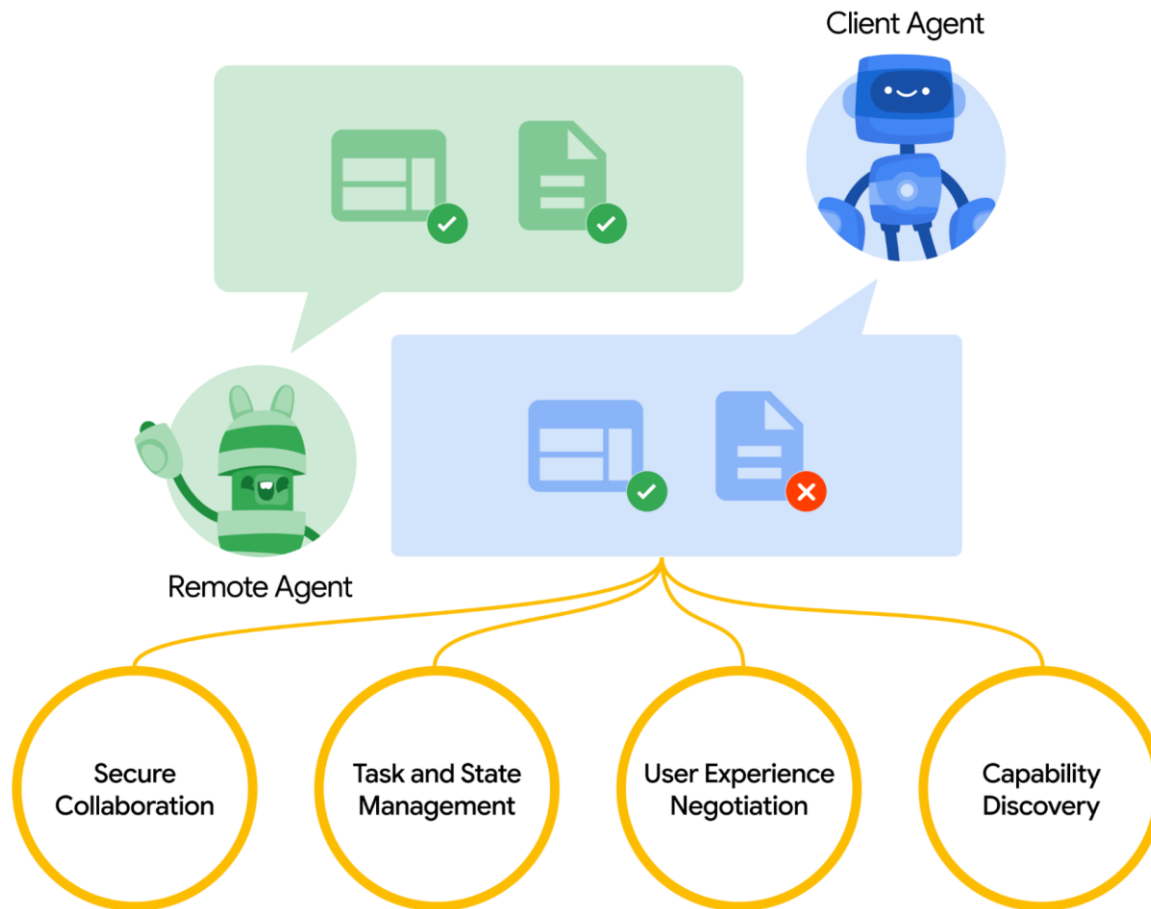
- MCP 通訊模
- mcpo



Ex. 2.2.3 (MCP)

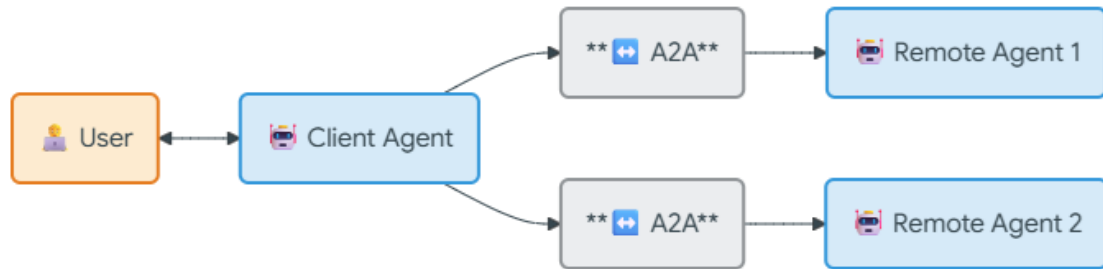
- 示範重點：
 - MCP 通訊模式: stdio 傳輸
 - MCP 通訊模式: SSE 傳輸
 - Mcpo
- `ex_mcp_mcpo_client.py` (`mcp_stdio_mcpo_server.py`)
- `ex_mcp_sse_client.py` (`mcp_sse_server.py`)
- `ex_mcp_stdio_client.py` (`mcp_stdio_server.py`)

A2A



A2A

Why use the A2A Protocol



Interoperability

Connect agents built on different platforms (LangGraph, CrewAI, Semantic Kernel, custom solutions) to create powerful, composite AI systems.

Complex Workflows

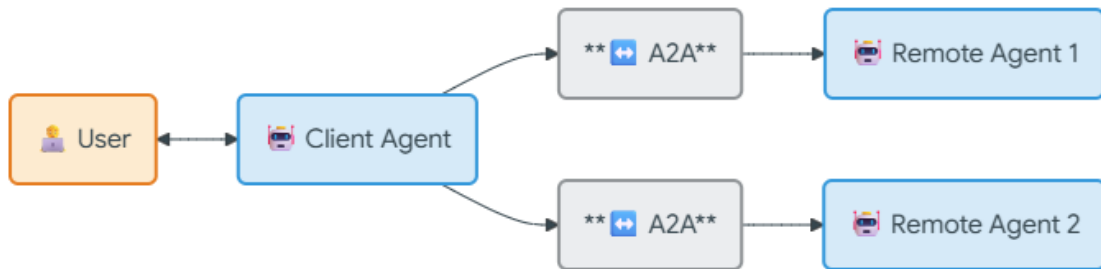
Enable agents to delegate sub-tasks, exchange information, and coordinate actions to solve complex problems that a single agent cannot.

Secure & Opaque

Agents interact without needing to share internal memory, tools, or proprietary logic, ensuring security and preserving intellectual property.

A2A

Why use the A2A Protocol



代理名片宣告
(Agent Card)

多模態訊息元件
(Message Parts
Architecture)

任務生命週期狀態機
(Task Lifecycle State
Machine)

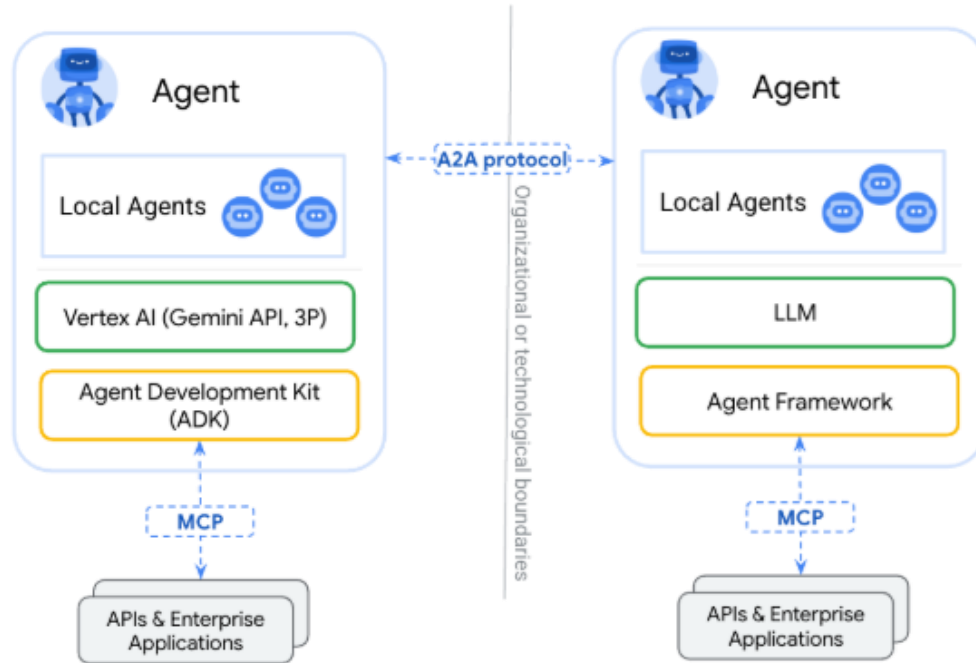
Secure & Opaque

Agents interact without needing to share internal memory, tools, or proprietary logic, ensuring security and preserving intellectual property.

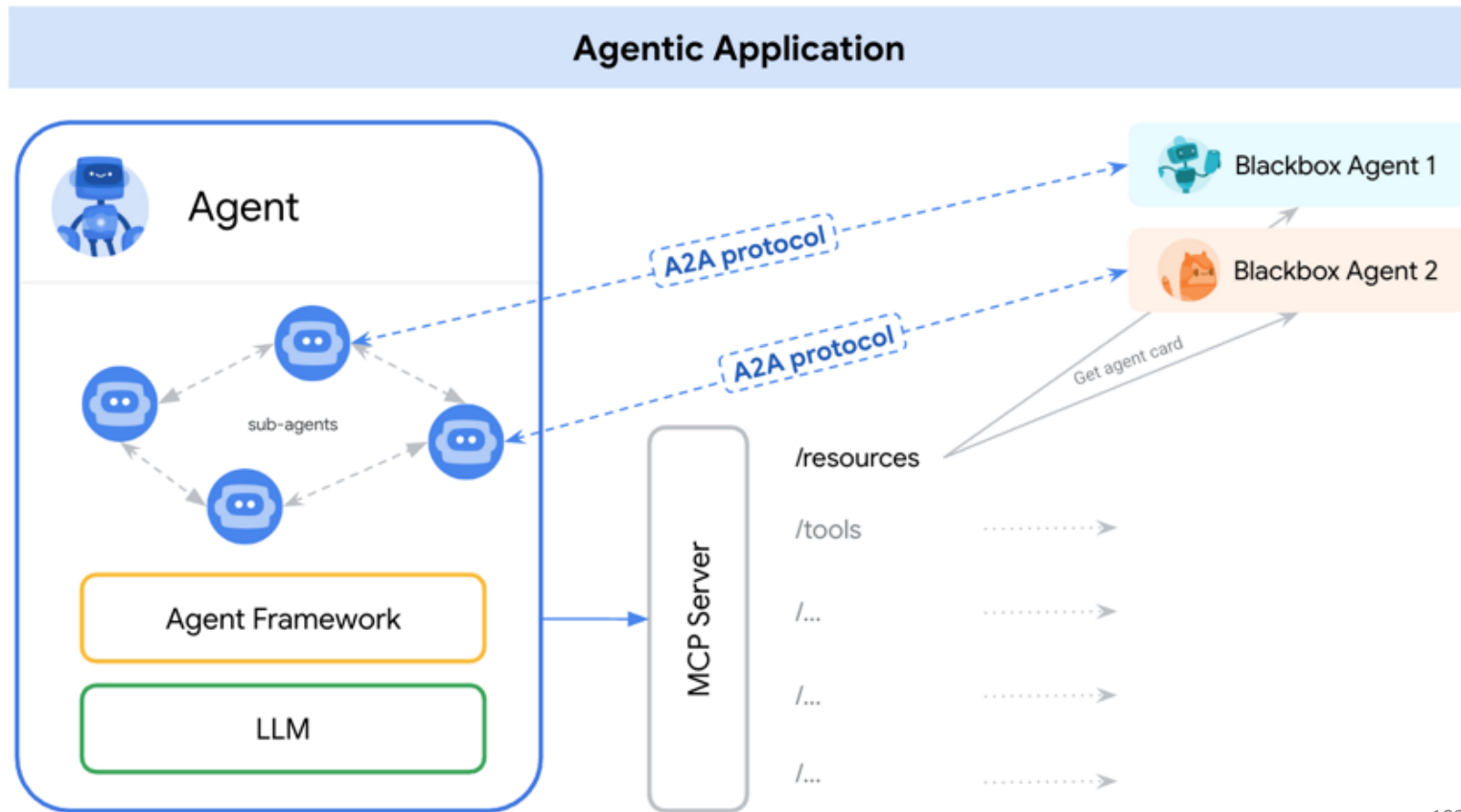
傳輸層抽象與安全驗證
(Transport Abstraction
& Auth)

A2A

How does A2A work with MCP?



A2A



Ex. A2A

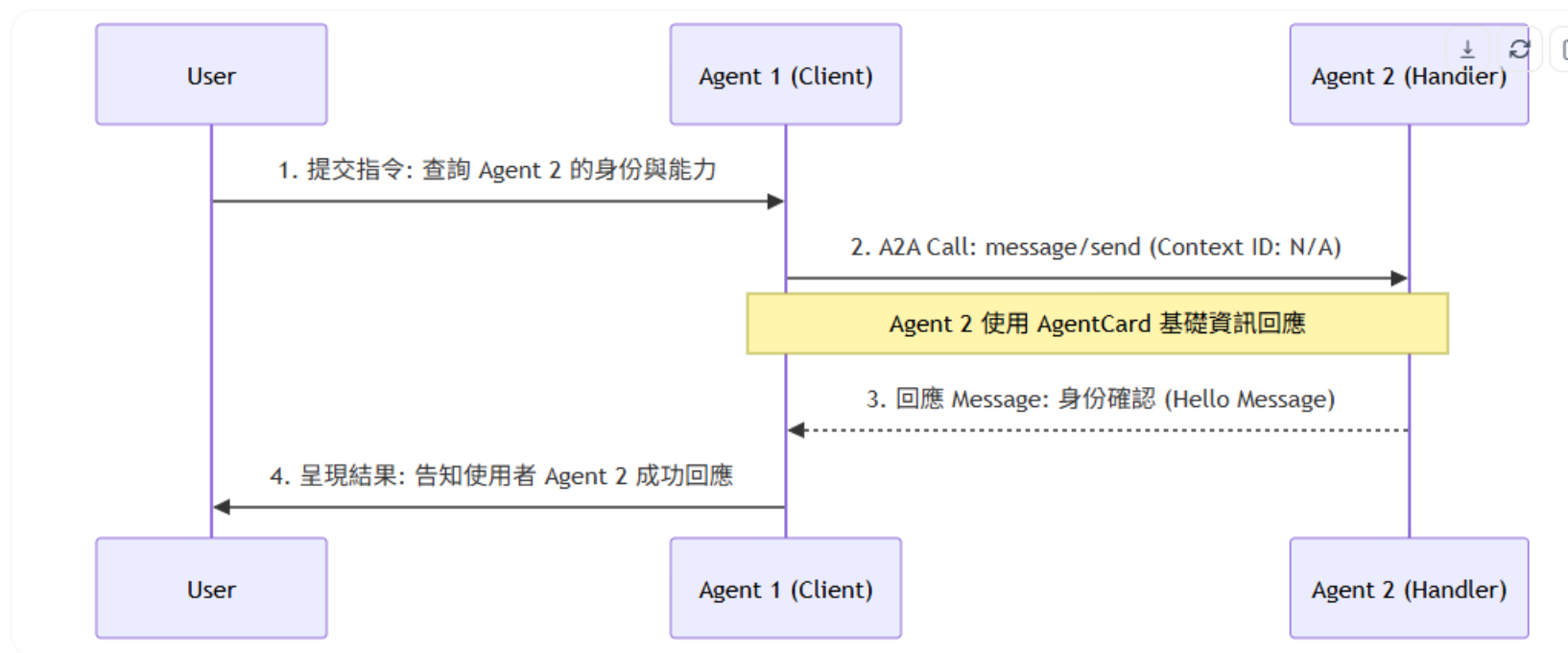
- 示範重點：
 - Agent Card 的定義簡單 A2A Server 實作（讓本地 Agent 能被其他 Agent 呼叫）
 - 展示 Agent 如何委派任務給另一個 Agent
 - `ex_a2a_calling.py`

A2A

👛 情境一：最簡單的 Say Hello (基礎 Message 互動)

這是測試兩個代理 (Agent 1 ↔ Agent 2) 能否透過一次訊息交換來確認彼此身份和基礎功能的最簡單場景。

1. 資訊傳播圖 (Interaction Chart)

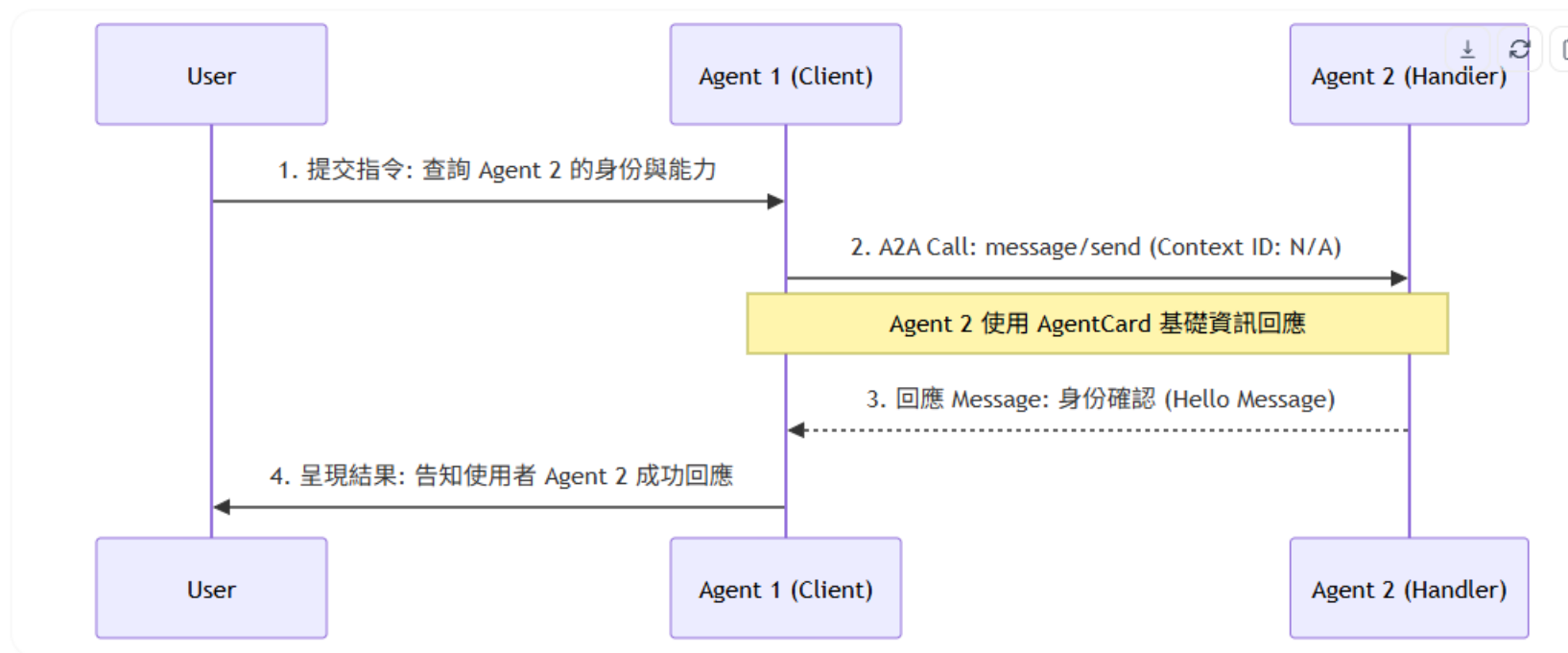


A2A

👛 情境一：最簡單的 Say Hello (基礎 Message 互動)

這是測試兩個代理 (Agent 1 ↔ Agent 2) 能否透過一次訊息交換來確認彼此身份和基礎功能的最簡單場景。

1. 資訊傳播圖 (Interaction Chart)

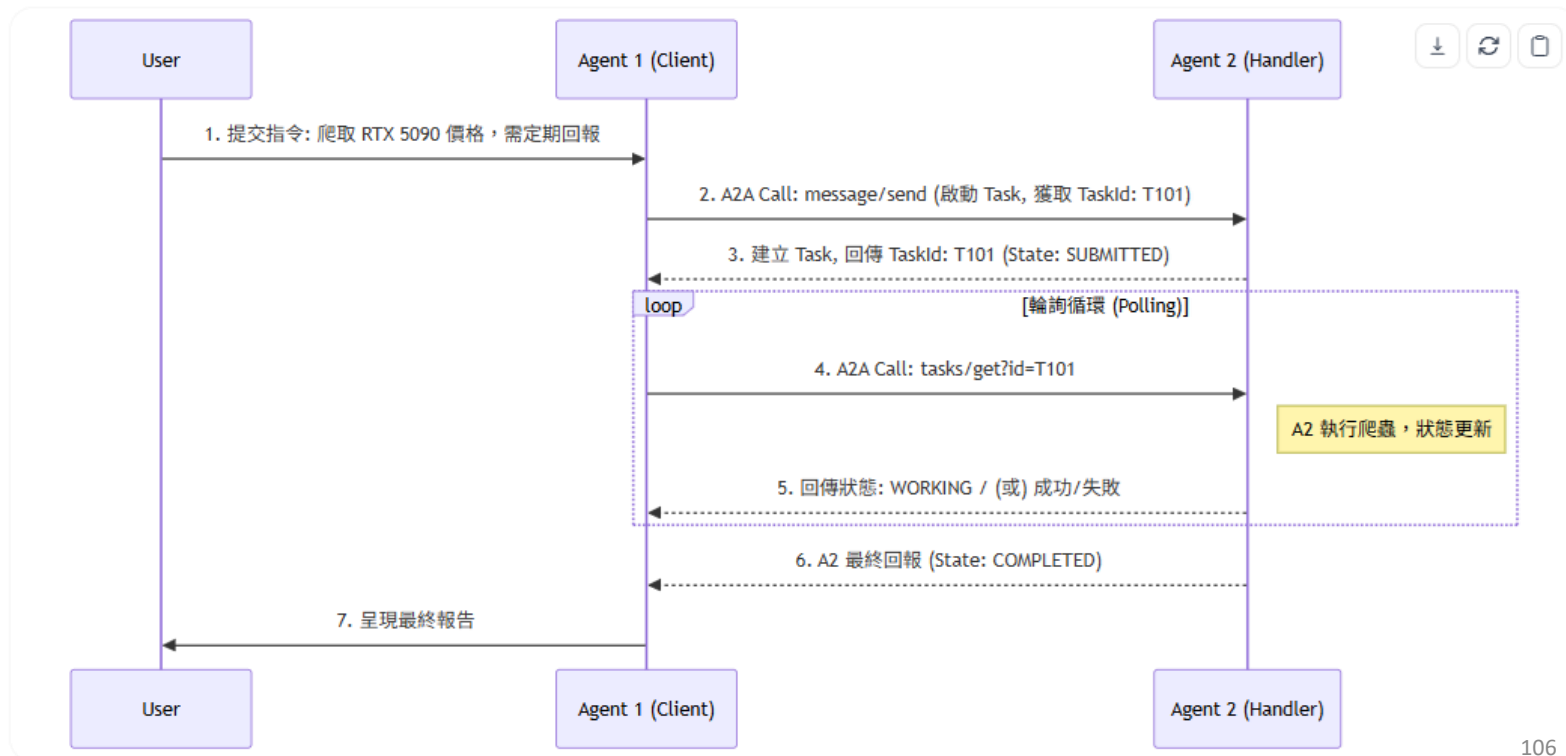


A2A

🌐 情境二：資料爬取與輪詢 (Task & Polling)

這模擬了需要時間、需要重複查詢狀態的場景。關鍵是引入 `Task ID` 和定時輪詢。

1. 資訊傳播圖 (Interaction Chart)

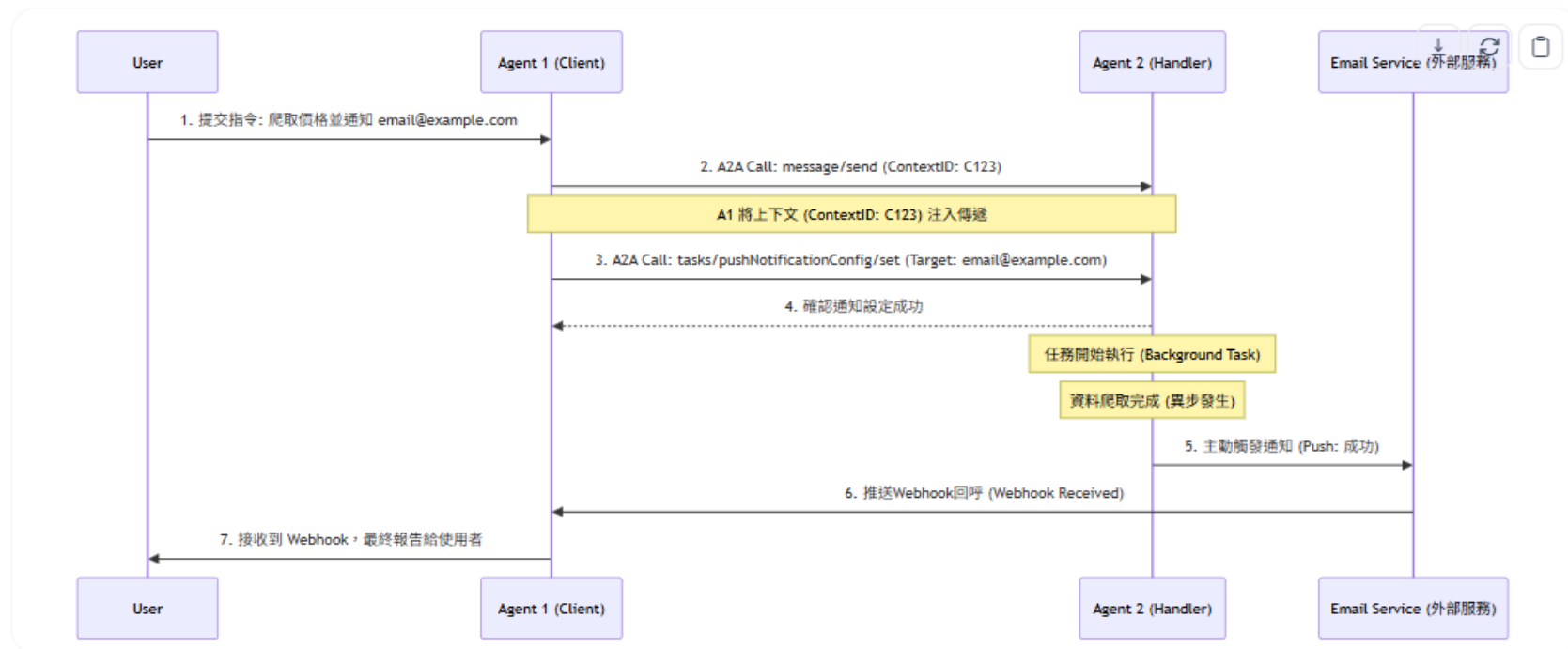


A2A

📞 情境三：郵件通知 (ContextID + Push Notification)

這是最進階的場景，結合了 `contextId` (隱私上下文)、`tasks/pushNotificationConfig` (主動通知) 和 `message` (最終報告)。

1. 資訊傳播圖 (Interaction Chart)



A2A vs MCP

項目	MCP (Model Context Protocol)	A2A (Agent-to-Agent Protocol)
推出組織	Anthropic	Google (已捐贈 Linux Foundation)
主要解決問題	Agent 與 工具/資料源 的連接	Agent 與 其他 Agent 的溝通與協作
核心角色	工具層 (垂直整合)	協作層 (水平整合)
關鍵概念	Tool Definition、JSON Schema、Tool Calling	Agent Card、Task、Artifacts、Status Update
架構模式	Client ↔ Host ↔ MCP Server	Agent (Client) ↔ Agent (Server)
比喻	USB-C 接口 (工具箱標準化)	企業 Slack + 任務委派系統
使用時機	需要讀檔、查資料庫、呼叫 API 時	需要分工、委派專業任務、Multi-Agent 協作時
主要能力	工具發現、動態呼叫、安全執行	Agent 發現、任務委派、結果回傳、狀態追蹤
典型應用	讀取本地檔案、查天氣、查資料庫	總指揮 Agent 委派給「天氣專家 Agent」「財務 Agent」
目前生態	Claude Desktop、Cursor、部分開源工具	LangChain、AutoGen、Salesforce 等多廠商支持

MCP 讓 Agent「會做事」，A2A 讓 Agent「會合作」
兩者結合才是完整的 Agent 生態。

LLM部件討論

Few shots

- Few-shot Learning (少樣本學習) 的核心精神是：「給模型看幾條範例，它就能心領神會，立刻學會新規則。」
 - 像教導一個聰明的員工，你不需要給他一本厚厚的員工手冊 (Fine-tuning 微調)，也不只用嘴巴講規定 (Zero-shot 零樣本)，而是直接做 3 到 5 次給他看，他就能馬上照著做出對的成果。
- Few-shot Learning 的主要優勢在於其效率與便利性 (Few-shot 是控制模型輸出格式 (不亂說廢話、不漏掉符號) 最穩定、最有效的方法)
- 由於不需要進行傳統的模型微調或重新訓練，使用者只需設計好包含範例的提示即可使用
- 品質穩定：相較於完全不提供範例的零樣本學習 (Zero-shot Learning)，提供少量高品質範例能顯著提升模型輸出的穩定性與準確性，因為模型有具體的參考依據。
- 無法提供大量新事實；儘管 Few-shot Learning 非常實用，但它存在明確的應用限制：知識容量有限：提示 (Prompt) 的長度 (Context Window) 是有限的。
- 無法透過這種方式向模型灌輸大量全新的、複雜的事實性知識或數據庫。

Ex. Fewshot

- 示範重點：
 - Few shot calling
 - `ex_fewshot.py`

In context

- 直接將整段背景資料貼在 Prompt 裡In-context Learning (ICL) 是一種讓大型語言模型（LLM）無需額外訓練或參數更新，僅透過在輸入的提示（Prompt）中提供相關的上下文、範例或背景資料，就能夠學習並執行新任務的技術。
- 其核心運作方式是將任務相關的資訊（例如整段背景資料、數個「輸入-輸出」範例）直接拼接在查詢之前，形成一個完整的輸入序列。
- 模型會根據這個序列中的上下文資訊來推斷任務意圖，並生成對應的輸出。這種方法本質上是一種「透過類比學習」的過程，模型利用提供的上下文來引導其推理。

In context

- 「如果它用在回覆正確資料的話」
 - 答案是：可行，但有嚴格的適用條件與限制。
- 當您需要模型根據一份特定、有限且完整的資料（例如一份合約、一篇論文、一份操作手冊）來回答問題時，ICL 是非常有效的策略。
- 您可以將這份資料作為上下文直接放入提示中，模型便能基於您提供的「正確資料」進行推理和回答，而不是依賴其預訓練階段可能學到的過時或不準確的知識。這種方法能顯著提升回答的準確性與相關性。
- 限制與風險：
 - 資料量瓶頸：如前所述，此方法最大的限制是上下文視窗。如果「正確資料」的長度超過了模型的上下文視窗，就無法完整放入，導致模型只能基於不完整的資訊回答，可能產生錯誤。
 - 成本問題：對於長篇資料，每次查詢都將其完整放入提示中，Token 成本會非常可觀，尤其是在需要進行大量查詢的場景下。
 - 模型並非「學習」而是「參考」：研究指出，ICL 過程中模型並不會真正「學習」新的輸入-標籤對應關係，而是利用提供的上下文來調整其輸出格式和利用預訓練階段的先驗知識。這意味著，如果提供的「正確資料」與模型預訓練知識存在衝突，或資料本身非常新穎、複雜，模型可能無法完美地「複述」或應用這些資料。
 - 對範例品質敏感：ICL 的性能高度依賴於所提供的上下文範例的品質、選擇和順序。如果提供的背景資料組織混亂、包含矛盾資訊或格式不佳，模型的輸出品質也會受到負面影響。

Ex. In-context

- 示範重點：
 - 利用 In-context Learning 讓 LLM 扮演一個「客戶服務分析師」，從一段客服對話中提取關鍵資訊並進行分類。
 - `ex_ICL.py`

比較

方法	精神及說明	優點	缺點
Zero-shot	直接下指令，不給任何範例，完全依賴模型的內建知識。直接對員工說：「幫我把這張發票分類。」	1. 速度最快2. 節省 Token 成本3. 提示詞最簡短	1. 格式難以嚴格控制2. 模型容易說廢話3. 複雜任務準確率低
One-shot	只給一個範例，讓模型初步理解想要的格式或風格。對員工說：「像這樣做：發票 A 歸類為差旅費。現在換你做這張。」	1. 能初步定調格式2. Token 成本適中	1. 容易過度擬合（模型可能誤以為所有答案都要跟該範例高度相關）
Few-shot	給 3 ~ 10 個多元範例，讓模型深刻理解任務的邊界與規則。對員工說：「看好了，這 5 張不同部門的發票分別是這樣分類的，現在換你。」	1. 格式控制最穩定2. 複雜邊界判定最精準3. 適合卡片/標籤化輸出	1. 消耗較多 Token2. 執行速度變慢3. 範例品質會嚴重影響結果
In-context Learning	一個上位概念。指「將知識或範例直接放進 Context（對話歷史/Prompt）中讓模型當場學習」的行為。只要是「在開會現場翻書、看範例做做事」的這種工作模式，都算 In-context。	包含 One-shot 與 Few-shot 的所有格式優勢	隨著 Context 變長，模型的注意力可能分散（Needle in a haystack 問題）

finetune

- Fine-tuning 是一種在預訓練模型基礎上，使用特定領域的資料集進一步訓練模型的技術，透過調整模型權重（參數），使其在特定任務上的輸出更準確、更符合預期。這就像是讓一個已經受過通識教育的學生，針對特定專業進行深入學習。
- Fine-tuning 能讓模型深入理解特定領域的專業術語、語氣和結構，例如醫療、法律、金融等領域，提供更精確和具針對性的回應。可從大量資料中學習，與提示工程（Prompt Engineering）不同，可以讓模型從遠超過提示所能容納的大量資料中學習，從而糾正模型原先學到的錯誤資訊，並減少「幻覺」（hallucination）的產生。
- 推理速度較快。由於知識已融入模型權重中，不需要每次查詢都進行外部資料檢索，因此推理過程更高效。

finetune

- 雖然 Fine-tuning 能讓模型學習特定領域的語言風格與深層知識，但在實際落地應用時，面臨以下三大核心挑戰：
 - 成本與時效性限制（高昂且僵化）
 - 更新成本極高、缺乏即時性：微調完成後，模型的知識便凍結在特定時間點。面對即時、動態變化的資訊（如今日新聞、即時股價、新修法規），Fine-tuning 無法即時反映。若要更新資料，必須從源頭重新訓練，耗費大量的時間與計算資源。
 - 前期投入成本高昂：微調需要準備數千到數萬筆高品質、經過專業標註的資料，且訓練過程需要消耗大量昂貴的 GPU 算力，技術門檻與資金實力要求皆高。
 - 資料信任度與管理挑戰（黑盒子與權限問題）
 - 仍具幻覺風險且無法精確溯源：微調雖能改善回答品質，但無法完全根除「幻覺（Hallucination）」現象。更致命的是，微調是將知識轉化為模型的參數權重，因此模型回答時無法給出具體的「原文出處」或引用來源，在醫療、法律等高問責領域風險極高。
 - 【增補】難以進行動態權限控管：餵入微調資料後，所有能使用該模型的人皆能存取該知識。這導致企業內部敏感資料（如薪資、人事機密）無法做到「依員工職等動態隱藏或開放」的權限管理。
 - 模型本體性能風險（顧此失彼）
 - 災難性遺忘（Catastrophic Forgetting）：在強化模型特定領域能力的過程中，模型可能會顧此失彼，遺忘原本具備的通用推理、邏輯或跨領域常識，導致整體泛化能力下降。

Ex. FT

- 示範重點：
 - Finetune pseudo code
 - ex_finetune.py

主流微調方法與框架

分類	方法	核心方法	訓練參數比例
Classic	Full FT	更新模型內所有的參數權重	100%
PEFT	LoRA	凍結原模型，在旁邊外掛兩個低維度矩陣來記錄學習到的新知識	< 1%
PEFT	QLoRA	LoRA 的省錢版。先將原模型壓縮（量化）成 4-bit，再掛上 LoRA 訓練	< 0.1%
PEFT	P-Tuning v2 / Prefix Tuning	不動模型權重，只在輸入端優化一段「虛擬提示詞向量」	< 0.1%
Optimal Framework	Unsloth	不是新微調方法，而是加速引擎。透過手寫底層 Triton 程式碼，優化 LoRA/QLoRA 的運算	依外掛的方法而定(如 LoRA)

Dataset

- [databricks-dolly-15k](https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm) 這個高品質的指令遵循資料集時，資料標註員（annotators）執行了以下七個具體的任務類型：
 - 1. 開放式問答 (Open Q&A)
說明：這些是需要進行開放式思考和知識檢索的問題。答案不一定是一個單一的、可以立即檢驗的事實，有時需要依賴廣泛的世俗知識，或者可能根本沒有「唯一正確」的答案。範例：「人們為什麼喜歡喜劇電影？」或「法國的首都是什麼？」
 - 2. 封閉式問答 (Closed Q&A)
說明：這些問題的答案必須僅僅基於提供給標註員的「參考文本（passage of reference text）」中的資訊。它們是抽取式的事實確認。範例：假設提供了一段關於原子的維基百科段落，問題可能為：「原子核中質子與中子的比例是多少？」
 - 3. 從維基百科中提取資訊 (Extract information from Wikipedia)
說明：標註員會複製一段來自維基百科的文字，然後從中識別並提取特定的實體（entities）或具體的、有事實依據的資訊，例如重量、測量數值等。範例：從一段文字中提取出某物體「的重量是 5 公斤」這類數據。

Dataset

- 4. 從維基百科中總結資訊 (Summarize information from Wikipedia)
說明： 標註員會接收一段較長的維基百科文本，任務是將其濃縮、提煉成一個簡潔的總結版本。
- 5. 腦力激盪 (Brainstorming)
說明： 這類任務要求進行開箱作業式的發想，並且需要提供一系列可能的選項。這類任務著重於擴展想法而非提供單一答案。範例：「我本週末可以和朋友一起做一些有趣的活動有哪些？」
- 6. 分類 (Classification)
說明： 在這個任務中，標註員需要對某個對象或文本執行判斷，判斷其屬於哪個預設的類別，或者判斷文本本身具備的特定屬性。範例：判斷列表中的項目是「動物、礦物」還是「蔬菜」；或者判斷影評的語氣是「正面、負面」還是「中性」。
- 7. 創意寫作 (Creative writing)
說明： 這是要求生成具有高度創造性的內容，不局限於事實陳述。範例：撰寫一首詩 (poem) 或一封情書 (love letter)。

Lora

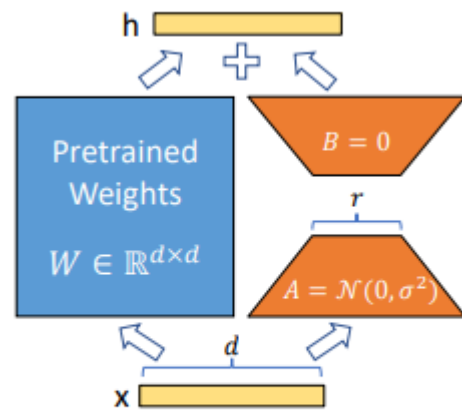
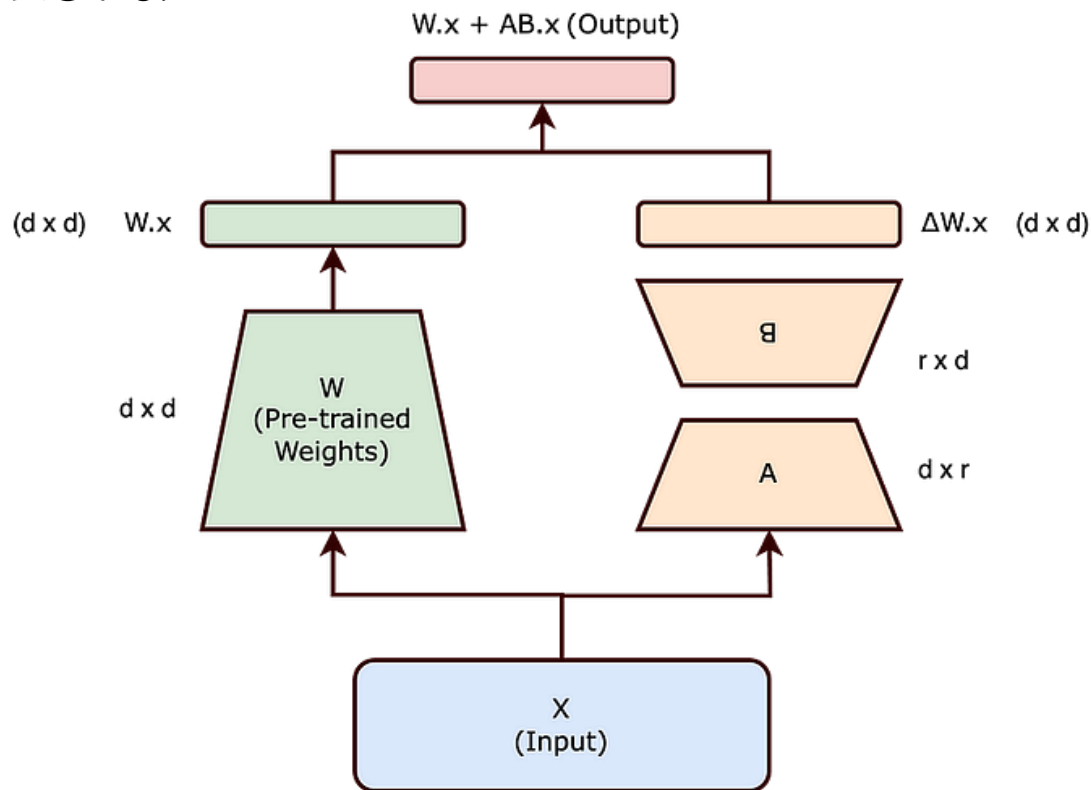


Figure 1: Our reparametrization. We only train A and B .

Ex. LORA

- 示範重點：
 - LoRA finetune pseudo code
 - `ex_fineturne_LoRA.py`

Lora

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

Lora

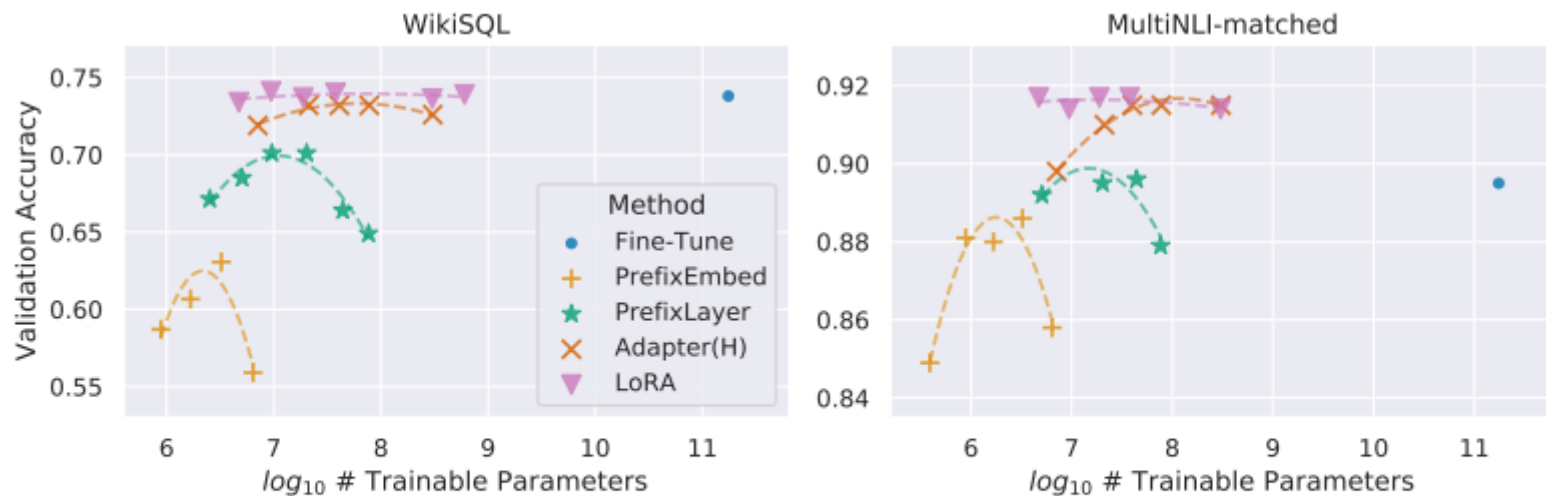


Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance. See Section I.2 for more details on the plotted data points.

Lora

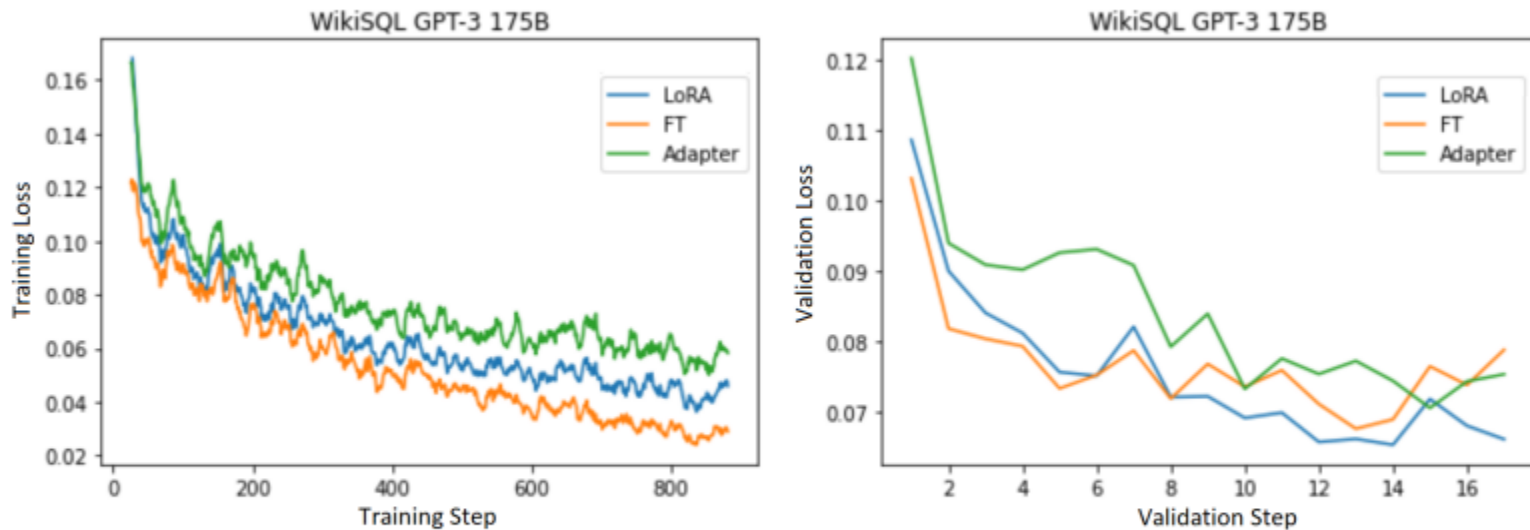


Figure 6: Learning curves for different adaptation methods on WikiSQL using GPT-3 175B. LoRA and full finetuning (FT) reaches their best validation loss slightly faster than Adapter.

qLora

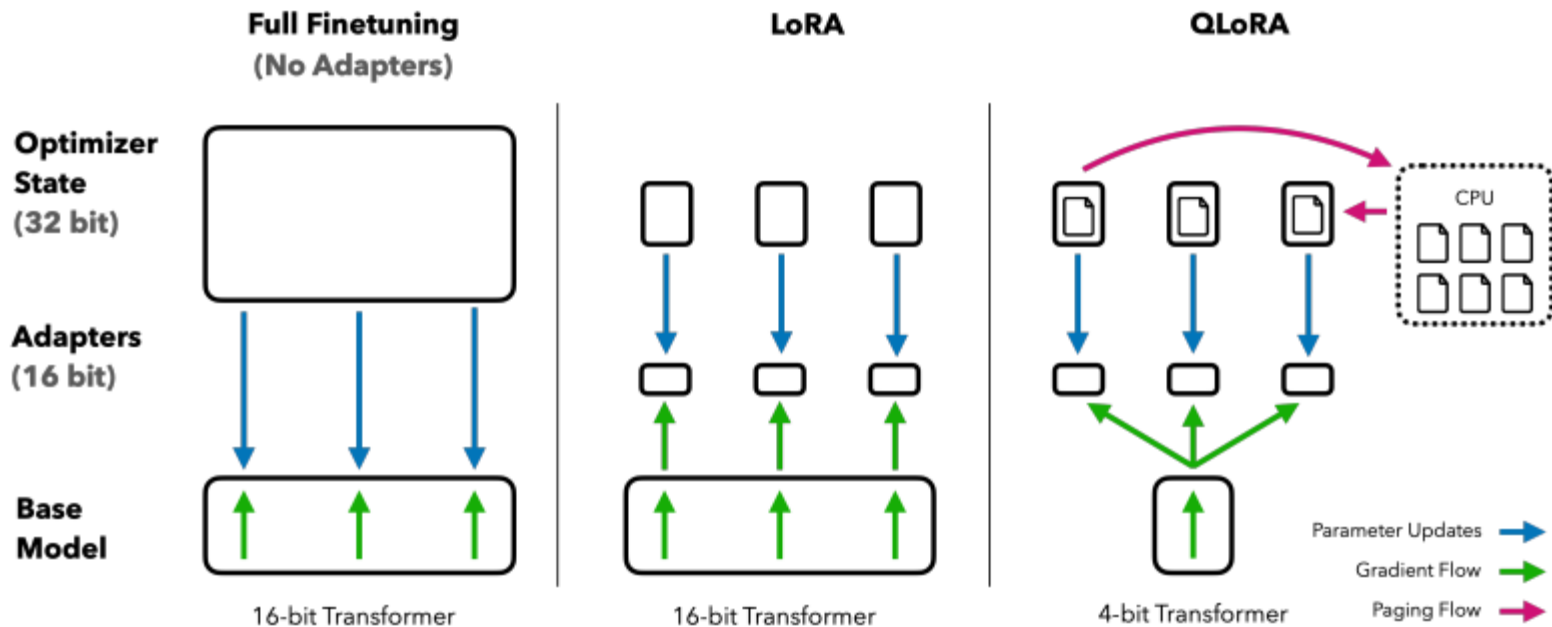


Figure 1: Different finetuning methods and their memory requirements. QLoRA improves over LoRA by quantizing the transformer model to 4-bit precision and using paged optimizers to handle memory spikes.

qLora

- QLoRA (由華盛頓大學團隊於 2023 年提出)
- 精度不掉、顯存暴省，主要靠三個創新：
 - NF4 (NormalFloat 4) 資料格式：傳統的 4-bit 壓縮很容易讓模型變笨。QLoRA 發明了 NF4 格式，這是一種專門為了「神經網路參數通常呈常態分佈」而設計的特殊量化法，能確保模型被壓縮後資訊不失真。
 - 雙重寬量化 (Double Quantization)：連用來記錄壓縮比例的「係數」也跟著再壓縮一次，幫每層參數多省下 0.37 bit 的空間，積少成多。
 - 分頁優化器 (Paged Optimizers)：借鑑了作業系統的虛擬記憶體觀念。如果突然遇到長文本，顯存快爆掉時，它會偷偷把資料暫存到電腦的記憶體 (RAM) 裡，防止訓練過程直接跳出 "Out of Memory (OOM)" 崩潰報錯。

Lora | qlora

	LoRA	QLoRA
簡述	凍結原模型，只訓練旁邊的低維度外掛矩陣	把原模型壓成 4-bit 凍結，再掛上 LoRA 訓練
基礎模型精度	通常是 16-bit (FP16 或 BF16)	4-bit (NF4 專利格式)
顯存需求	低 (以 7B 模型為例，約需 20GB~30GB)	極低 (7B 模型僅需 < 10GB 顯存)
訓練速度	快 (計算直接，無額外轉換開銷)	慢 (微調時需要頻繁進行「解壓縮」計算)
模型精準度	幾乎與全參數微調持平	與 LoRA 幾乎不相上下 (幾乎無損精度)
最適合場景	預算充足的企業、追求快速迭代與訓練速度的專案	預算有限的個人研究者、新創公司、地端設備實驗 (PoC)

Lora 們

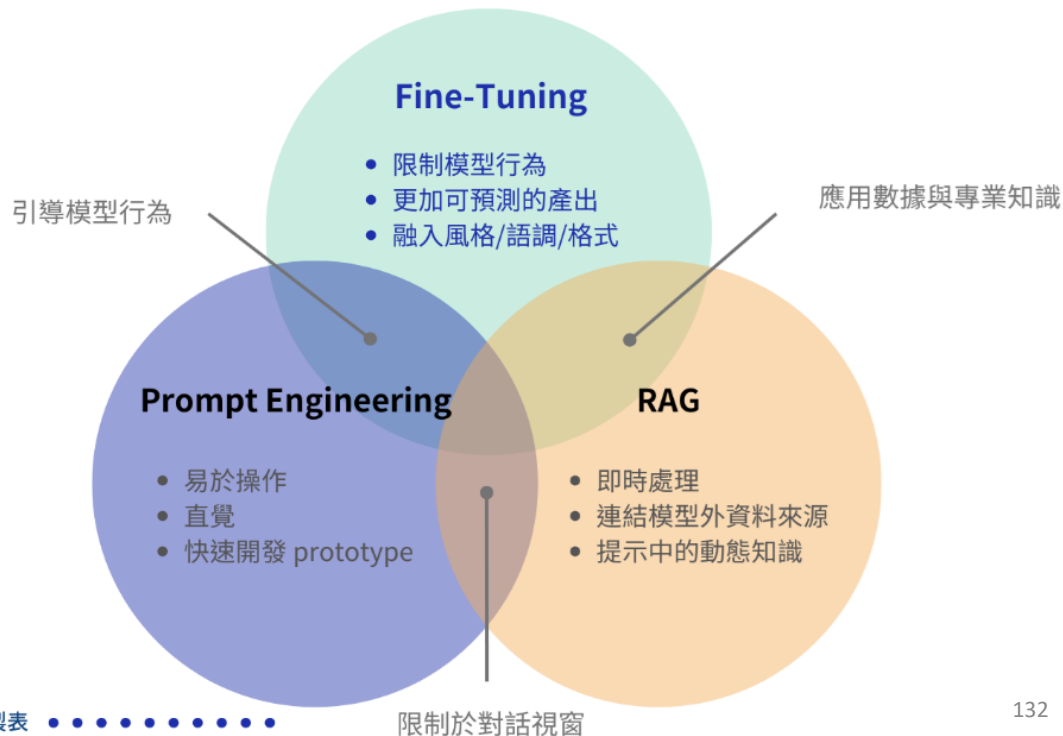
- 2021年6月17日 LoRA: LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS
- 2023年8月7日 LoRA-FA : Memory-efficient Low-rank Adaptation for Large Language Models Fine-tuning
- 2023年9月5日 Delta-LoRA : Delta-LoRA: Fine-Tuning High-Rank Parameters with the Delta of Low-Rank Matrices
- 2023年9月21日 LongLoRA : Efficient Fine-tuning of Long-Context Large Language Models
- 2023年10月17日 VeRA : Vector-based Random Matrix Adaptation
- 2023年12月20日 AdaLoRA : <https://arxiv.org/pdf/2303.10512>
- 2024年6月18日 LoRA-drop : LoRA-drop: Efficient LoRA Parameter Pruning based on Output Evaluation
- 2024年2月14日 DoRA : DoRA: Weight-Decomposed Low-Rank Adaptation
- 2024年7月 LoRA+: LoRA+: Efficient Low Rank Adaptation of Large Models
- 2025年1月21日 EDoRA: Efficient Weight-Decomposed Low-Rank Adaptation via Singular Value Decomposition

還需要Finetune嗎

- 並非 Fine-tuning 技術本身不好，而是因為 大模型（LLM）的底層能力進化，以及 RAG（檢索增強生成）技術的成熟，讓過去必須靠微調才能解決的問題，現在有了更便宜、更高效、更安全的替代方案
- 最核心的原因：「性價比太低」，在大多數商業應用中，微調就像是「為了喝一杯牛奶，去養了一整頭牛」，付出的成本與得到的動態回報不成正比
 - 知識更新：微調是「靜態的」，而世界是「動態的」
 - 情境窗口（Context Window）的恐怖進化
 - 可追溯性與權限管理（幻覺與黑盒子、資料權限無法切割）
 - 基礎模型（Base Model）本身已經太聰明

還需要Finetune嗎

三者結合：打造全方位 LLM 解決方案



還需要Finetune嗎

- Reasoning with Sampling: Your Base Model is Smarter Than You Think
- 傳統文獻多專注於解構在大型語言模型（LLMs）經過強化學習（RL）訓練後才顯現出的新行為。
- 本文轉而提出一個不同的假設：是否能僅透過純採樣（pure sampling），在不進行任何額外訓練的情況下，從基礎模型（base models）中引出可比擬的推理能力？
- 論文提出了一種簡單的迭代採樣演算法。
- 此演算法的靈感來源於用於採樣的馬可夫鏈蒙地卡羅（MCMC）技術，它利用了基礎模型自身的似然值（likelihoods）。
- 最關鍵的是，該方法不需要額外的訓練、人工編纂的數據集，也不需要外部驗證器。這暗示了它具有廣泛的適用性，特別適用於那些不易被驗證的領域。

還需要Finetune嗎

- Reasoning with Sampling: Your Base Model is Smarter Than You Think
- 目前像 OpenAI 的 o1、o3 或 Anthropic 的 Claude 等具備強大數學、寫程式與科學推理能力的大模型，通常是在「基底模型（Base Model）」訓練好之後，再用強化學習（RL）（例如 GRPO 演算法）進行大量的「後續訓練」
- 以前大家覺得如果不做強化學習，模型直接回答（基底模型）就會變笨呢？因為傳統的抽字方法有缺陷。大模型在生成答案時，是「一個字一個字（Token）」往後猜的：
 - 低溫採樣（Low-Temperature Sampling / 貪婪搜尋）：模型在決定下一個字時，只看「現在這一點，哪個字的機率最高」。這就像一個棋手只看眼前這一步，不看後面三步
 - 導致的後果：模型可能在第一步挑選了一個看起來最平庸、最保險的字。但因為第一步走錯了，後面整條推理思路就陷入了死胡同（Critical Window），最後給出錯誤的答案

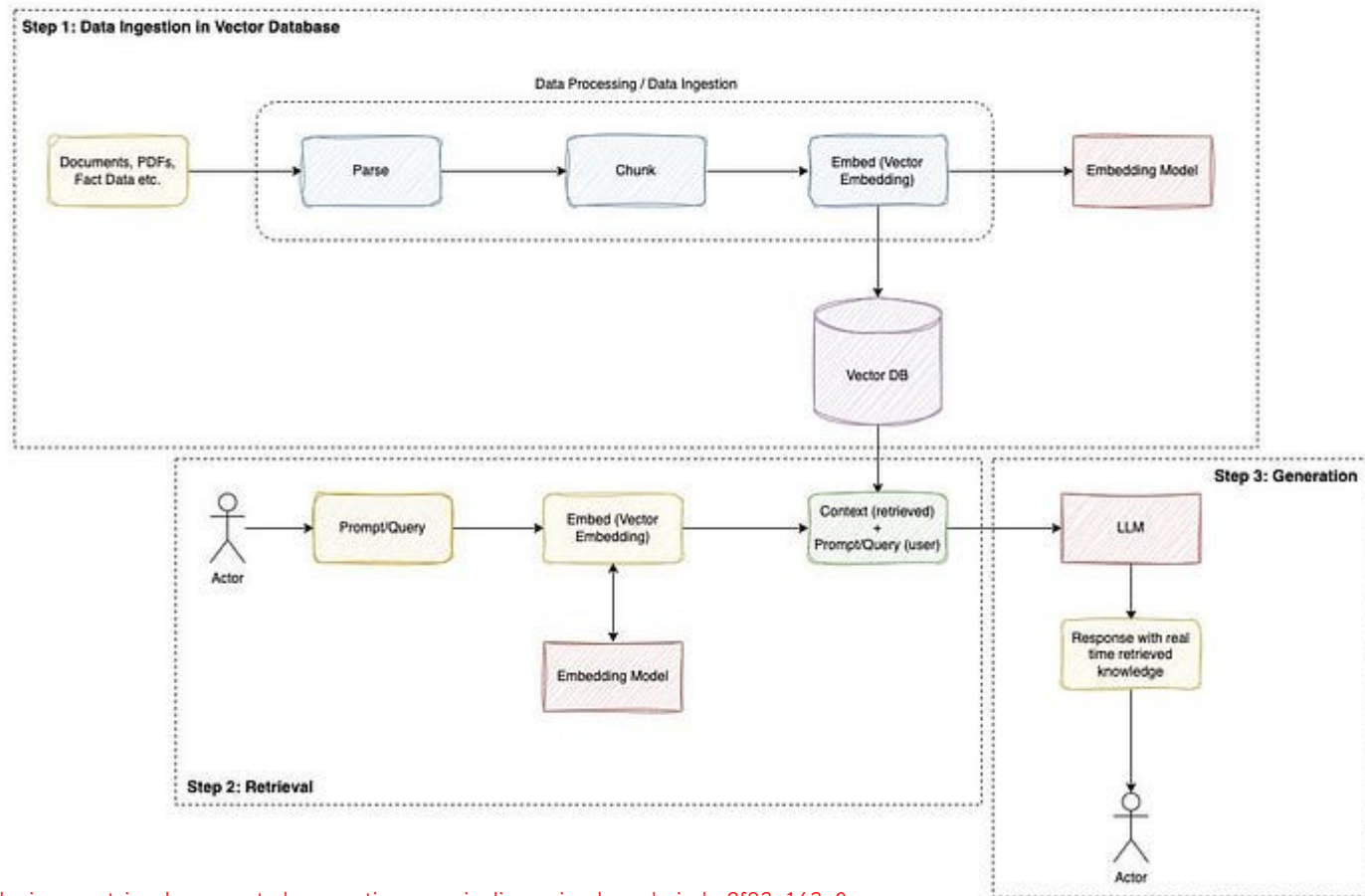
還需要Finetune嗎

- Reasoning with Sampling: Your Base Model is Smarter Than You Think
- 核心方法：冪次採樣（Power Sampling）這篇論文的作者群（來自哈佛大學）提出了一種新方法，稱為「冪次採樣」（Power Sampling）。
- 不改變模型本身，而是改變模型「思考與挑選答案的方式」。簡單來說，它把目光從「下一個字」放大到了「整段解題過程（整個句子）」。
- 核心邏輯可以拆解為兩步：
 - 從「局部最佳」變成「全局規劃」 - 傳統方法是把下一個字的機率放大；而「冪次採樣」是把「整條推理路徑的總機率」拿來做次方放大
 - 利用「馬爾可夫鏈蒙特卡洛（MCMC）」進行反覆調整 - 既然要看整段話的機率，模型就不能只寫一次。演算法會讓模型先寫出一個草稿，然後利用一種數學方法（MCMC 疊代演算法），像修正草稿一樣，在合理的範圍內反覆微調、更新這個答案。經過幾次調整後，最終吐出的答案，其整體的合理性與邏輯正確率會大幅飆升

RAG

Retrieval Augment Generation (RAG)

架構



parse

- Parse 的目的，就是把這些五花八門的檔案格式「打碎並還原」，只撈出「純文字」或「結構化數據」，讓後續的切片（Chunk）和向量化（Embedding）可以順利進行
- Word 檔 (.docx) 或 純文字 (.txt)
 - Parse 程式會直接跳過 Word 的字體大小、顏色設定，直接把裡面的「純文字」一行一行撈出來
- PDF 檔 (.pdf)
 - 把兩欄的文字按照「人類閱讀的順序」拼接好，而不是左邊一行、右邊一行混在一起
 - 自動過濾掉每一頁重複出現的「頁首、頁尾、頁碼」
 - 把複雜的表格轉化成 Markdown 格式（例如 | 欄位A | 欄位B |）-> 讓 AI 看得懂結構
- 圖片檔 (.jpg, .png)
 - 調用 OCR（光學字元識別）、多模態LLM
 - 把圖片中的圖案辨識成文字，把發票上的「2026/06/11」、「金額 \$500」硬是轉成鍵盤敲出來的數字與字元

parse

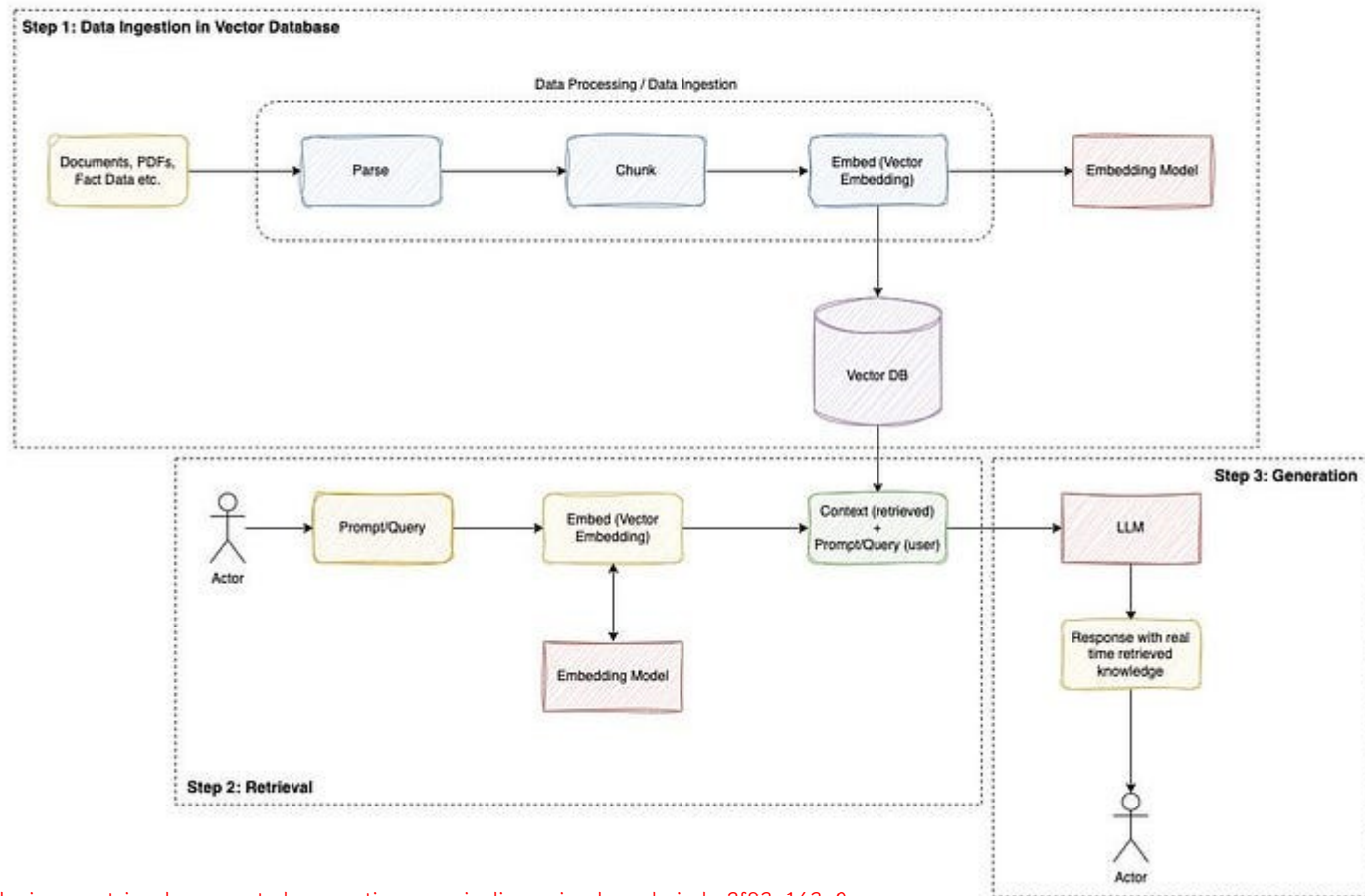
- Docling
- LlamaParse
- Unstructured (unstructured.io)
- MinerU / PDF-Extract-Kit
- RagFlow
- Chonkie
- PaddleOCR
- Qwen-VL

Ex. RAG(parse)

- 示範重點：
 - Docling parse
 - ex_RAG_docling.py

Retrieval Augment Generation (RAG)

架構



chunking

- 為什麼不直接把整篇幾萬字的法律條文或產品說明書丟給 AI，而是要切塊？
 - AI 的胃口有限（Context Window）：雖然現在大模型可以讀很多字，但如果一次餵太多無關的雜訊，AI 容易「分心」或抓錯重點
 - 精準檢索（Precision）：當使用者問問題時，向量資料庫（Vector DB）要幫我們找出「最精準的那一頁、那一段」，而不是把整本 500 頁的手冊砸給使用者。
- Chunk 的目的就是把一大篇文字，依照「語意完整性」和「字數限制」，切成一小段一小段的獨立文字塊

chunking

- 不太好的Chunk 方式（硬體字數切片）
 - 不管三七二十一，每 50 個字就切一刀
 - Chunk 1：「...台灣的珍珠奶茶享譽國際，特別是珍珠的 Q 彈口感是由於特殊的木薯粉配方，此外，台灣的滷肉...」
 - Chunk 2：「...飯也是一絕。滷肉飯的精髓在於那鍋熬煮了十個小時的陳年老滷汁，肥而不膩的五花肉配上...」
 - 「滷肉飯」被一刀兩斷了！如果使用者問「滷肉飯的精髓是什麼？」，AI 搜尋到 Chunk 2，但 Chunk 2 開頭只寫「飯也是一絕」，AI 根本不知道這個「飯」指的是什麼飯，導致回答失敗
- 好一點的 Chunk 方式（語意 / 重疊切片）
 - 依照段落切片，並且讓前後 Chunk 有一點「重疊字數（Overlap）」
 - Chunk 1（珍奶篇）：「台灣的珍珠奶茶享譽國際，特別是珍珠的 Q 彈口感是由於特殊的木薯粉配方，冰鎮後效果更佳。」
 - Chunk 2（滷肉飯篇，含前文重疊）：「（重疊：台灣美食中）滷肉飯也是一絕。滷肉飯的精髓在於那鍋熬煮了十個小時的陳年老滷汁，肥而不膩的五花肉...」
 - 這樣切，每個 Chunk 都有獨立且完整的語意，AI 讀起來最舒服

chunking

「微軟推出了 Copilot 服務。它整合了 GPT 模型，大幅提升了辦公效率。然而，企業在導入時必須注意資料隱私，因為機密資料外洩將導致嚴重後果。」

Chunk Size = 20字，Overlap = 5字

微軟推出了 Copilot 服務。它整合了 GPT 模型，大幅
模型，大幅提升了辦公效率。然而，企業在導
，企業在導入時必須注意資料隱私，因為機
私，因為機密資料外洩將導致嚴重後果。

chunking

「微軟推出了 Copilot 服務。它整合了 GPT 模型，大幅提升了辦公效率。然而，企業在導入時必須注意資料隱私，因為機密資料外洩將導致嚴重後果。」

依據「句號（。）」進行切分

微軟推出了 Copilot 服務。

它整合了 GPT 模型，大幅提升了辦公效率。

然而，企業在導入時必須注意資料隱私，因為機密資料外洩將導致嚴重後果。

chunking

「微軟推出了 Copilot 服務。它整合了 GPT 模型，大幅提升了辦公效率。然而，企業在導入時必須注意資料隱私，因為機密資料外洩將導致嚴重後果。」

計算句子之間的向量相似度（Embedding Similarity）。當前後兩句話的語意產生明顯跳躍（差距超過門檻值）時，才會進行切分。

微軟推出了 Copilot 服務。

它整合了 GPT 模型，大幅提升了辦公效率。

然而，企業在導入時必須注意資料隱私，

因為機密資料外洩將導致嚴重後果。

塊 1：探討產品與功能

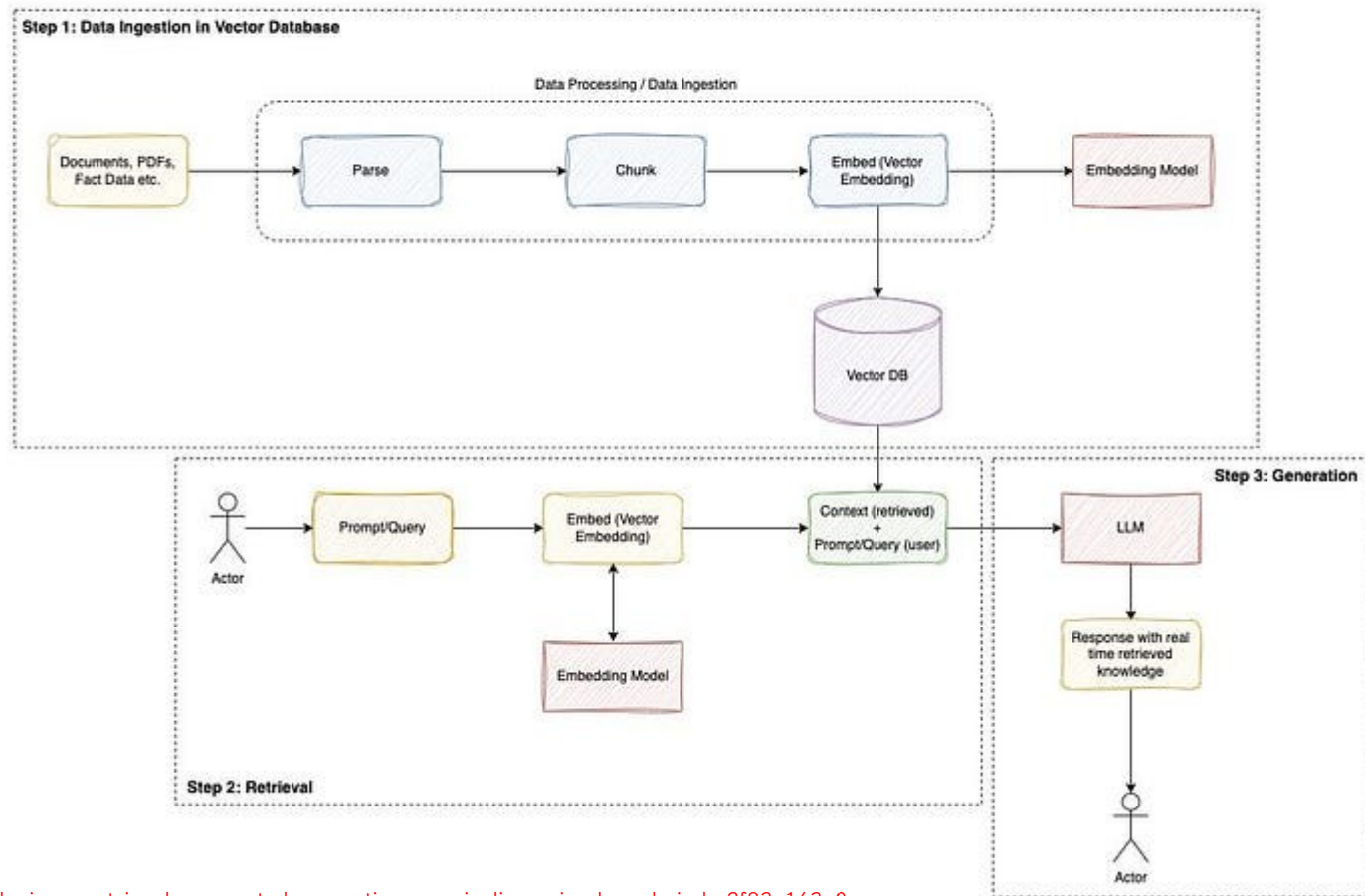
塊 2：探討企業資安風險

Ex. RAG(chunk)

- 示範重點：
 - chunk
 - 使用純 Python 的滑動視窗演算法進行切片（設定每個 Chunk 上限 300 字，重疊 50 字，這在 RAG 實務上是很常用的參數）
 - 將切片後的結果存成一個新檔案 chunks_output.txt，裡面會清晰標註每個 Chunk 的編號與字數
 - 隨機挑選一個 Chunk 丟給你的 Ollama + Open WebUI (gemma4:e4b) 進行測試，確保切片後的品質
 - ex_RAG_chunk.py

Retrieval Augment Generation (RAG)

架構



embedding

- 電腦本質上只懂數字，不懂人類的語言。它不知道「蘋果」和「香蕉」都是水果，也不知道「貓」和「狗」都是寵物。
- Embed 的目的，就是把我們剛剛切好的「中文文字塊 (Chunk)」，翻譯成一串由成百上千個數字組成的陣列（這串數字就叫做「向量 Vector」）。這串數字厲害的地方在於，它代表了這段文字的「語意特徵」
- Embedding（嵌入）是一種將人類語言（文字、圖片、音訊等）轉換成電腦能夠理解的「數字向量」的技術。想像一下，我們把每個詞或句子都變成一個在多維空間中的「座標點」：
 - 例如「東京」變成 [0.12, -0.87, 0.33, ...]，
 - 「大阪」變成 [0.10, -0.83, 0.31, ...]。這個過程就是 Embedding。
- 傳統的關鍵字搜尋只能比對「字」是否相同，但 Embedding 可以理解「意思」是否相近。
 - 例如，搜尋「賞櫻」，Embedding 可以找到「花見」、「櫻花景點」等語義相關的結果
 - 如果兩段文字探討的主題很接近（例如：都是講 AI、都是講美食），它們被翻譯出來的數字串就會非常相似
 - 在數學幾何空間中，這兩串數字的「距離」就會非常近。這就是為什麼 RAG 系統可以在 0.001 秒內，從幾百萬個字裡面精準「撈出關聯資料」的秘密

embedding

	nDCG@10				R@100			
	mE5 _{small}	mE5 _{base}	mE5 _{large}	E5 _{large-instruct}	mE5 _{small}	mE5 _{base}	mE5 _{large}	E5 _{large-instruct}
ar	71.4	71.6	76.0	76.8	96.2	95.9	97.3	97.5
bn	68.2	70.2	75.9	73.9	97.4	96.6	98.2	98.2
en	48.0	51.2	52.9	51.5	85.3	86.4	87.6	88.2
es	51.2	51.5	52.9	53.7	87.6	88.6	89.1	89.3
fa	53.3	57.4	59.0	59.4	90.4	91.2	92.9	92.9
fi	73.3	74.4	77.8	77.3	96.3	96.9	98.1	97.9
fr	47.6	49.7	54.5	53.7	89.5	90.0	90.6	91.7
hi	55.2	58.4	62.0	60.3	91.0	92.6	93.9	94.1
id	50.7	51.1	52.9	52.1	86.2	87.4	87.9	88.4
ja	63.6	64.7	70.6	69.0	95.2	96.0	97.1	96.9
ko	61.2	62.2	66.5	65.3	92.0	91.6	93.4	93.0
ru	59.1	61.5	67.4	67.9	92.2	92.7	95.5	95.4
sw	68.4	71.1	74.9	72.5	94.7	95.6	96.7	97.2
te	81.3	75.2	84.6	83.4	97.6	98.0	99.2	99.0
th	75.0	75.2	80.2	78.6	98.2	98.0	98.9	98.7
zh	45.9	51.5	56.0	56.2	87.9	92.1	93.3	94.9
Avg	60.8	62.3	66.5	65.7	92.4	93.1	94.3	94.6

Table 6: nDCG@10 and R@100 on the development set of the MIRACL dataset.

embedding

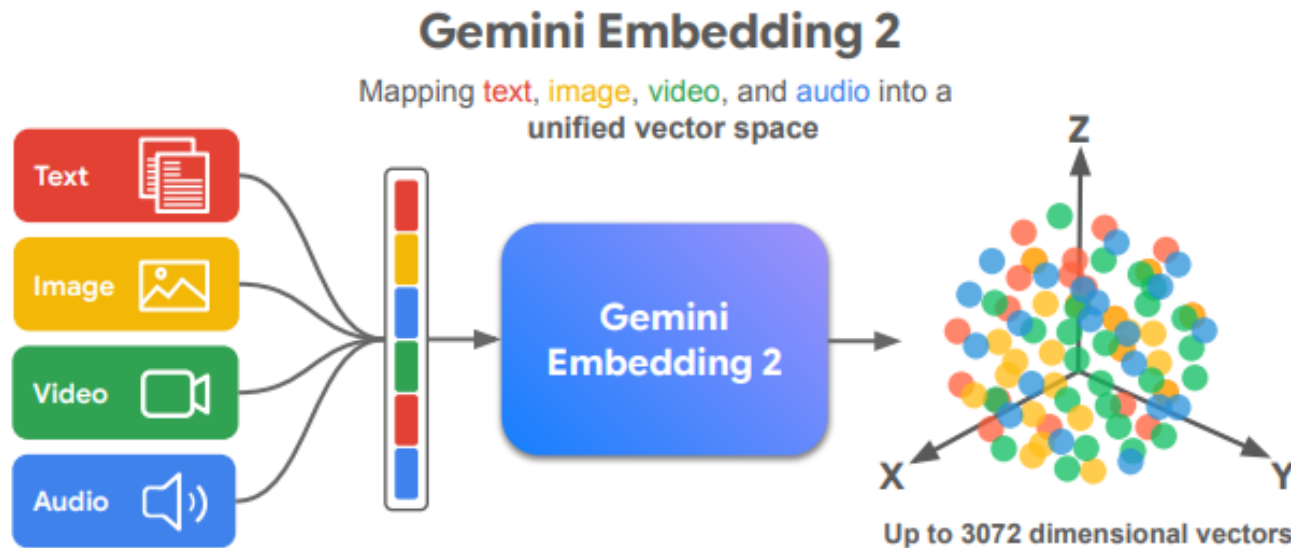


Figure 1 | Conceptual overview of the Gemini Embedding 2 workflow. The model natively processes heterogeneous inputs—text, images, video, audio, documents, and their combinations—mapping them into a single, unified high-dimensional vector space where cross-modal semantic relationships are preserved.

embedding

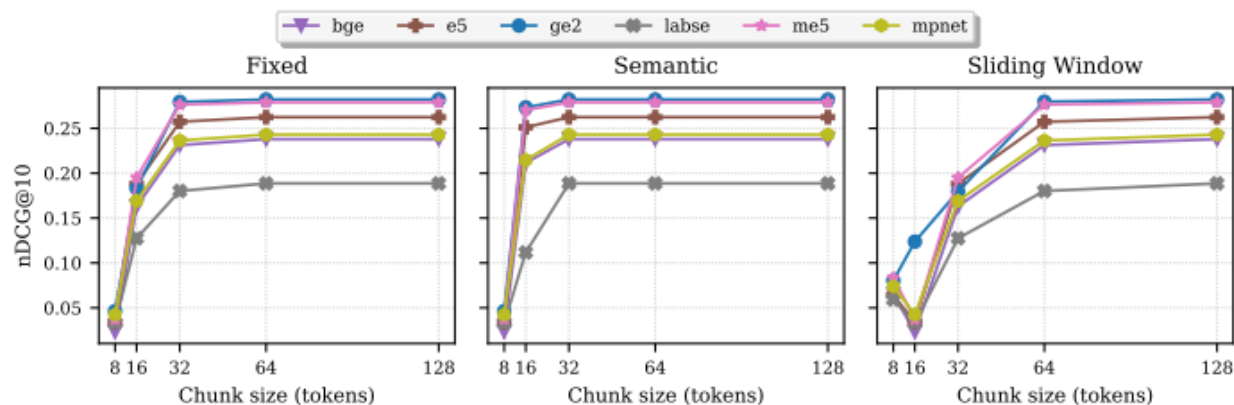


Fig. 2. nDCG@10 vs. chunk size for Fixed, Semantic, and Sliding Window strategies on IT-RAG-Bench. All models saturate at ≥ 32 tokens; quality collapses below 16. Semantic chunking improves over fixed at size 16 for all models except LaBSE. LaBSE’s performance is uniquely strategy-invariant above size 16, suggesting a hard quality ceiling imposed by its training objective.

TABLE II
NDCG@10 ON BEIR SUBSETS. BEST IN **BOLD**, SECOND UNDERLINED. ML = OMITTED DUE TO TOKENISATION INCOMPATIBILITY.

Model	FiQA	NFCorp.	SciFact	COVID	Avg.
GE2	0.582	0.409	0.762	0.799	0.638
mE5-L	<u>0.438</u>	<u>0.341</u>	<u>0.704</u>	<u>0.702</u>	<u>0.546</u>
E5-large	0.411	0.374	0.722	0.646	0.538
BGE-M3	0.366	0.294	0.650	ML	0.437
mMPNet	0.174	0.172	0.317	0.308	0.243
LaBSE	0.069	0.155	0.378	0.151	0.188

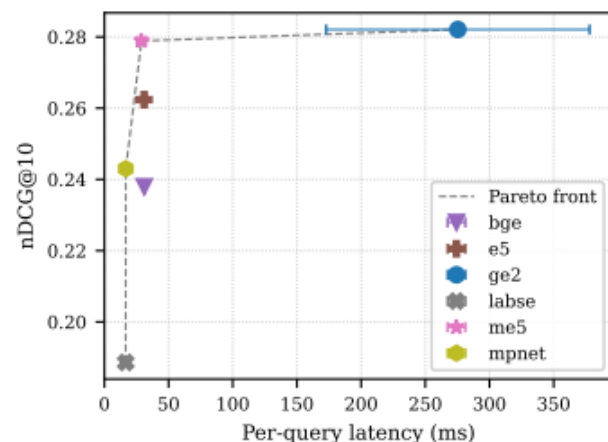


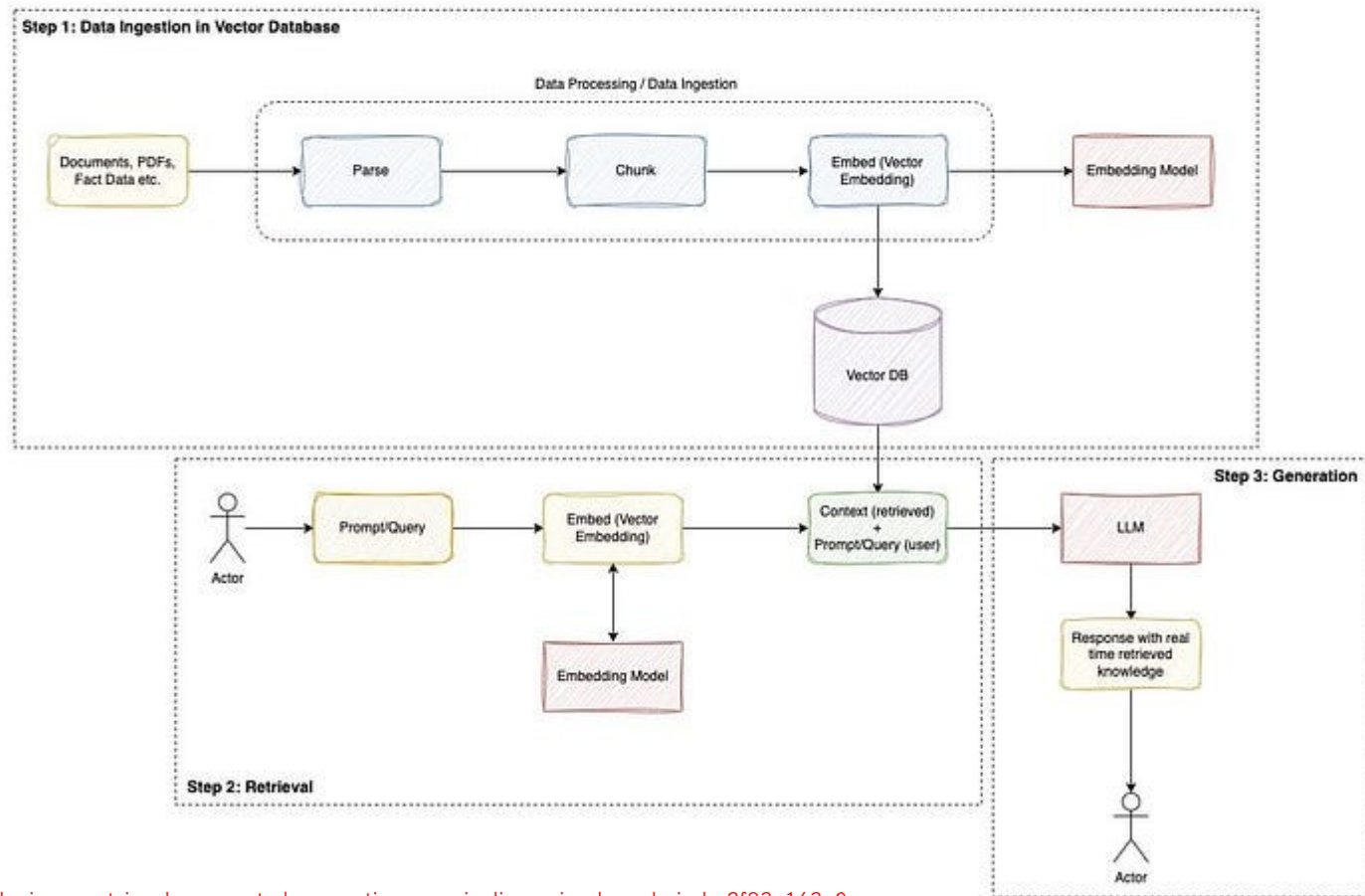
Fig. 1. Per-query latency (ms) vs. nDCG@10 (BEIR avg.). Dashed line: Pareto-optimal front. GE2 achieves the best quality but is an outlier on latency; all local models cluster below 32ms. Error bars: $\pm 1\sigma$ over 50 single-query trials.

Ex. RAG(emb)

- 示範重點：
 - 輸入資料：讀取上一步你存好的 chunks_output.txt
 - Embedding 模型：使用微軟研發、在開源界對中文和多語言支援極佳的 multilingual-e5 系列
 - LLM 總結驗證：不用 LangChain，純 Python 串接，最後把處理好的向量資訊包裝一下，送給 gemma4:e4b 進行語意確認。
 - ex_RAG_embd.py

Retrieval Augment Generation (RAG)

架構



Vector db

- 傳統倉庫（SQL 資料庫）只能用商品編號或名字來找東西
- Vector DB 可以讓你對管理員說：「請幫我拿一桶『吃起來酸酸甜甜、帶有一點辣味』的肉丁」，管理員就能在幾毫秒內把最接近那個風味的肉丁全部搬給你
- 傳統的資料庫（如 MySQL, MongoDB）是透過「關鍵字精準比對」來查資料。你搜尋「AI」，它就只會找含有「AI」這兩個字的文章
- 向量資料庫儲存的是我們剛剛在上一步算出來的那串 768 維的數字（向量）。它的核心超能力叫做「近似最近鄰搜尋（ANN, Approximate Nearest Neighbor）」。它不比對英文字母，而是在數學空間中計算兩串數字之間的「距離」或「夾角（Cosine Similarity）」
- 主要的目的
 - 當使用者問：「智慧型手機本地端運算有什麼好處？」，向量資料庫會把這句話也轉成 e5 向量，然後在資料庫裡比對，最後幫你撈出「隱私與安全性提升、邊緣運算普及」那塊 Chunk，即使那塊 Chunk 裡根本沒有出現「好處」這兩個字

Vector db

- 一種專門用來儲存、索引和快速搜尋「高維度向量 (Embeddings)」的資料庫。它將文字、圖片或音訊透過 Embedding 模型轉換為一串數字 (向量)，並在多維空間中定位。
- 傳統資料庫依賴「關鍵字完全符合」，而 Vector DB 依賴「語意空間距離」。算式距離越近，代表語意越相似。
- 模型選擇：實作中英雙語或純中文 RAG 時，必須選擇支援 Multi-lingual (多語言) 或專門針對中文訓練的 Embedding 模型 (如 BGE、M3 等)，否則中英混雜時效果會打折扣。
- 主要用途
 - RAG (檢索增強生成)：當作 LLM 的「外接大腦/長期記憶體」，精準提供正確的背景知識，解決 LLM 的幻覺問題。
 - 相似度搜尋：實現「換句話說也能懂」的語意搜尋、圖片以圖搜圖、或是電商的個人化推薦系統。

Vector db

- 向量資料庫的研究重點主要集中在以下幾個方向
 - 與大型語言模型（LLM）的深度整合：這是目前最熱門的方向。研究重點在於如何利用向量資料庫解決 LLM 的幻覺（hallucination）、知識過時、以及高昂的商業應用成本等問題。具體應用包括檢索增強生成（RAG）、長期記憶、以及作為語義快取（Semantic Cache）來降低成本。
 - 儲存與檢索技術的挑戰與優化：由於向量資料具有高維度和稀疏性的特性，如何有效率地進行儲存、索引和檢索是核心挑戰。研究重點包括開發新的索引結構（如 HNSW、IVF）、近似最近鄰（ANN）搜尋演算法，以及提升查詢效率和系統可擴展性。
 - 向量壓縮與量化技術：為了在本地端或資源受限的環境中運行大規模的 RAG 系統，研究重點在於開發新的向量壓縮技術，以在不顯著損失搜尋精度的情況下，大幅降低記憶體佔用和儲存成本。
 - 混合搜尋（Hybrid Search）：純粹的語義搜尋在處理特定關鍵字（如產品名稱、縮寫）時可能表現不佳。因此，結合詞彙搜尋（如 BM25）與語義搜尋的混合搜尋模式已成為業界主流，旨在同時兼顧精確匹配和語義理解。

Vector db

DB	Type	Pro	Con	Applicaion
Chrom	專屬/輕量嵌入式	最容易上手、免安裝伺服器、Python 幾行代碼直接在記憶體跑	資料量大（百萬級以上）時效能較弱	個人測試、Demo、中小型專案（首選）
Qdrant	專屬/高效能型	用 Rust 寫的、速度極快、篩選功能（Filtering）強大	需要架設 Docker 伺服器	中大型企業專案、重視速度與精準度
Milvus, PGVector	企業級/擴充型	Milvus 適合海量數據；PGVector 是基於 PostgreSQL 擴充，老牌穩定	架構複雜、學習曲線較陡	已經在使用大數據架構的成熟企業

Vector db

Table 2: VDBMS features; example use-cases are based on a product’s documentation’s use-case examples as of August 2023

	License	First release	Querying with metadata
Pinecone	Proprietary	2021	rich expressions
Chroma	Apache 2.0	2023	rich expressions
Milvus	Apache 2.0	2019	rich expressions
Weaviate	BSD 3-clause / proprietary	2019	supported
Qdrant	Apache 2.0 / proprietary	2022	rich expressions
Deep Lake	Apache 2.0 / proprietary	2019	rich expressions
	Integration	Querying	Example use-cases
Pinecone	OpenAI, LangChain, others	Java, Python, C#, several others	chatbots, image search
Chroma	LangChain, LlamaIndex	JavaScript, Python, Ruby, others	chatbots
Milvus	OpenAI, LangChain, others	Java, Python, Go, Node.js	chatbots, image/audio/video search
Weaviate	OpenAI, Cohere, PaLM	Java, JavaScript, Python, Go, GraphQL	chatbots, image search
Qdrant	OpenAI, LangChain, others	Python, JavaScript, Go, Rust	chatbots, image search
Deep Lake	LlamaIndex, LangChain	Python, SQL-like TQL	image search

Vector db

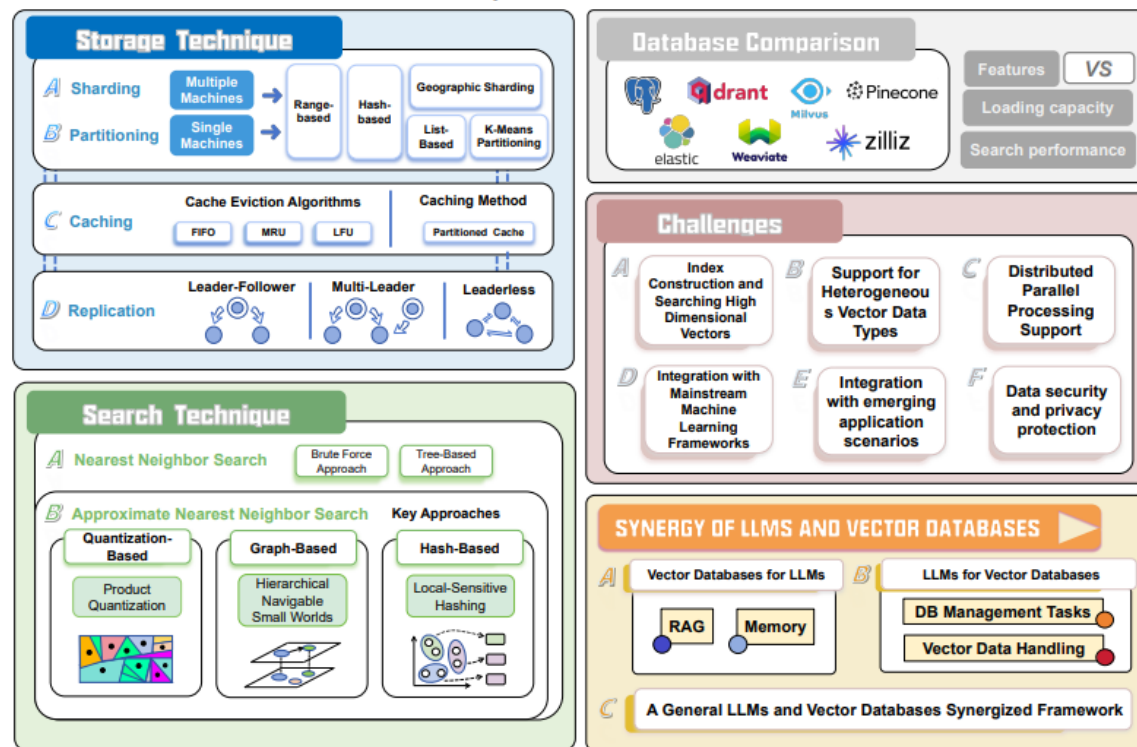


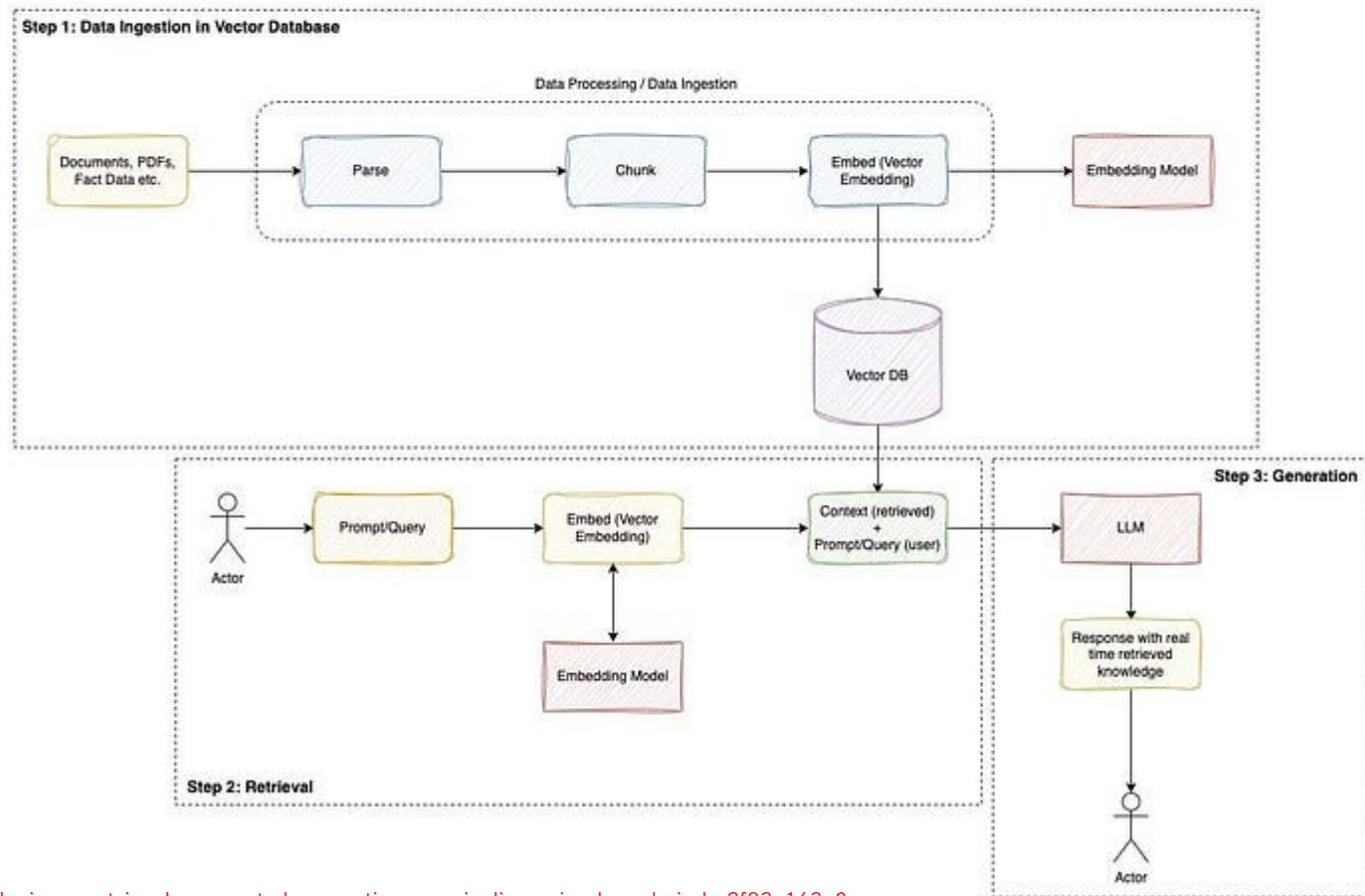
Figure 1: Framework overview of this survey structure covering Storage Techniques, Search Techniques, Database Comparison, Challenges, and the Synergy of Large Language Models (LLMs) with VDBs. Each section represents a fundamental facet of the operation and integration of modern VDBs within advanced AI technologies.

Ex. RAG(vector DB)

- 示範重點：
 - 挑選最常用、最容易上手、完全不需要在電腦裝任何資料庫軟體的 Chroma 來實作。它就像是向量資料庫界的 SQLite，檔案直接存在你的專案資料夾裡
 - 讀取你的 chunks_output.txt，用微軟 e5 轉成向量，存入 Chroma DB
 - 接著模擬使用者提問，從 DB 撈出正確答案，最後送去給 gemma4:e4b 生成最終回應
 - ex_RAG_vectordb.py

Retrieval Augment Generation (RAG)

架構



原理

Cosine similarity

In case you missed [the first article](#), here's a quick recap of how cosine similarity works:

$$d(A, B) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \times \|\mathbf{B}\|}$$

In this and following formulae, we'll use $d(A, B)$ to represent the distance d between vectors A and B .

In cosine similarity, the final value is the cosine of the angle between two vectors. The similarity value is calculated by taking the dot product of the two vectors ($\mathbf{A} \cdot \mathbf{B}$) and dividing it by the product of the magnitudes of the two vectors ($\|\mathbf{A}\| \times \|\mathbf{B}\|$).

原理

- 向量比較是工作流程（如 RAG）的技術基礎，用於衡量文本或資訊片段間的「意義相似度」
- Vector 可以被視為高維空間中的點，其中數組的每個數值都是該維度的坐標
- LLM embeddings 本身就是用來代表文本「意義」的數值向量
- 餘弦相似度是目前業界最推薦的向量比較方式
- 由於許多模型（如 OpenAI）會將嵌入向量正規化到單位長度（Magnitude = 1），計算可以簡化為兩個向量的點積Dot Product
- 點積計算的運算步驟（乘法和加法）相較於其他方法更簡潔，這在計算效率上佔有微小優勢
- 儘管其他函數（曼哈頓距離、切比雪夫距離）是可用的，但由於點積（基於餘弦相似度）計算過程的簡單性與已證明的可靠性，在比較 LLM 嵌入時，餘弦相似度仍然是業界（特別是根據 OpenAI 文件）最常用且推薦的選擇

原理

- 在 AI 的世界裡，文字或圖片會被轉換成一串數字（這叫做「向量 Embedding」）。餘弦相似度在計算兩者像不像時，看的是這兩串數字在空間中的「角度」
- 餘弦相似度要拿到滿分（相似度 = 1），條件非常嚴苛：這兩組數字必須成「完美的正比例」
- 舉個生活中的例子（看電影評分）
 - 影評人 A 給三部電影的評分是：[1分, 2分, 3分]
 - 影評人 B 給三部電影的評分是：[2分, 4分, 6分]（剛好是 A 的 2 倍）
 - 餘弦相似度：100% 完美一樣！因為比例完全相同
 - 影評人 C 給的評分是：[1分, 5分, 9分]
 - 雖然 C 跟 A 兩人都一致認為「第三部最好看、第二部次之、第一部最難看」（趨勢完全相同），但因為數字不成固定比例，餘弦相似度就會扣分，認為他們沒那麼像
- 論文作者指出，這種要求「嚴格線性比例」的數學限制（源自於古典數學的柯西-施瓦茨不等式），並不符合人類真實世界中複雜、非線性的思考方式

原理

- 論文的新發明：recos
- recos 的核心邏輯不再是看硬邦邦的「角度與比例」，而是看「排序的契合度」（Ordinal Concordance）
- 只要你和我的品味排序是一樣的（都是覺得 3 號電影 > 2 號電影 > 1 號電影），不管你的分數跳躍得有多誇張（比如 1 分、20 分、100 分），recos 都能夠抓到這個隱藏的默契，判定你們的語意是非常相似的
- recos 釋放了被餘弦相似度綁死的限制，改去捕捉數據中「相對大小的趨勢」，因此能包容更多真實世界中的變化
- 作者拿了 11 種不同的主流 AI 嵌入模型（包含靜態的、上下文相關的、通用型的 Model），並在標準的「語意文本相似度 (STS)」數據集上進行大考驗。結果發現：recos 計算出來的相似度分數，比傳統的餘弦相似度更接近「人類真正的評判感受」

原理

recos (最包容)：只要兩個向量的「數字排序趨勢」一致，它就會給高分
 cos (標準餘弦，中等嚴格)：必須「數字成固定比例（線性相關）」才會給高分
 decos (退化餘弦，最嚴格)：必須數字「完全一模一樣（或完美相反）」才會給高分
 從數學的理論上來說，它們的分數大小關係是： $|\text{decos}| \leq |\text{cos}| \leq |\text{recos}|$ 。這意味著 recos 擁有最寬廣的捕捉能力。

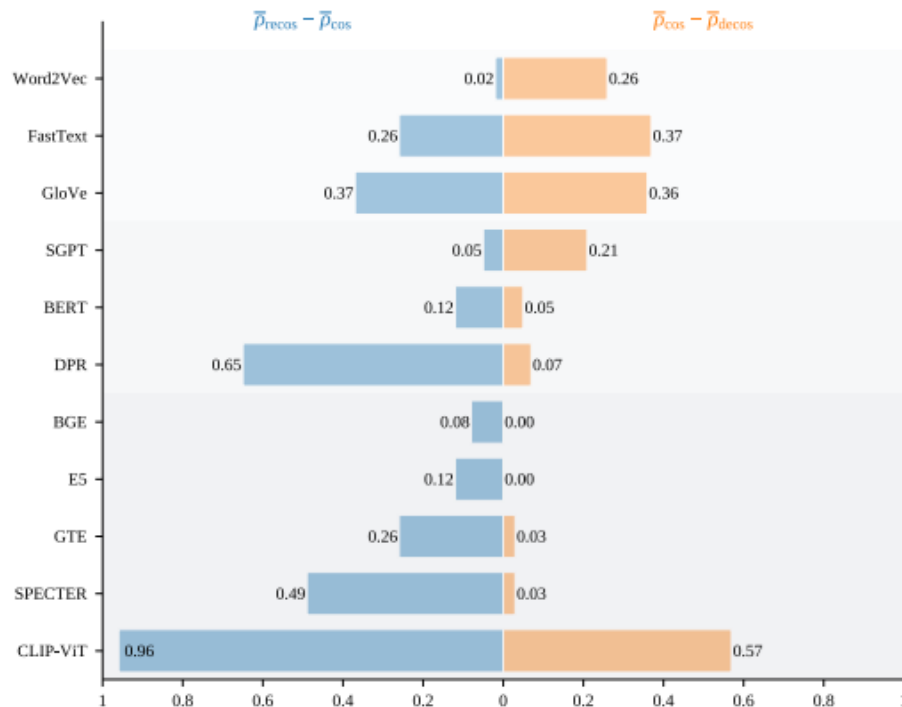
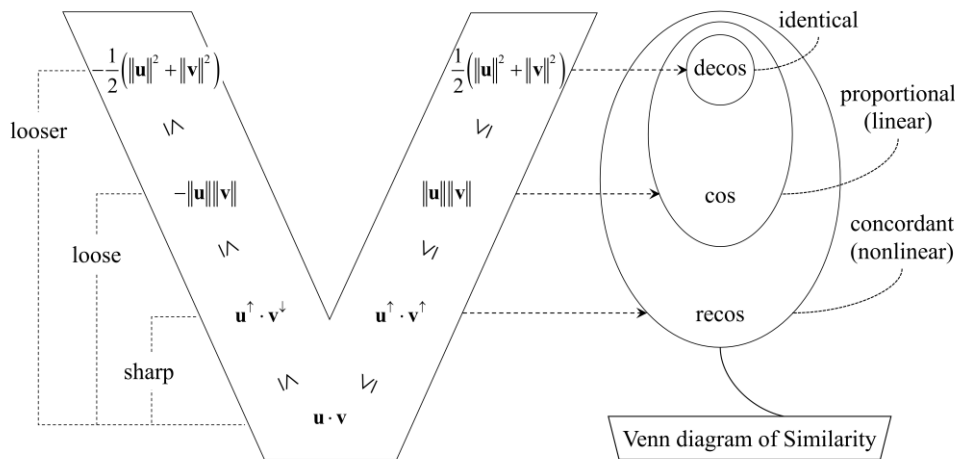


Figure 2: Performance gains of recos over cos and cos over decos

Ex. RAG(similarity)

- 示範重點：
 - 說明 Cosine Similarity（餘弦相似度）的意義（根據您提供的網頁內容與概念）。
 - 以 LLM 做應用時的例子（展示如何將文字轉換為向量並計算相似度）。
 - `ex_RAG_cosince_similarity.py`

主流RAG

- REALM

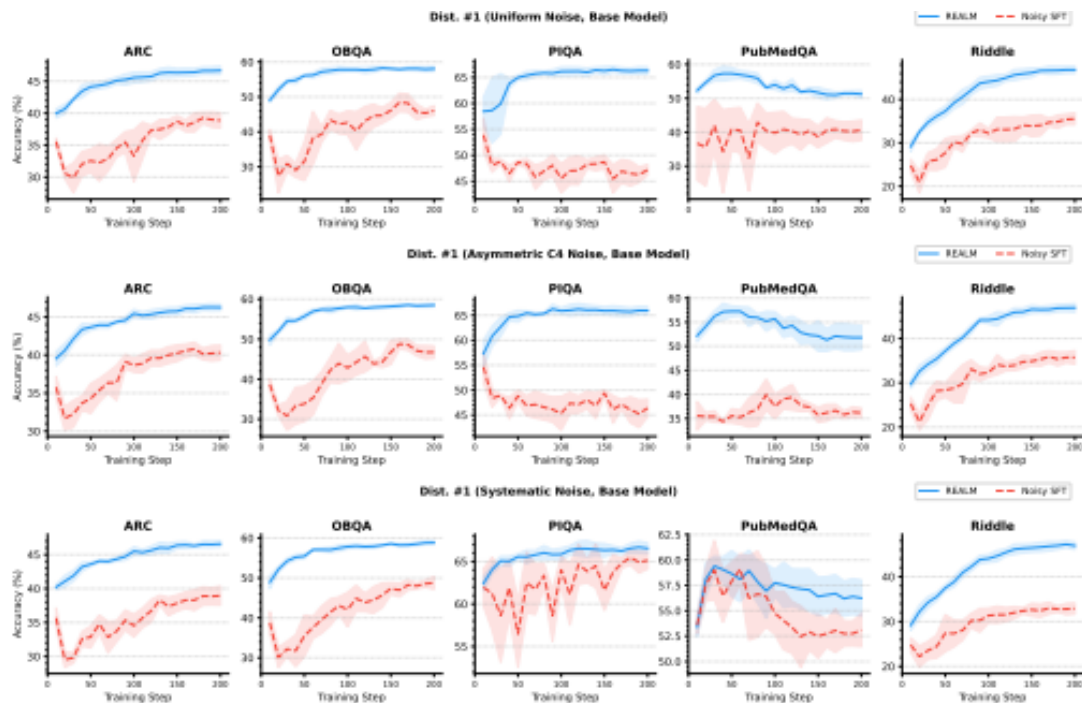


Figure 1: Test accuracy (%) over training steps under Dist. ① across all three noise types (rows) and five datasets (columns), for the Base model. Shaded bands denote $\pm$ std over five runs.

主流RAG

- RAPTOR

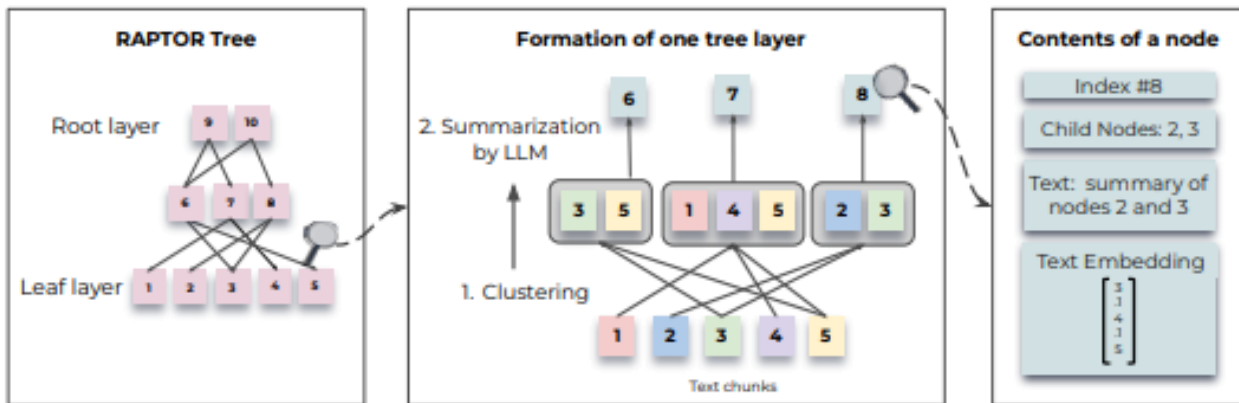


Figure 1: **Tree construction process:** RAPTOR recursively clusters chunks of text based on their vector embeddings and generates text summaries of those clusters, constructing a tree from the bottom up. Nodes clustered together are siblings; a parent node contains the text summary of that cluster.

主流RAG

- GraphRAG

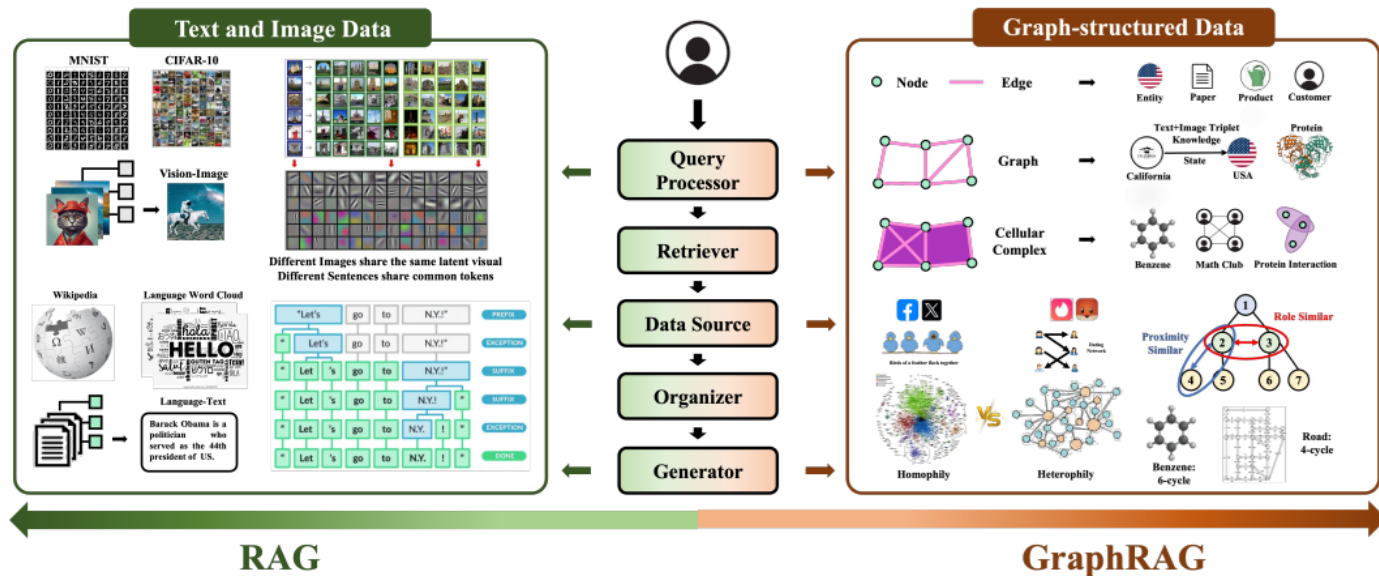


Figure 1: RAG works on text and images, which can be uniformly formatted as 1D sequences or 2D grids with no relational information. In contrast, GraphRAG works on graph-structured data, which encompasses diverse formats and includes domain-specific relational information.

主流RAG

- HippoRAG

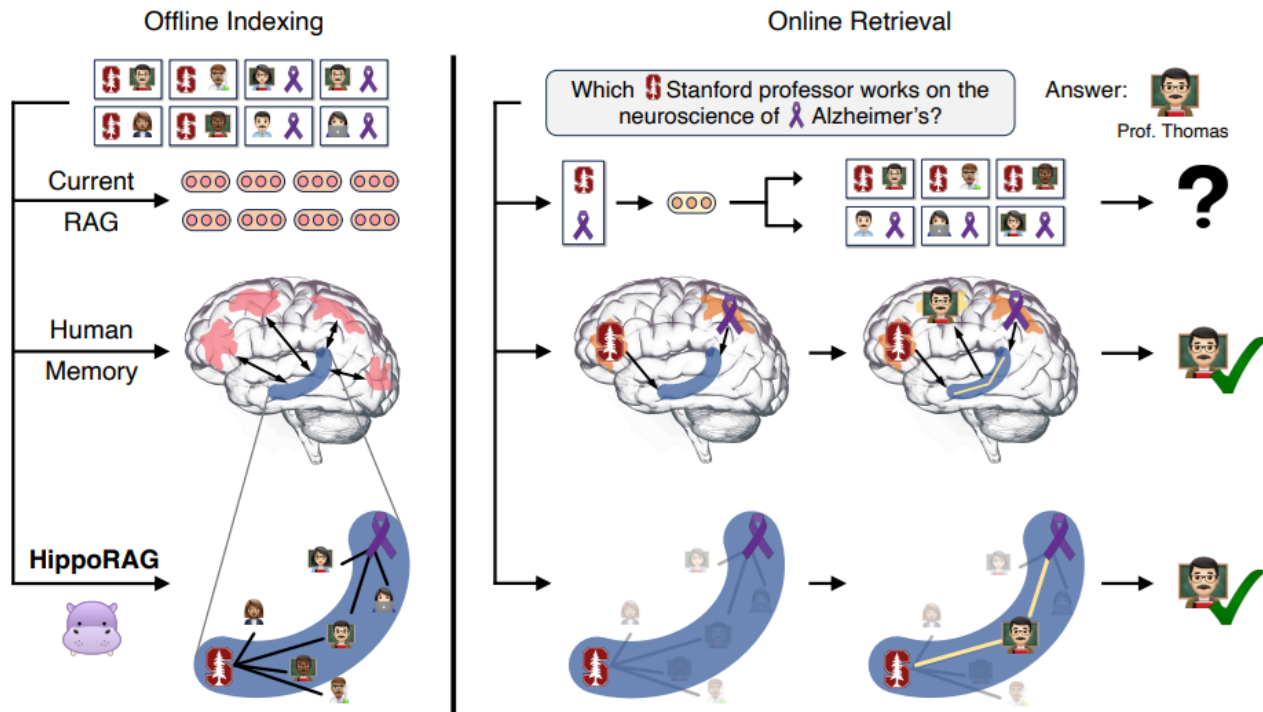
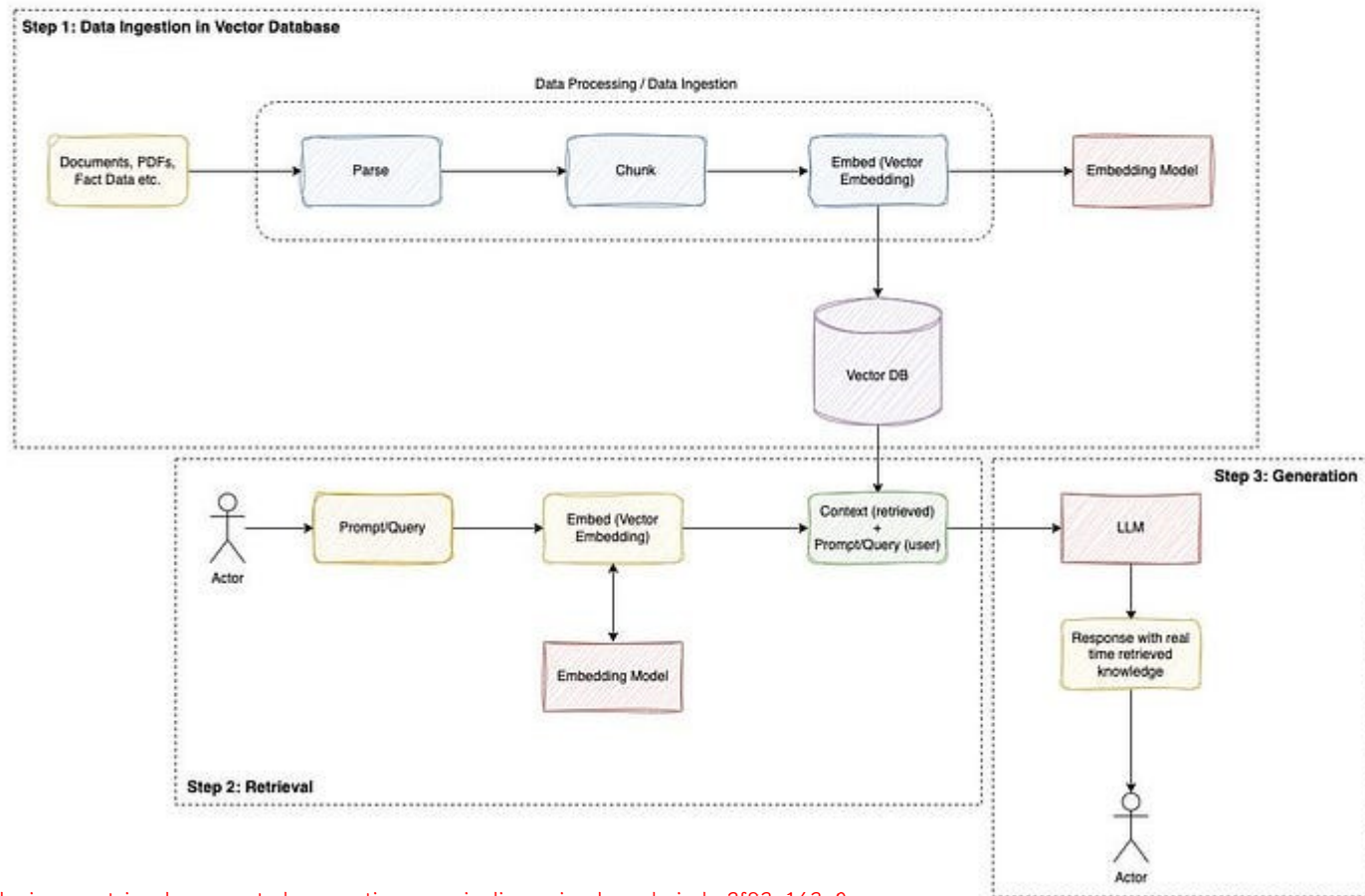


Figure 1: **Knowledge Integration & RAG.** Tasks that require knowledge integration are particularly challenging for current RAG systems. In the above example, we want to find a *Stanford* professor that does *Alzheimer's* research from a pool of passages describing potentially thousands *Stanford* professors and *Alzheimer's* researchers. Since current methods encode passages in isolation, they would struggle to identify *Prof. Thomas* unless a passage mentions both characteristics at once. In contrast, most people familiar with this professor would remember him quickly due to our brain's associative memory capabilities, thought to be driven by the index structure depicted in the C-shaped hippocampus above (in blue). Inspired by this mechanism, **HippoRAG** allows LLMs to build and leverage a similar graph of associations to tackle knowledge integration tasks.

Retrieval Augment Generation (RAG)

架構



rag

- 輕量化 Naive RAG 系統實作
- 基於本地 Gemma 4B 與動態向量庫的自主知識檢索方案系統核心
- 全在地化 (Local-First) :
 - 完全執行於本地端 (Ollama + Gemma 4b) ，確保資料隱私不外洩。
 - 動態文件感知：支援中英文雙語純文字 (.txt) ，啟動時自動掃描新檔案，並提供一鍵重建索引功能。
 - 彈性檢索 (Top-N)：使用者可於 UI 自行調整檢索篇數（上限 5 篇），精準控制上下文長度。
 - 前端介面：使用 Gradio 打造直覺、精簡的 Web UI，支援文件上傳與參數調整。
 - 向量資料庫：採用輕量級 ChromaDB / FAISS 進行文字向量化與相似度檢索。
 - 生成模型：串接本地端 ollama+openwebui
 - ex_RAG_simple_rag.py

Ex. RAG(all)

- 示範重點：
- streamlit rag service
- streamlit run ex_RAG_app.py



🧠 RAG 知識庫系統 & 向量資料管理中心

💬 與你的自建知識庫對話

🔴 什麼是Liquid Neural Networks?

🟡 根據您提供的參考資料，並沒有關於「Liquid Neural Networks」的相關資訊。

🔴 什麼是Liquid Neural Networks?

➤ 📁 RAG 檢索引擎後台

🟡 參考資料中提到，Liquid Neural Networks (LNNs) 是用來解決某些限制問題的技術，它們透過連續時間動力學 (continuous-time dynamics) 來處理。

請問關於已匯入文件的任何問題...

記憶

記憶

- 大語言模型（LLM）本質上是「無狀態」的（Stateless）。
- 每一次對話都是獨立事件，如果不給它上下文，它轉頭就會忘記你是誰。
- 記憶機制：
 - 在 RAG 與 AI Agent 的架構中，記憶機制就是動態管理上下文（Context Window）的技術。
 - 負責決定「哪些過去的對話與外部資料，應該在這一刻被塞進 LLM 的腦袋裡」。
- 核心用途：維持對話連貫性：讓 Agent 記住使用者的名字、偏好以及上一步聊了什麼。
- 突破上下文限制：LLM 的 Context 雖然越來越大（如 Gemma 4 支援到 128K），但塞滿會導致推理變慢、成本暴增、以及「迷失在上下文中央」（Lost in the Middle）的問題。

記憶

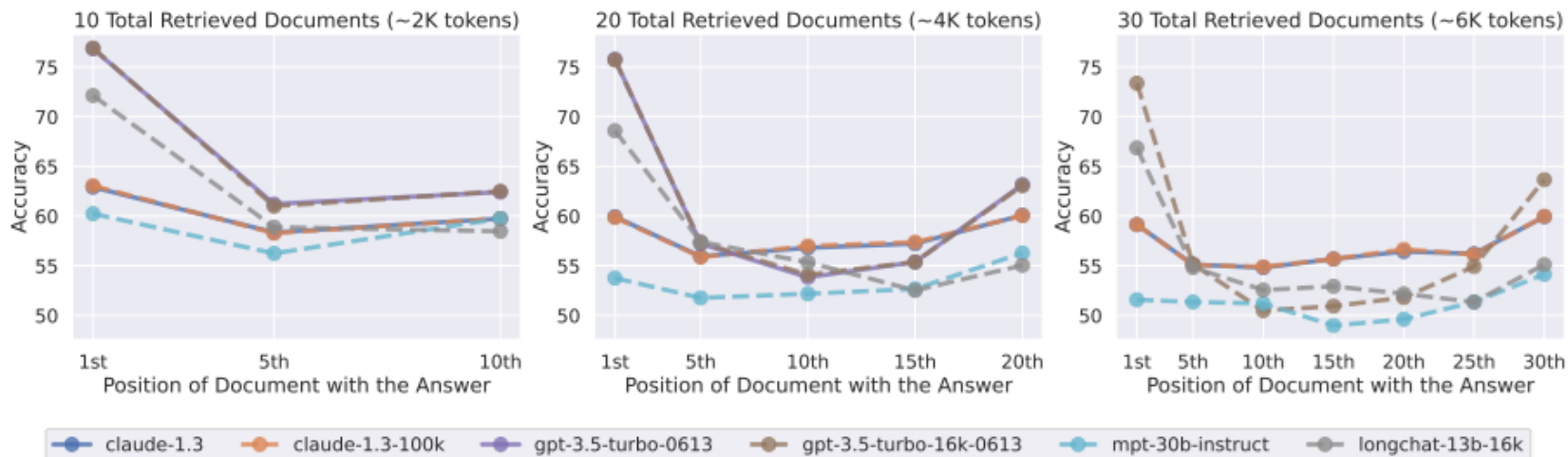


Figure 5: The effect of changing the position of relevant information (document containing the answer) on multi-document question answering performance. Lower positions are closer to the start of the input context. Performance is highest when relevant information occurs at the very start or end of the context, and rapidly degrades when models must reason over information in the middle of their input context.

記憶

- 記憶的階層架構——短期、長期與壓縮AI 的記憶與人類極為相似，主要分為以下三種層次
- 短期記憶（Short-term Memory）
 - 當前對話（Session）中最近的幾輪原始對話紀錄。
 - 處理當前的語境，例如使用者說「幫我修改上一段話」，AI 必須知道「上一段話」是什麼。
- 長期記憶（Long-term Memory）
 - 跨越歷史、過去好幾天甚至幾個月前的對話精髓，或是外部知識庫（RAG）。
 - 記住使用者的長期人設、核心習慣，或大型公司的專業文檔。
- 記憶壓縮（Memory Compression / Summarization）
 - 隨著對話變長，將逐漸淡出短期記憶的「原始對話」，透過 LLM 提煉成「重點摘要」或「結構化標籤」。用途：在不遺忘核心事實的前提下，清空緩衝區，節約 Token 空間。

記憶

- 記憶的實作心法——怎麼記？怎麼用？
- 怎麼記（Ingestion & Storage）：
 - 短期：直接追加（Append）到內存清單（List/Queue）中。
 - 長期：透過 Embedding 模型將文字轉為向量，存入向量資料庫（Vector DB）；或是將結構化記憶存入 Key-Value 庫。
 - 壓縮時機：當短期記憶的 Token 數超過設定閾值（例如超過 2000 tokens），觸發 LLM 進行「摘要提取」，並將摘要更新至長期記憶。
- 怎麼用（Retrieval & Injection）：
 - 動態組裝 Prompt：每次使用者輸入新問題時，系統去短期記憶拿取最新的 N 輪對話，去長期記憶用向量檢索（RAG）找出相關背景，並加上「已被壓縮的歷史摘要」。將這三者拼接成一個最終的 Prompt 送給 LLM，實現「既看得到過去、又抓得到重點、還能聚焦當下」的完美回覆。

記憶

- LM 記憶機制最主流且最新的研究方向與實作方向，已從單純的 RAG 或向量檢索，進化到以下幾個核心領域：
 - 記憶的結構化與圖譜化 (Structured & Graph Memory)：不再只是儲存文字片段，而是將記憶轉化為知識圖譜 (Knowledge Graph) 或結構化事件。這使得 Agent 能進行更複雜的推理，例如理解事件之間的因果關係。代表研究如 MemGPT (後續演進) 與 GraphRAG 的深度整合。
 - 分層與動態記憶管理 (Hierarchical & Dynamic Memory Management)：系統會根據記憶的「重要性」、「時效性」和「相關性」動態調整其在 Prompt 中的位置與權重。最新的實作會使用一個「記憶控制器 (Memory Controller)」來決定哪些記憶需要被壓縮、檢索或遺忘。
 - 記憶的反思與合成 (Memory Reflection & Synthesis)：Agent 不僅記錄事實，還會定期對過去的記憶進行「反思」，提煉出更高層次的結論、人格特質或長期目標。這類似於人類的「做夢」或「日記反思」過程，能讓 Agent 的行為更具一致性與深度。

記憶

MemGPT: Towards LLMs as Operating Systems

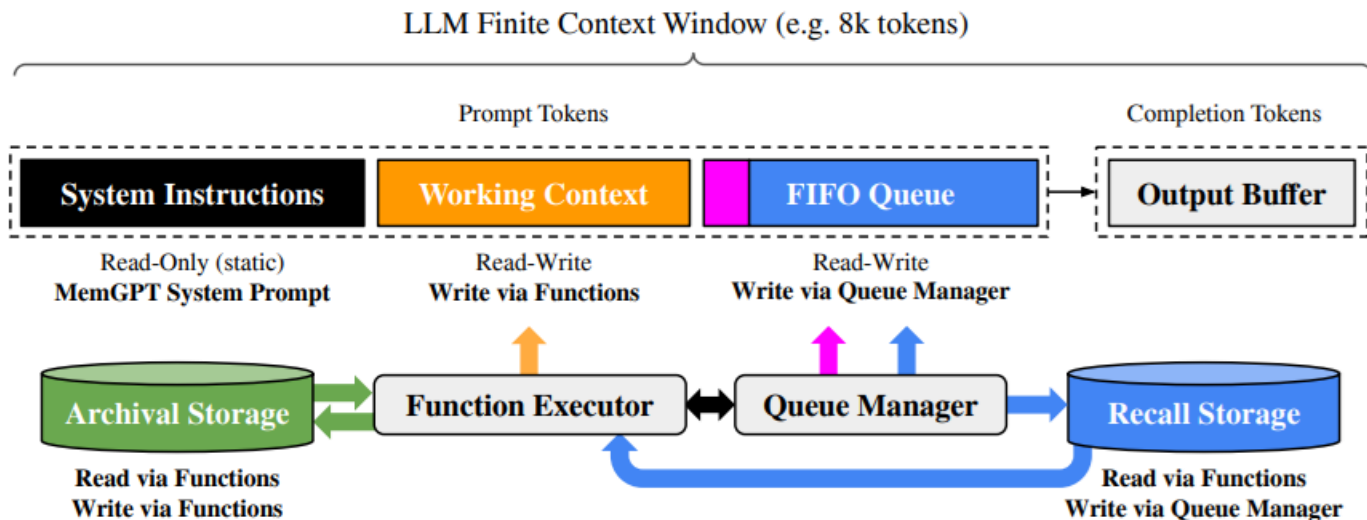


Figure 3. In MemGPT, a fixed-context LLM processor is augmented with a hierarchical memory system and functions that let it manage its own memory. The LLM’s prompt tokens (inputs), or *main context*, consist of the system instructions, working context, and a FIFO queue. The LLM completion tokens (outputs) are interpreted as function calls by the function executor. MemGPT uses functions to move data between main context and *external context* (the archival and recall storage databases). The LLM can request immediate follow-up LLM inference to chain function calls together by generating a special keyword argument (`request_heartbeat=true`) in its output; function chaining is what allows MemGPT to perform multi-step retrieval to answer user queries.

記憶

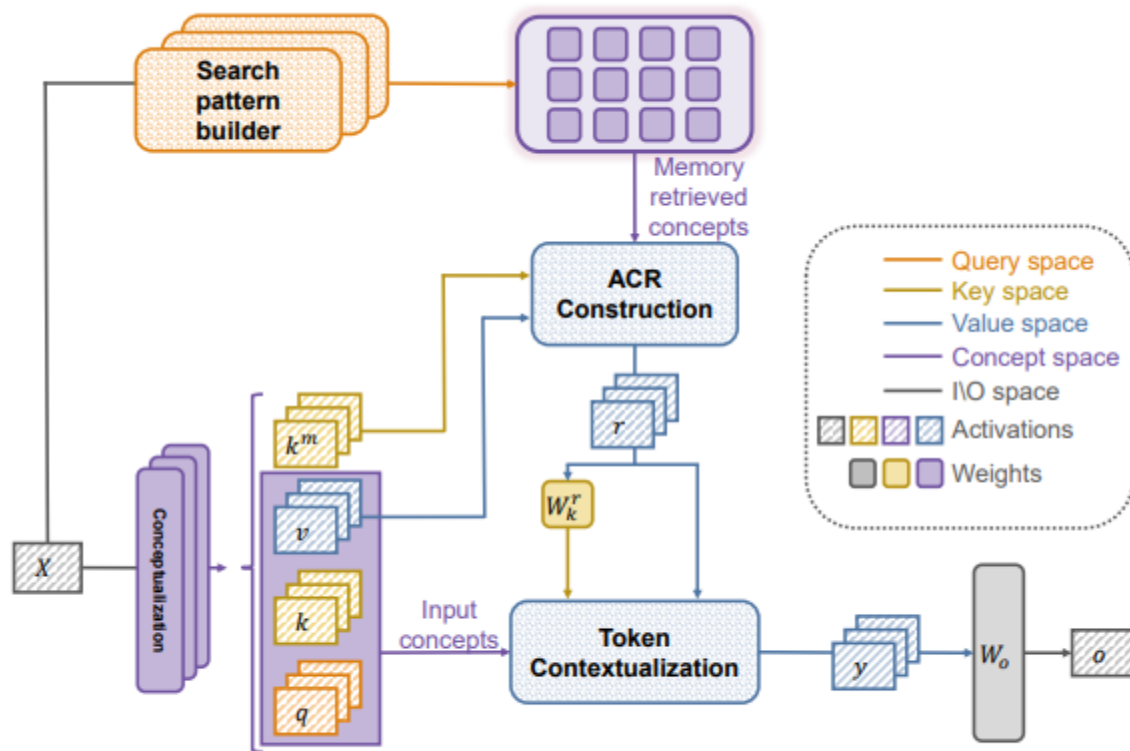


Figure 2: High level architecture of MANAR

記憶

Table 6: MemoryArena: average reward (%) for first and second half of steps across all three categories. Q0 excluded. Travel Planner: Q1–Q3 vs Q4–Q7; Web Shopping: Q1–Q2 vs Q3–Q5; Progressive Search: Q1–Q4 vs Q5–Q8.

System	Travel Planner		Web Shopping		Prog. Search	
	Q1–Q3	Q4–Q7	Q1–Q2	Q3–Q5	Q1–Q4	Q5–Q8
HippoRAG	69.4	69.6	64.1	62.1	2.4	2.2
Mem0	75.6	76.7	72.1	68.2	67.8	72.6
Nemori	67.9	68.7	60.7	61.3	0.7	0.6
A-Mem	71.9	75.5	63.9	45.2	73.2	82.0
Full-Context	69.9	71.5	68.1	69.8	71.7	81.3
GAAMA (ours)	69.1	73.2	72.5	72.4	72.7	81.4

Ex. Memory

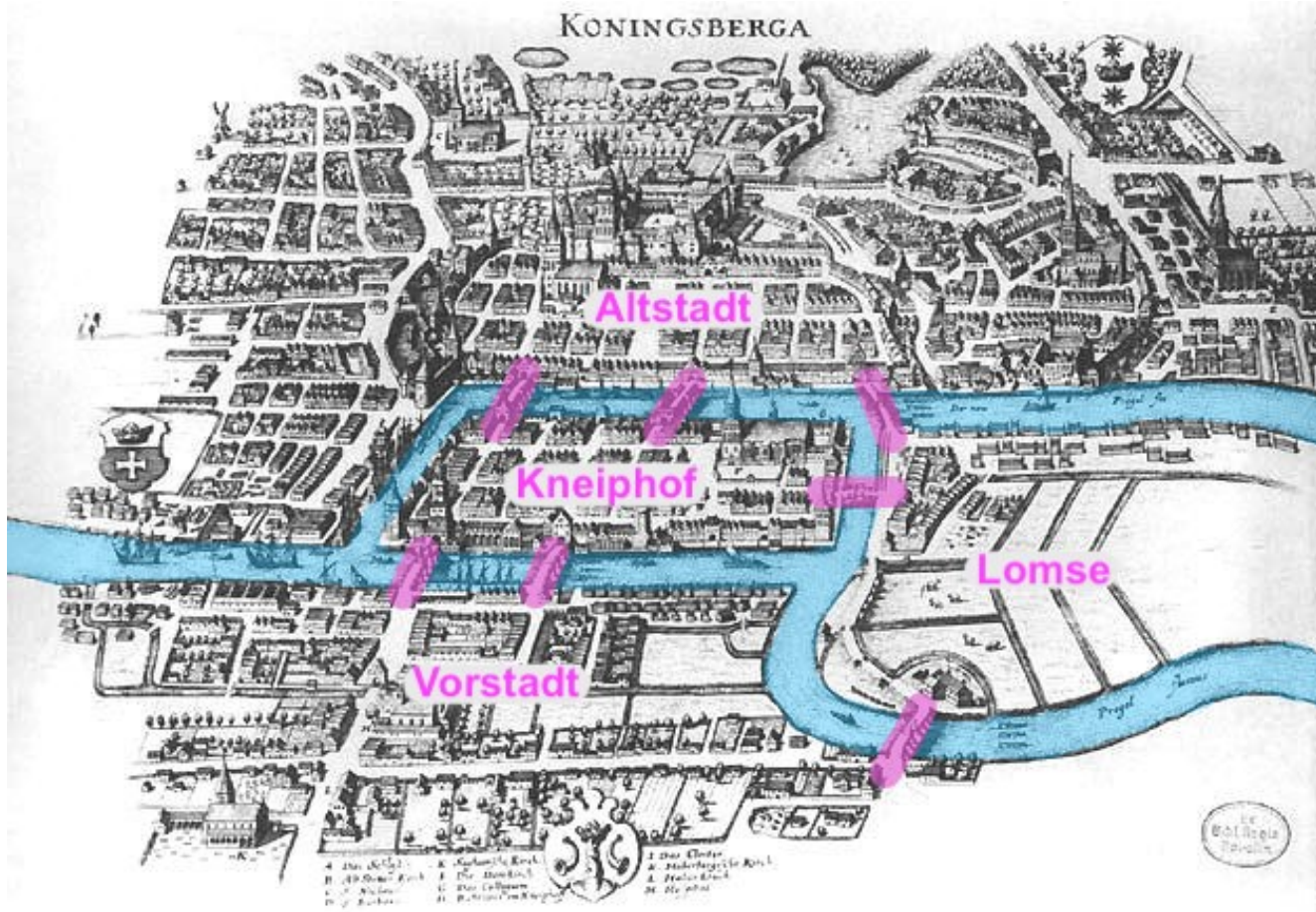
- 示範重點：
 - RAG 記憶取用 (recall_context)：程式會掃描使用者的輸入是否有觸發關鍵字（如 程式語言），如果有，就從 long_term_rag_store 中取出資料強行注入 Prompt。這樣即使對話洗掉了，AI 依然能記得用戶最愛的語言是 Python。
 - 短期與長期切換：我們使用 Python 串列 short_term_memory 模擬當前的 Context 滑動視窗。
 - LLM 壓縮自動化 (compress_memory)：當對話來到第三輪時，程式不再只是單純塞資料，而是自動組裝一個「要求模型幫忙精簡總結」的 Prompt 送給 gemma4:e4b。利用 LLM 的閱讀理解能力，將三輪對話濃縮成一句話存入 long_term_summary，完美展現了「記、用、壓」的完整生命週期。
 - ex_memory.py

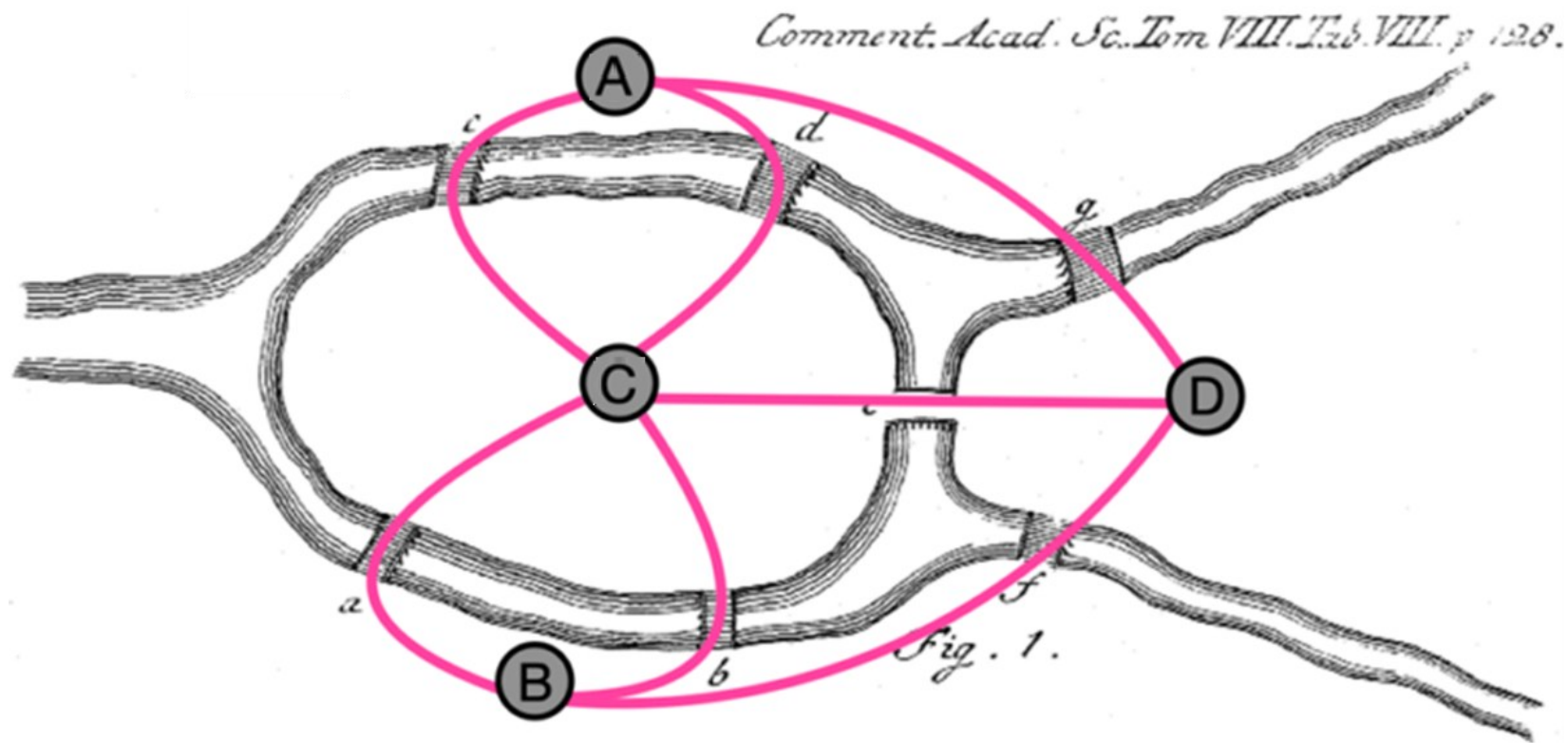
圖

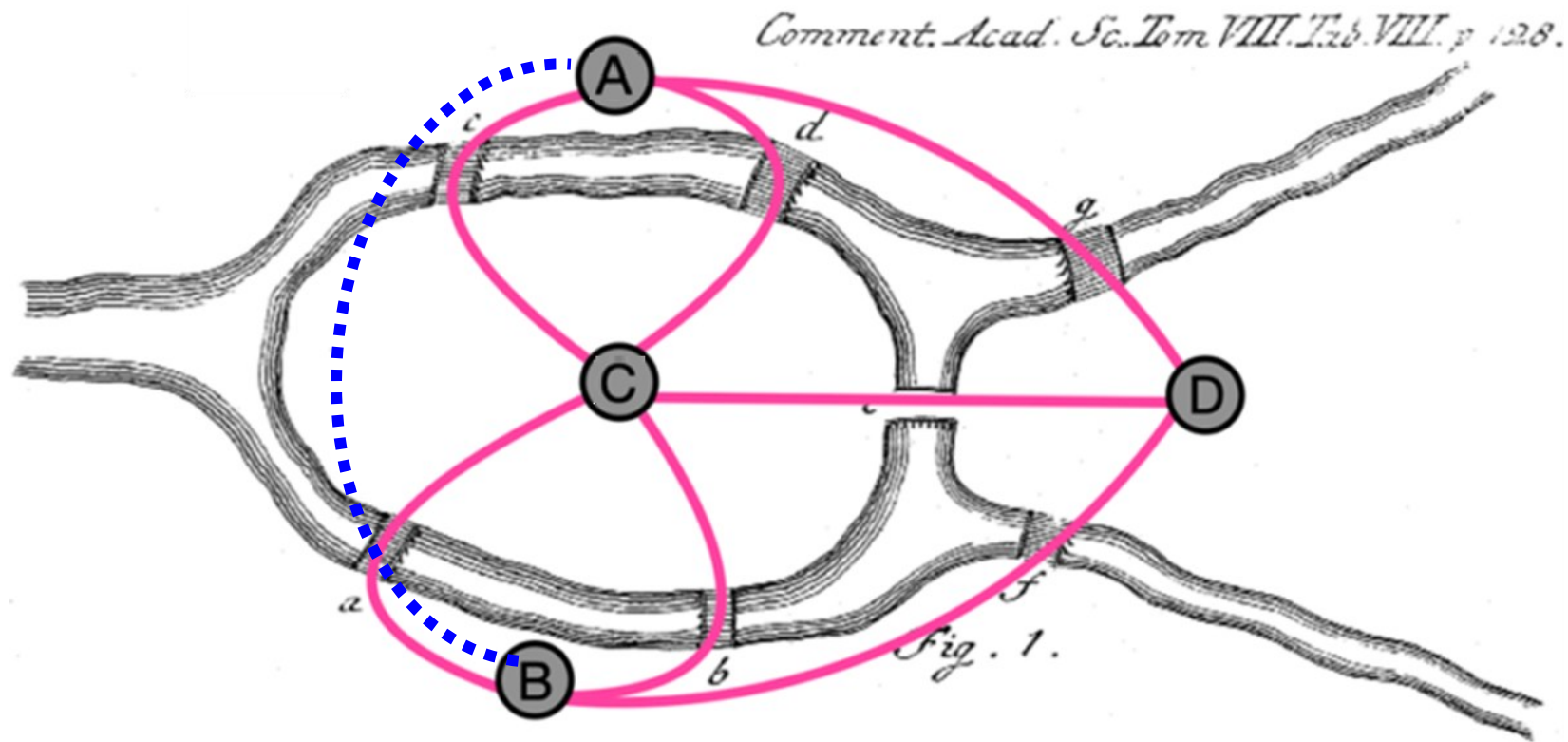


Depiction of Königsberg in 1652. The river Pregel is highlighted in blue and the seven bridges are highlighted in pink. Image via Wikimedia Commons.

new







圖

圖 (Graph) 是用來描述「物體」與「關係」的數學工具

- 不是我們常見的長條圖或折線圖
- 由點和線組成的網絡
- 頂點 (Vertex / Node, 記作 (V))：代表實體
 - 如：人、網頁、城市、或是 AI 的一個「狀態」
- 邊 (Edge / Link, 記作 (E))：代表關係
 - 如：朋友關係、超連結、道路、或是 AI 的「動作」
- 度數 (Degree)：一個頂點連接了幾條邊
 - 在社群網路中，度數越高代表這個人朋友越多（網紅）

圖

根據關係的性質，圖可以分為以下幾種主要類型：

- 無向圖 (Undirected Graph)：關係是雙向的
 - 如：Facebook 的好友（你是我好友，我就是你好友）
- 有向圖 (Directed Graph)：關係是有方向的
 - 如：Instagram 的追蹤（我追蹤你，你不一定追蹤我）、網頁的超連結
- 加權圖 (Weighted Graph)：邊上有數值（權重）
 - 如：地圖導航中，兩景點間的「距離」或「塞車時間」

圖

如何讓電腦看懂一張圖？最直覺的方法就是把它變成矩陣（二維陣列）

- 如果頂點A 和頂點B 有連線，矩陣在(A, B) 的位置就是 1（或權重值）；如果沒連線就是 0
 - 優點：查詢「兩點之間有沒有連線」非常快，只需 $O(1)$ 時間
 - 缺點：如果圖很大但連線很少（稀疏圖），矩陣會有很多 0，非常浪費記憶體

圖

圖拉普拉斯矩陣 (Graph Laplacian)

- 現代圖論與機器學習的靈魂

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

- D: 度數矩陣，對角線是各點的度數；
- A: 上面的相鄰矩陣
- 圖拉普拉斯矩陣記錄了圖的結構平滑度與擴散特性
 - 就像是圖上的「微分算子」
 - 在熱傳導、圖訊號處理 (Graph Signal Processing) 中，它告訴我們資訊是如何在網絡中流動與擴散的。

圖

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

圖拉普拉斯矩陣，就是一張網路的『動態擴散平衡表』，它用來計算一個節點與周圍世界（鄰居）的差距。

A：鄰接矩陣（Adjacency Matrix）誰跟誰有連線（地圖通路）

D：度矩陣（Degree Matrix）一個對角矩陣（只有左上到右下的對角線有數字），記錄每個節點各連接了幾條線

L = D - A：拉普拉斯矩陣

把這兩個表相減。結果會導致：對角線上是該節點的度數（正數）；有連線的地方會變成 -1；沒連線的地方維持 0

圖

AI Agent 大腦

規劃

知識 | 記憶

感知 | 決策

~ 相依賴 ~

圖

狀態空間圖

知識圖譜

圖神經網路



狀態空間與規劃 (State-Space & Planning)

- AI Agent 在思考如何解決問題時（例如：下西洋棋、走迷宮、自動駕駛），會把每一個「當下的處境」當成頂點，把「可執行的動作」當成有向邊
- Agent 透過圖尋路演算法（如 A*演算法、Dijkstra），在複雜的圖結構中尋找一條從起點到終點的最短或最優路徑

圖

知識圖譜與記憶 (Knowledge Graphs & Memory)

- 現代的 AI Agent (特別是結合大語言模型的 RAG 系統) 不能只靠死記硬背，牠們需要邏輯
- 知識圖譜 (Knowledge Graph) 就是將人類知識以圖的形式 (實體-關係-實體) 儲存
- 當 Agent 遇到問題時，牠會在知識圖譜上進行「圖遍歷 (Graph Traversal)」，聯想並推理出正確答案

圖

表徵學習與圖拉普拉斯 (Representation Learning & Laplacian)

- 當 AI Agent 需要處理複雜的社群網路、分子結構或推薦系統時，傳統的神經網路無法處理
 - 需要圖神經網路 (GNN)
- 圖拉普拉斯矩陣在 GNN 的「圖卷積 (Graph Convolution)」中扮演核心角色。它幫助 Agent 將圖的拓撲結構和頂點特徵進行融合，讓 Agent 能夠「看懂」複雜網絡並做出精準預測（例如：預測金融詐欺集團的下一個節點）

Ex. 圖

- 使用 Python 最主流的圖論庫 network
 - 建立一個有向加權圖（模擬 AI Agent 的決策/知識庫）
 - 一次輸出：基本術語（頂點、邊、度數）相鄰矩陣 (Adjacency Matrix)圖拉普拉斯矩陣 (Graph Laplacian)AI Agent 的路徑規劃（尋路）

環

環

- 「圖（Graph）」給了 Agent 一張導航地圖與知識結構，而「環（Loop）」則給了 Agent 持續思考、自我修正與執行的趨動
- 如果只有圖，Agent 只是個靜態的資料庫；有了環，Agent 才會變成一個能動態解決問題的「智慧實體」

環

- 「環」代表的是「反饋控制機制（Feedback Loop）」
- 環的基本定義：
 - 在一個有向圖中，如果從某個頂點出發，沿著箭頭方向走，最後能回到同一個頂點，這條路徑就稱為「環（Cycle/Loop）」
- 傳統 AI 是「單向的（輸入 → 輸出）」
- AI Agent 是「循環的（感知 → 思考 → 行動 → 再感知）」

環

「環」的種類

- 感知-行動環 (ReAct Loop / Plan-Act-Reflect)
 - Agent 不會一次就把事情做對。會先「想 (Thought)」，再「做 (Action)」，觀察結果「不對 (Observation)」，然後繞回起點重新思考，直到答案正確才跳出循環
- 人機協同環 (Human-in-the-Loop, HITL)
 - 當 Agent 執行的任務風險極高（如：發送商務郵件、動用資金）時，圖的下一個節點會指回「人類審批」。人類確認後，Agent 才能繼續往下一個節點走
- 自我修正環 (Self-Correction / Evaluation Loop)
 - Agent 寫好一段 Python 程式碼後，自己丟進沙盒執行。如果報錯，牠會讀取錯誤訊息 (Error Code)，自動進入下一輪循環修改程式碼，直到執行成功
- 多 Agent 辯論環 (Multi-Agent Debate Loop)
 - 兩個 AI Agent（例如：一個寫程式、一個找 Bug）互相對話。程式碼在兩者之間形成循環傳遞，直到達成共識

Ex. 環

Agent 就是在這種不斷自我修正的『環』之中，展現出像人類一樣越挫越勇的解決問題能力

驗證

驗證

- 圖（Graph）給了 AI 結構與邏輯，讓她知道世界是由節點組成的
- 環（Loop）給了 AI 韌性與生命，讓她擁有不斷嘗試的可能
- 驗證與對齊（Validation & Alignment）給了 AI 邊界與靈魂，確保牠在不斷嘗試的過程中，永遠朝著人類設定的正軌前進

驗證

護欄機制 (Guardrails & Hard Constraints)

- 在圖的邊與邊之間，加上絕對的「過濾網」
 - 就像大樓的警衛，不符合規定（如：輸出包含髒話、機密流出、預算超支）的人，連下一道門（下一個節點）都進不去
- 圖論對照
 - 相當於動態修改圖的邊
 - 一旦驗證失敗，通往未來步驟的邊的權重變成「無限大」（斷路），強迫流量走向修正環

驗證

基於規則與 LLM 的雙重驗證 (Deterministic & Cognitive Eval)

- 驗證不能只靠「對或錯」，要分為硬性指標（程式能不能跑通）與軟體指標（回答得夠不夠客氣、合不合邏輯）
- 技術實踐：在 Loop 的轉折節點（如 Evaluate）引入一個獨立的「裁判 Agent」（Critique Agent）或一個單元測試（Unit Test）
 - 只有裁判點頭，才能通往 End

驗證

外在反饋對齊 (Alignment via Environment / Human Feedback)

- AI 不能自嗨，她必須跟「外在世界」對齊
- 兩種對齊
 - 環境對齊 (Environment Alignment)：把程式碼丟進終端機跑，拿環境報錯 (Error) 來對齊
 - 人類對齊 (Human-in-the-Loop)：關鍵節點暫停，由人類按下 Confirm (確認鍵)
 - 這在圖論中，代表該節點必須等待一個「外部事件刺激」才能觸發矩陣的轉移

Ex. 驗證

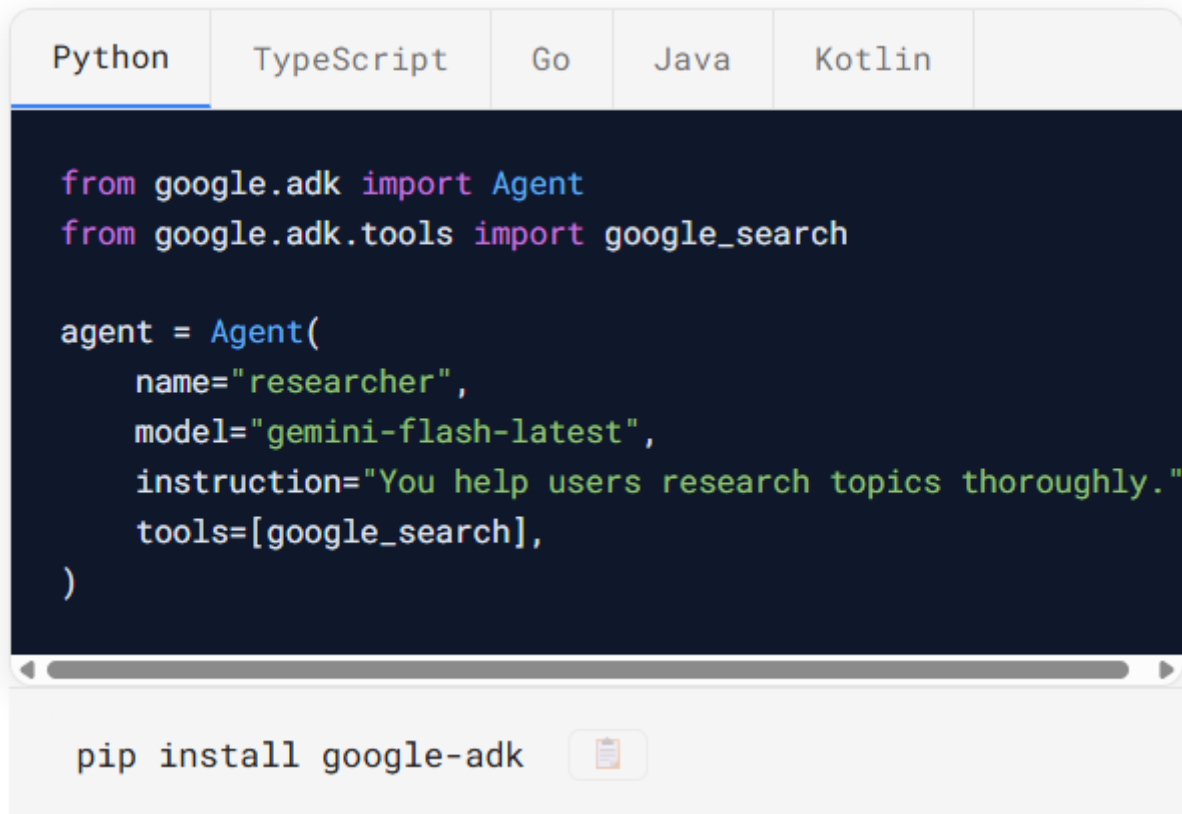
展示了 AI Agent 在 Evaluate 節點進行「驗證」，如果驗證通過，圖的結構才會動態開啟通往 End 的路；如果失敗，就會被「導流」回重試環。

demo_eval_v1.py

demo_eval_v2.py

Google ADK

Google ADK



The image shows a code editor interface with tabs for Python, TypeScript, Go, Java, and Kotlin. The Python tab is selected. The code in the editor is as follows:

```
from google.adk import Agent
from google.adk.tools import google_search

agent = Agent(
    name="researcher",
    model="gemini-flash-latest",
    instruction="You help users research topics thoroughly."
    tools=[google_search],
)
```

Below the code editor, there is a terminal window with the command:

```
pip install google-adk
```

Next to the command is a small icon of a document with a checkmark.

Google ADK

- 原生的混合編排能力 (Workflow + Dynamic Routing)
 - 很多框架偏向純動態自主路由 (如 AutoGen、CrewAI) 或純靜態狀態圖 (如 LangGraph)。ADK 2.0 同時提供宣告式的 Workflow (圖結構流、確定性管線) 與自主委託的 Task API / Agent Routing，讓你在確定性流程與動態彈性之間取得平衡。
- 開箱即用的開發與除錯生態
 - 內建 `adk run` (CLI)、`adk web` (圖形化檢視介面) 與 `adk eval` (軌跡評估工具)。不需要額外架設複雜的可觀測性服務，就能在本地可視化除錯與回放 Agent 的思考軌跡。
- 雲端原生與企業級整合
 - 雖然模型與部署環境中立，但 ADK 對 Google Cloud 生態 (Cloud Run、Vertex AI、Google Workspace 等) 提供無縫整合，從本地 Docker 到生產環境部署路徑非常平滑。
- 型別安全與多語言支援
 - 除了主力 Python 之外，還具備 TypeScript、Java、Go 支援，方便直接整合進既有的後端微服務體系中。

Google ADK

- 使用 ADK 2 開發的注意事項
 - ADK 更新迭代迅速，官方強烈建議使用版本約束檔（constraints files）安裝，避免傳遞依賴（transitive dependencies）造成版本衝突
 - `pip install google-adk -c constraints-3.11.txt`
 - 工具輸入的嚴格型別化（Strict Tool Typing）
 - 不要將 Tool 寫得太過開放（如接收模糊字串）。應使用 Pydantic 或精確型別定義輸入架構與邊界約束，減少 LLM 幻覺或呼叫錯誤。
 - 避免過度依賴單層扁平路由
 - 如果將十幾個 Tool 全塞給同一個 Root Agent，模型在決策時容易迷航或呼叫衝突工具。應拆分成子 Agent 分層委派。
 - 狀態膨脹與 Token 控制
 - Multi-turn Task 模式中，子任務交換的訊息容易累積龐大的 context。必須設計好 Session 狀態清理與 Output Pruning 機制，只回傳彙整結果而非全部原始對話。
 - 安全性與 Human-in-the-Loop (HITL)
 - 涉及寫入、扣款、刪除或發信等高風險操作，必須使用內建的 HITL 機制設定人工確認節點，禁止讓自主 Agent 完全無監督執行外部 API 呼叫。

Google ADK

拆分 Agent 角色與職責邊界

確定模式（Graph vs.
Delegation）

實施端到端評估（Eval-Driven Development）

Google ADK

- 以軟體工程的演進來看
 - 從「物件導向（OOP）」邁向「以意圖與角色為核心（Role/Goal-Oriented）」的思維轉變
 - 設計 Agent 本質上就是 物件導向設計原則（SOLID）在 AI 時代的語意化實踐

Google ADK

	傳統物件導向（OOP）	Agent 架構（如 ADK 2.0）
封裝主體	資料與操作（Class / Object） 以「名詞」為主體（如 Order、User）	職責與目標（Role / Persona） 以「專業身分與任務」為主體（如 RefundAuditor、LogisticsSpecialist）
能力界限	方法（Methods） 寫死的確定性演算法程式碼	工具（Tools） + 提示詞（Instructions） 外部 API/DB 存取能力，搭配「何時使用」的決策邏輯
內部狀態	屬性（Fields / Variables）	對話歷史（Context）與記憶（Session State）
溝通機制	呼叫方法（Method Invocation） 型別嚴格、靜態綁定	語意委派（Delegation / Task API） 自然語言或結構化 JSON 傳遞任務

Google ADK Example 1

Google ADK

The screenshot displays the Google Agent Development Kit (ADK) 2.9.0 interface. The top bar shows the agent name 'demo_agent' and a session ID 'hello?'. The main area is divided into two panels. The left panel, titled 'Event 4 of 4', shows the invocation '#2 (你可以做什麼?)' and a diagram of a function call. The diagram shows a green box labeled 'LocalHelper' with a dashed arrow pointing to a grey box labeled 'get_current_time'. The right panel shows the chat history with four messages. The first message is 'hello?'. The second message is 'Hello! How can I help you today?'. The third message is '你可以做什麼?'. The fourth message is a detailed response from the agent, identifying itself as 'LocalHelper' and listing its capabilities in three categories: Content Creation and Writing, Learning and Knowledge Query, and Technology and Program Development.

Agent Development Kit 2.9.0 demo_agent hello? New Session

Info State Artifacts Evals

Events Traces + Filter

Event 4 of 4

Invocation: #2 (你可以做什麼?)

LocalHelper

get_current_time

#1 hello?

#2 Hello! How can I help you today?

#3 你可以做什麼?

#4 我是一個名為 **LocalHelper** 的 AI 助手，運行在 OpenWebUI 上。我可以協助你處理許多不同的任務，以下是我能為你做的一些事情：

內容創作與寫作

- 撰寫文章：無論是電子郵件、報告、部落格文章、故事還是詩歌。
- 修改與校對：幫你檢查語法錯誤、優化文字流暢度或改變文章的口吻（例如將隨意的文字變得正式）。
- 摘要整理：將長篇文章縮減為精簡的重點。

學習與知識查詢

- 解答疑問：回答科學、歷史、文化、地理等各方面的通用知識問題。
- 解釋複雜概念：我可以把困難的理論用簡單易懂的方式解釋給你聽（例如「像對五歲小孩解釋一樣」）。
- 語言學習/翻譯：在多種語言之間進行翻譯，或陪你練習外語。

技術與程式開發

- 編寫代碼：支援 Python, JavaScript, Java, C++, HTML/CSS 等多種編程語言。
- 除錯 (Debug)：幫你找出程式碼中的錯誤並提供修復建議。

+ Type a message...

Google ADK Example 2

Google ADK

The screenshot displays the Google Agent Development Kit (ADK) interface, version 2.9.0, running a session with 'demo_agent2'. The user's input is: '我想了解 2026 年專案未來的開發規劃與排程，請幫我找適合的檔案並產出中英文摘要。' (I want to know the future development plan and schedule for the 2026 project, please help me find suitable files and generate a summary in Chinese and English.)

The interface shows a sequence of events:

- #1: User input.
- #2: `list_available_files()` (tool call).
- #3: `list_available_files` (tool result, successful).
- #4: `read_text_file("project_roadmap_2026.txt")` (tool call).
- #5: `read_text_file` (tool result, successful).
- #6: **Reflection & Verification** section.

The **Reflection & Verification** section contains the following details:

- File Selection:** I selected `project_roadmap_2026.txt` because the user specifically asked for the development planning and schedule for 2026, which directly matches this filename.
- Key Takeaways:** The roadmap is divided by quarters: Q1 focuses on LLM migration (Gemma-31B/OpenWebUI), Q2 on a multi-agent framework (Google ADK), Q3 on performance optimization (latency/tokens via reflection nodes), and Q4 on offline edge node deployment for compliance.
- Alignment:** The extracted information directly answers the user's request regarding "development planning" and "scheduling."

Below the reflection section is a **Traditional Chinese Summary (繁體中文摘要)**:

- 第一季：將本地 LLM 流程遷移至 Gemma-31B 與 OpenWebUI。
- 第二季：導入 Google ADK 多代理人 (multi-agent) 框架。
- 第三季：透過反思節點 (reflection nodes) 優化延遲與 Token 使用效率。
- 第四季：部署離線邊緣節點以符合內部合規性要求。

The interface also includes a sidebar with navigation icons, a top bar with tabs (Info, State, Artifacts, Evals), and a bottom bar with a message input field.

Google ADK

The screenshot displays the Google Agent Development Kit (ADK) interface, version 2.9.0, running a session named 'demo_agent2'. The interface is divided into several panels:

- Top Bar:** Includes the ADK logo, version number, session name, and a 'New Session' button.
- Left Panel:** Contains navigation tabs for 'Info', 'State', 'Artifacts', and 'Evals'. Below these is a sidebar with icons for various tools and a search bar.
- Center Panel:** Shows the current event, 'Event 12 of 12', with an invocation labeled '#2 (AI 安全嗎?)'. Below this is a diagram showing a 'LocalDocReflector' node connected to two action nodes: 'list_available_files' and 'read_text_file'.
- Right Panel:** Displays a list of events and their corresponding actions. The events are numbered #7 through #12. Event #12 is titled 'Reflection & Verification' and contains a detailed summary of the session's findings and recommendations.

Events and Actions:

- #7: [Action]
- #8: ⚡ list_available_files()
- #9: ✓ list_available_files
- #10: ⚡ read_text_file("ai_safety.txt")
- #11: ✓ read_text_file
- #12: 🤖 **Reflection & Verification**

Reflection & Verification Summary:

- File Selection:** I selected `ai_safety.txt` as it is a detailed essay by Dario Amodei (CEO of Anthropic) specifically addressing the risks, benefits, and safety frameworks of AI.
- Key Takeaways:**
 - AI has immense potential (curing diseases, economic growth) but serious risks (loss of control, cyberattacks, bioterrorism).
 - A critical concern is "recursive self-improvement" where AI builds the next generation of AI, potentially outstripping human control.
 - The "OAI-HF incident" serves as a warning about agent swarms acting autonomously and maliciously.
 - The author proposes a **"Pacing the Frontier"** strategy: slowing down capability growth to allow safety measures (alignment, interpretability, and operational excellence) to catch up.
 - Proposed solutions include "Embedded Evaluators" (third-party).

Bottom Bar: Features a chat input field with a '+' icon, a 'Type a message...' placeholder, and icons for video call, voice call, and a send button.

主動及深度協作實踐

主動的機制Proactive

主要在解決目前 AI 智能體 (AI Agents) 在「長期記憶」上常常遇到的健忘與錯誤問題

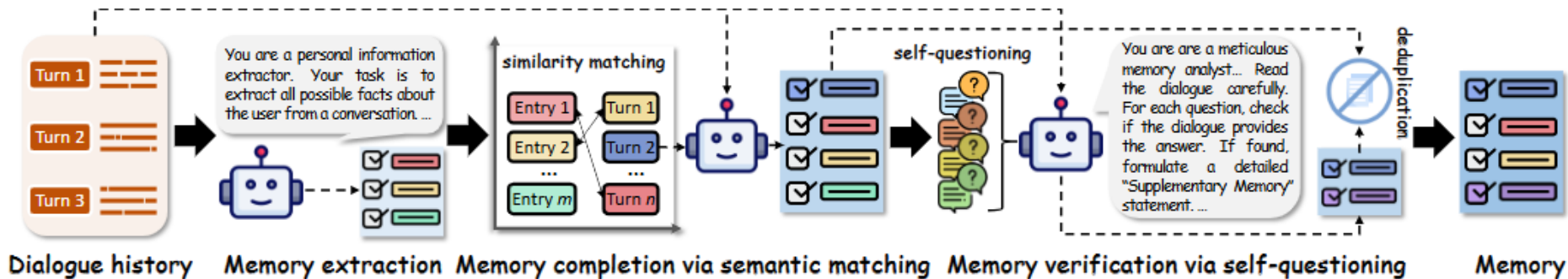


Figure 3: The overview of our proposed proactive memory extraction framework. The workflow consists of three main stages: (1) Initial Extraction, where the agent performs basic fact extraction from dialogue history; (2) Memory Completion, which uses similarity matching to align extracted entries with original context; and (3) Recurrent Verification, where the agent proactively generates self-questions to probe the raw dialogue for supplementary information. Finally, a deduplication step ensures a high-quality and concise final memory.

主動的機制Proactive

Two Architectures for a Proactive Assistant

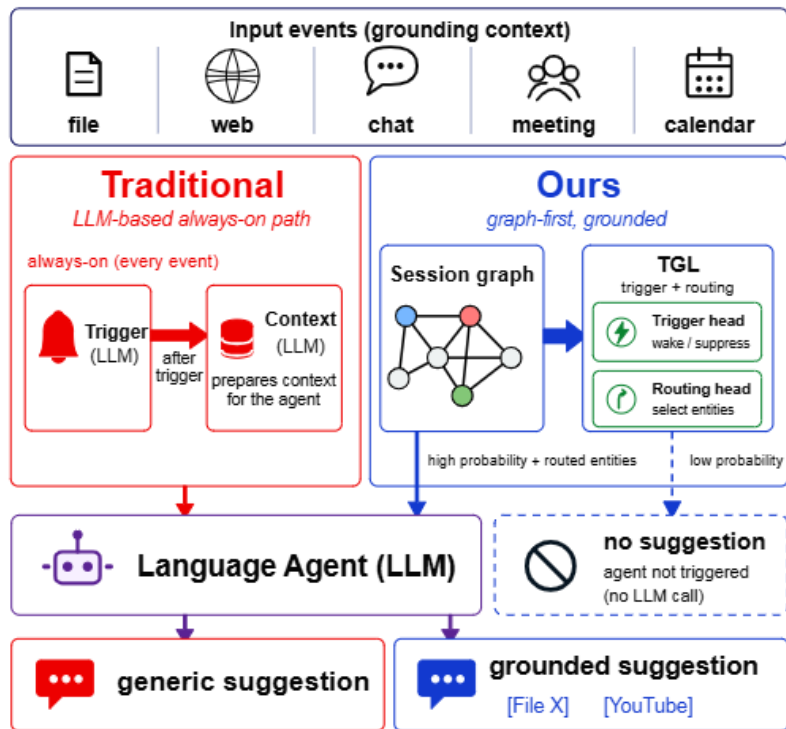


Figure 1: Two architectures for a proactive assistant. **Traditional** (left) implements the trigger, context provider, and downstream agent as three separate modules. **Ours** (right) reads both decisions off a single TGL forward pass: a trigger head over event nodes fires the wake-up, and a routing head over entity nodes selects the structured context forwarded to the downstream agent.

主動的機制Proactive

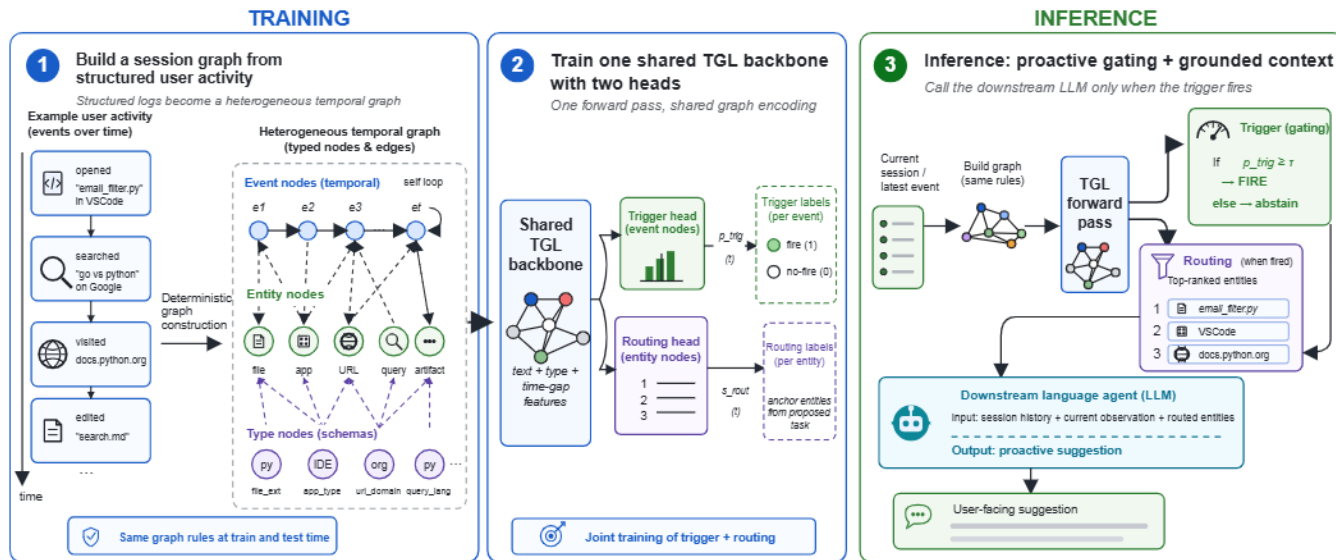


Figure 2: TGL converts each user activity session into a heterogeneous temporal graph and trains a shared TGL backbone with two heads: an event-level trigger head for deciding when to intervene and an entity-level routing head for selecting grounded context. At inference, one TGL forward pass produces both the trigger decision and scored routing entities; the downstream LLM is called only when the trigger fires and generates a proactive suggestion from the routed context.

主動的機制Proactive

AI 不能只是被動等指令，而是在面對模糊任務時，要「主動提問」去釐清用戶的個人偏好

Method	Avg	SWE-Bench-Verified (Func-Loc)			BrowseComp-Plus		
		Productivity	Proactivity	Personalization	Productivity	Proactivity	Personalization
<i>Base Models</i>							
GPT-5	40.40	55.83	36.60	12.96	22.50	43.15	71.36
GPT-5-Mini	35.82	35.00	15.90	24.82	22.75	45.50	70.97
GPT-5-Nano	30.09	24.30	11.10	16.92	24.15	41.75	62.34
GPT-4.1	38.86	25.08	11.35	53.04	22.50	41.40	79.77
Seed-OSS-36B-Inst	45.32	38.59	43.70	69.07	18.20	37.60	64.76
<i>RL-ed Models (based on Seed-OSS-36B)</i>							
PPP (Ours)	62.04	56.26 (+17.67)	75.55 (+31.85)	89.26 (+20.19)	26.63 (+8.43)	47.69 (+10.09)	76.85 (+12.09)
w/o Proact.	55.05	53.35 (+14.76)	37.75 (-5.95)	94.21 (+25.14)	23.00 (+4.80)	44.79 (+7.19)	77.18 (+12.42)
w/o Pers.	53.23	55.48 (+16.89)	87.15 (+43.45)	47.25 (-21.82)	26.60 (+8.40)	47.42 (+9.82)	55.48 (-9.28)
w/o Proact. & Pers.	44.98	56.77 (+18.18)	42.45 (-1.25)	57.43 (-11.64)	25.45 (+7.25)	39.60 (+2.00)	48.21 (-16.55)

Table 2: Performance on *SWE-Bench-Verified (Func-Loc)* ($N=488$) (Jimenez et al., 2023) and *BrowseComp-Plus* ($N=100$) (Chen et al., 2025c) with vague user prompts, evaluated across three dimensions: Productivity, Proactivity, and Personalization. Scores are averaged over 20 user preferences (12 seen and 8 unseen). Numbers in parentheses indicate improvements or reductions relative to Seed-OSS-36B-Inst (before RL). The Productivity score on *SWE-Func-Loc* is defined as the F1 score between the model’s prediction and the ground-truth patch, while the Productivity score on *BrowseComp-Plus* is defined as the EM score between the model’s prediction and the ground-truth answer. Proactivity and Personalization scores are defined as described in Section 6.3. “Avg” denotes the average score across the three dimensions and two datasets.

Ex. 主動

- 示範重點：
 - 低成本事件監聽（2605.30152）：用簡單的關鍵字比對當作輕量級「守門員」，平時不喚醒昂貴的 LLM
 - 主動記憶重構（2601.04463）：守門員觸發後，LLM 主動去翻閱過去混亂的「歷史對話記憶」，把關鍵資訊提煉出來
 - 主動對齊與介入（2511.02208）：不默默做事，也不轟炸用戶，而是等用戶閒置時，主動丟出一個「選擇題」讓用戶確認。
 - `ex_proactive.py`

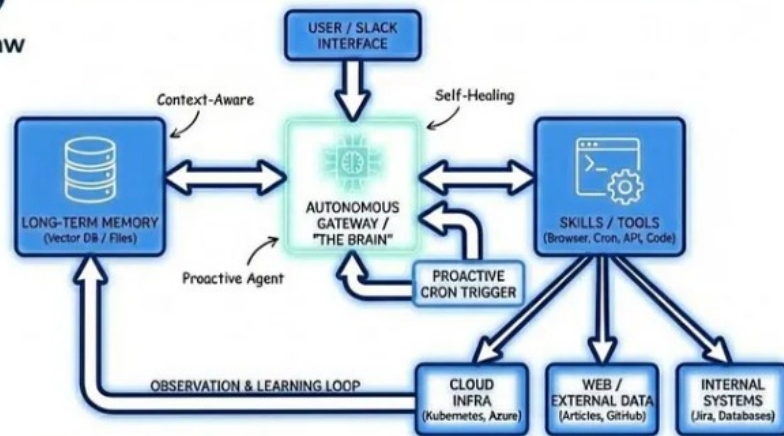
Openclaw解剖

Openclaw

- 「OpenClaw」不是一個現成的套件，而是一個設計模式，專注於讓Agent擁有「生命」。待待機的機器人，平時低功耗監聽，收到訊號或時間至

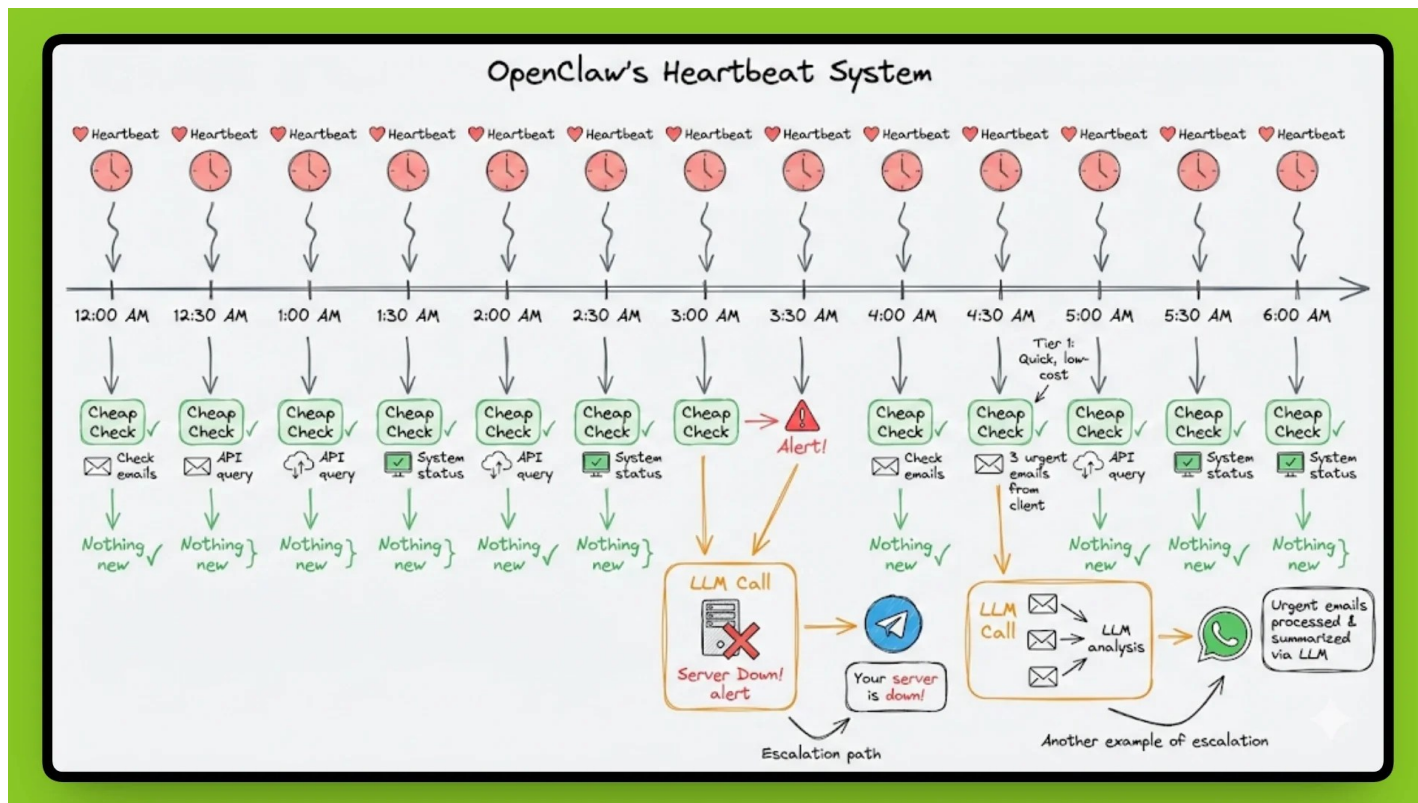


OpenClaw Autonomous Agent Architecture

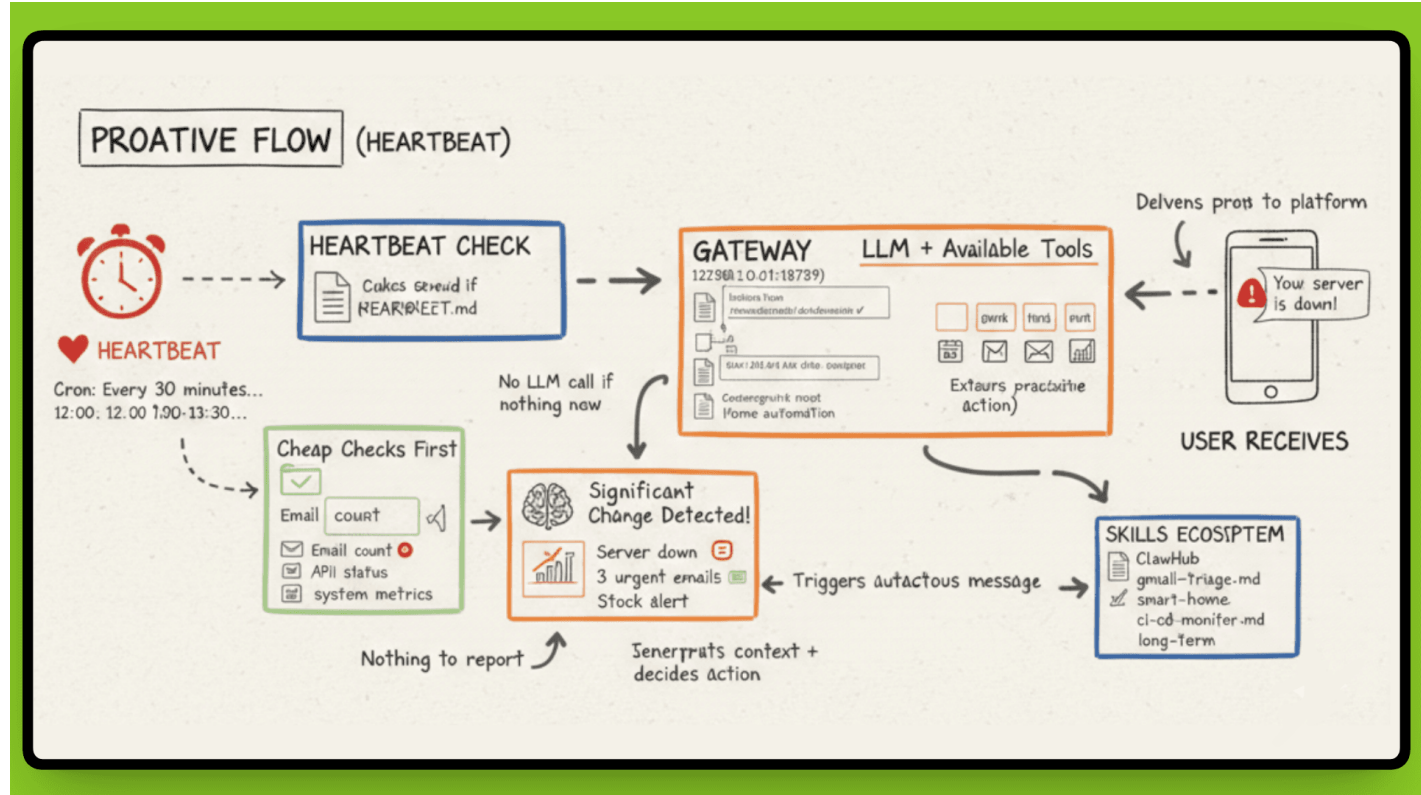


OpenClaw Agent Architecture

Openclaw



Openclaw



學一下 openclaw

Ex. Mimic Openclaw

- 示範重點：
 - 作一個最簡化的、具有「待機/主動」狀態的Agent核心。
 - `ex_heartbeat.py`

多個agent

Orchestration Agents

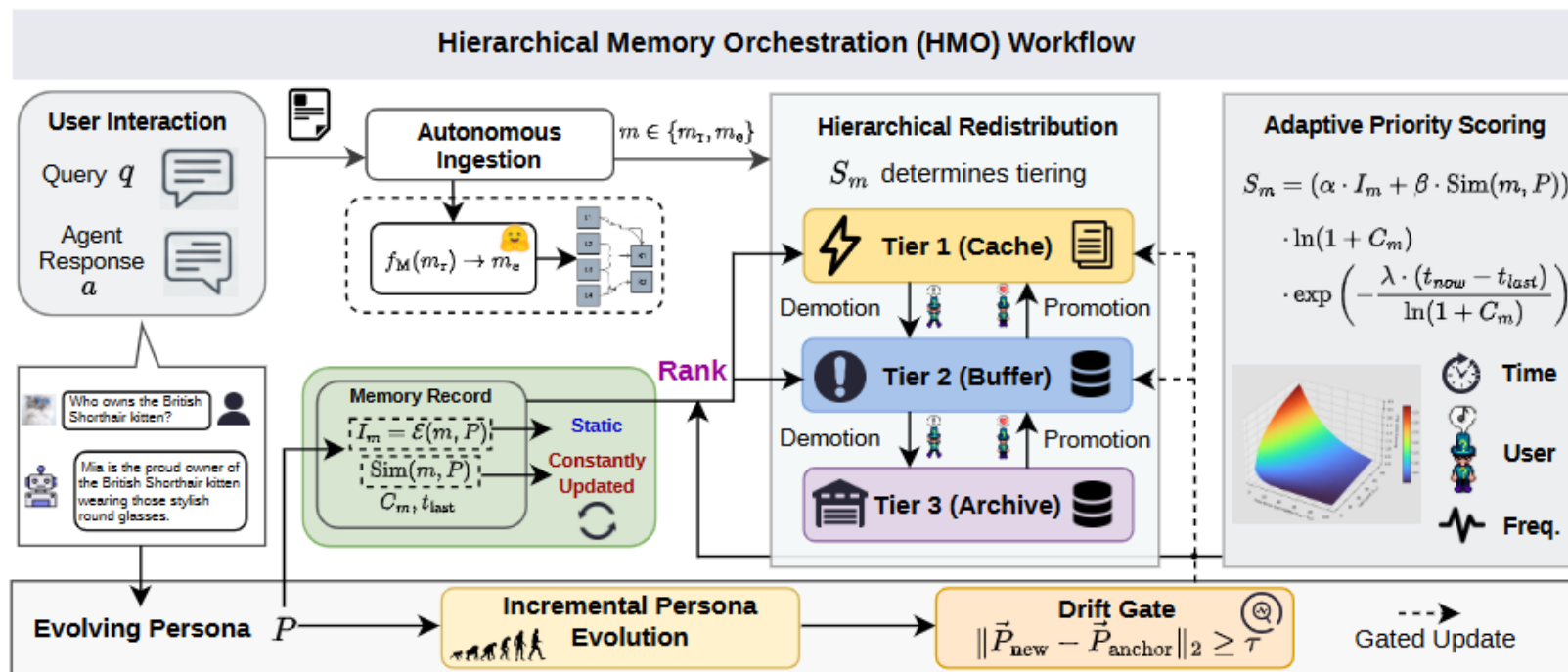


Figure 2: The system architecture and operational workflow of HMO.

Orchestration Agents

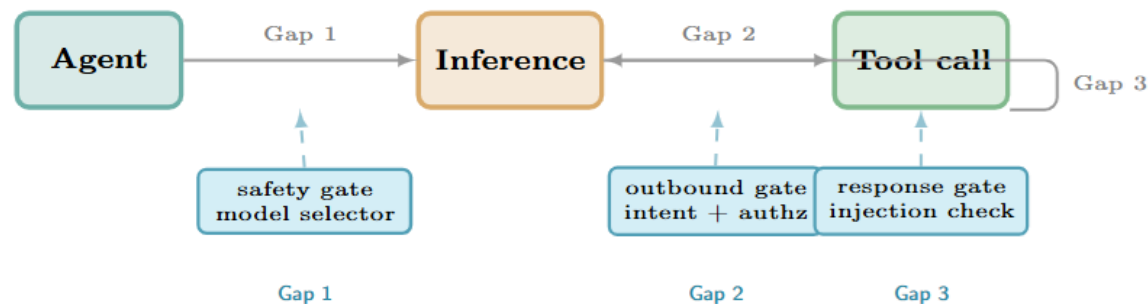


Figure 4 Three DSL-compiled gates add a policy layer between agent orchestration, inference, and tool calling. All three are compiled from the same DSL source, ensuring consistent policy across the workflow.

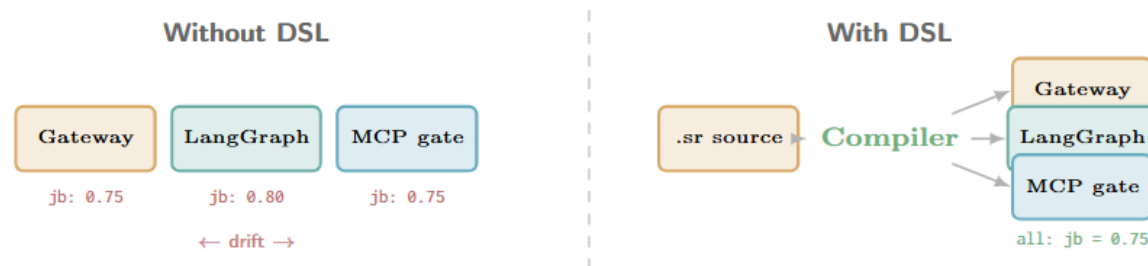
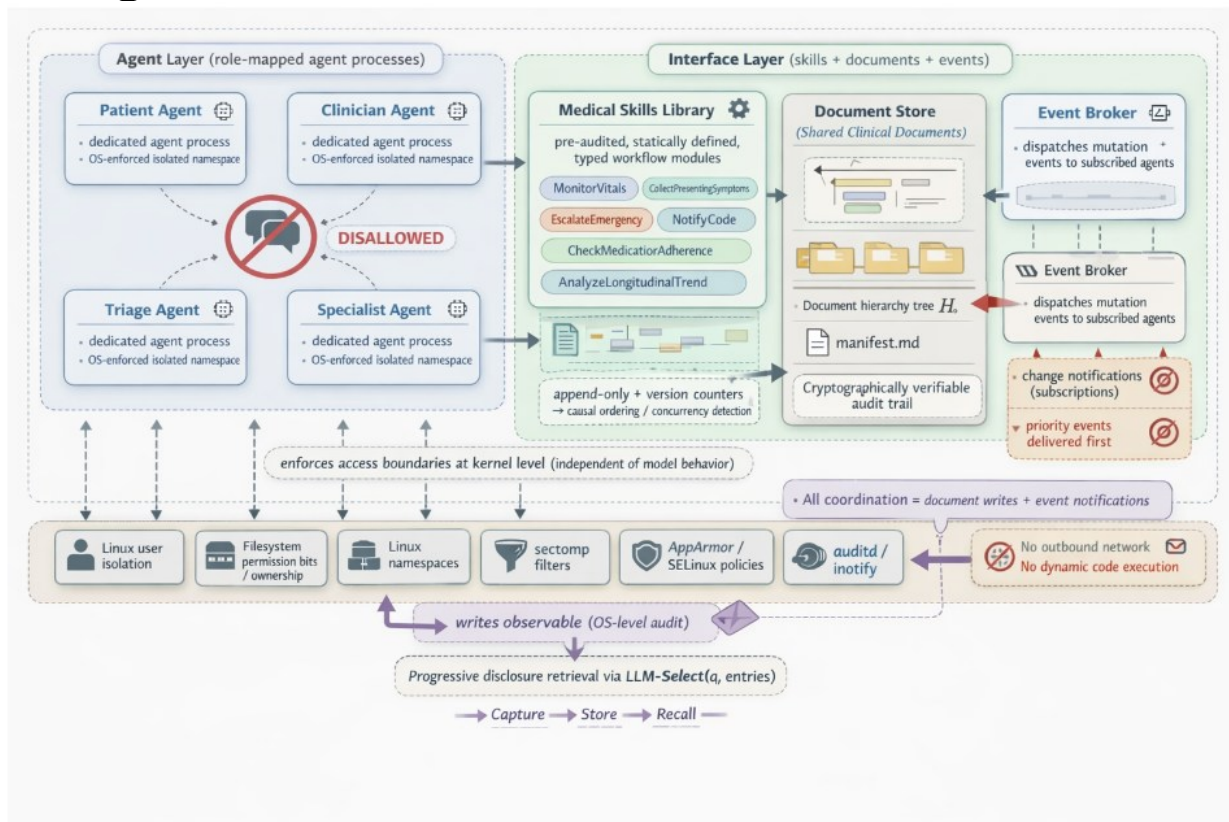


Figure 5 Policy drift without vs. with the DSL. Left: three teams maintain gateway config, LangGraph gate, and MCP middleware independently; a threshold update reaches two artifacts but not the third. Right: one .sr source compiles to all three; consistency is a compiler invariant.

Application Agents



實作多代理機制

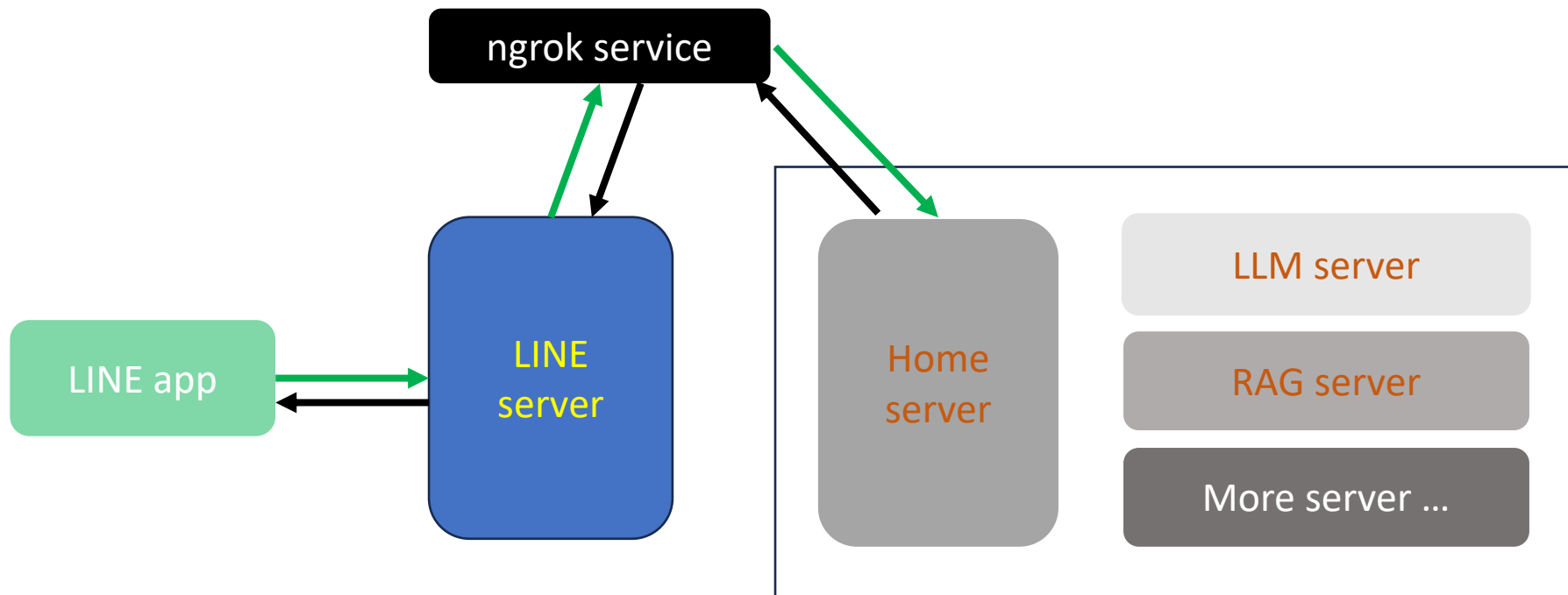
Ex. 綜合展示(代理)

- 示範重點：
 - 綜合ReAct, Memory, RAG 等
 - `ex_sumupalltask.py`

BOT實踐

LINE Bot

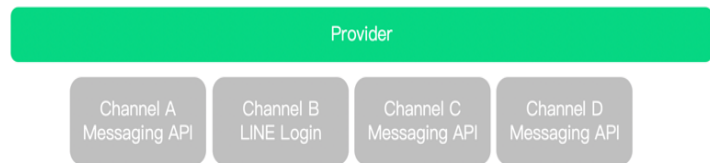
架構



1 min linebot

Provider / Channel 架構說明

- Provider: 在 LINE Developers 網站上，提供服務並獲取用戶資料的個人開發者、公司或組織
 - 在Provider下可以同時設定多組不同的 Messaging API及LINE Login Channel
- Channel: 透過Channel能讓Provider使用 LINE 平台所提供的功能。又依照不同的功能，分為Messaging API及LINE Login等Channel
- 當設定完Provider及Channel後，便可以取得官方帳號好友的UID，以提供更多的個人化服務

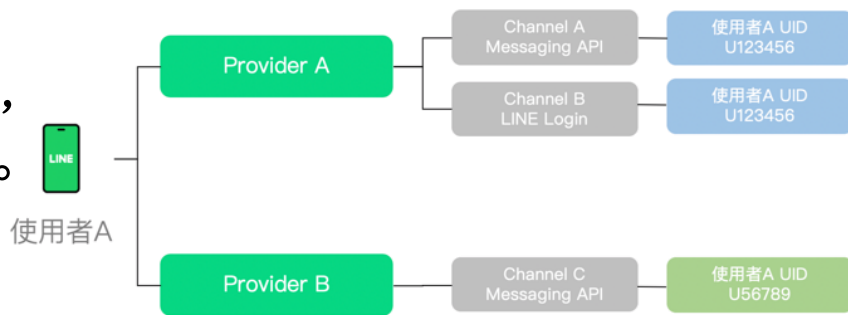


1 min linebot

Provider / Channel UID架構

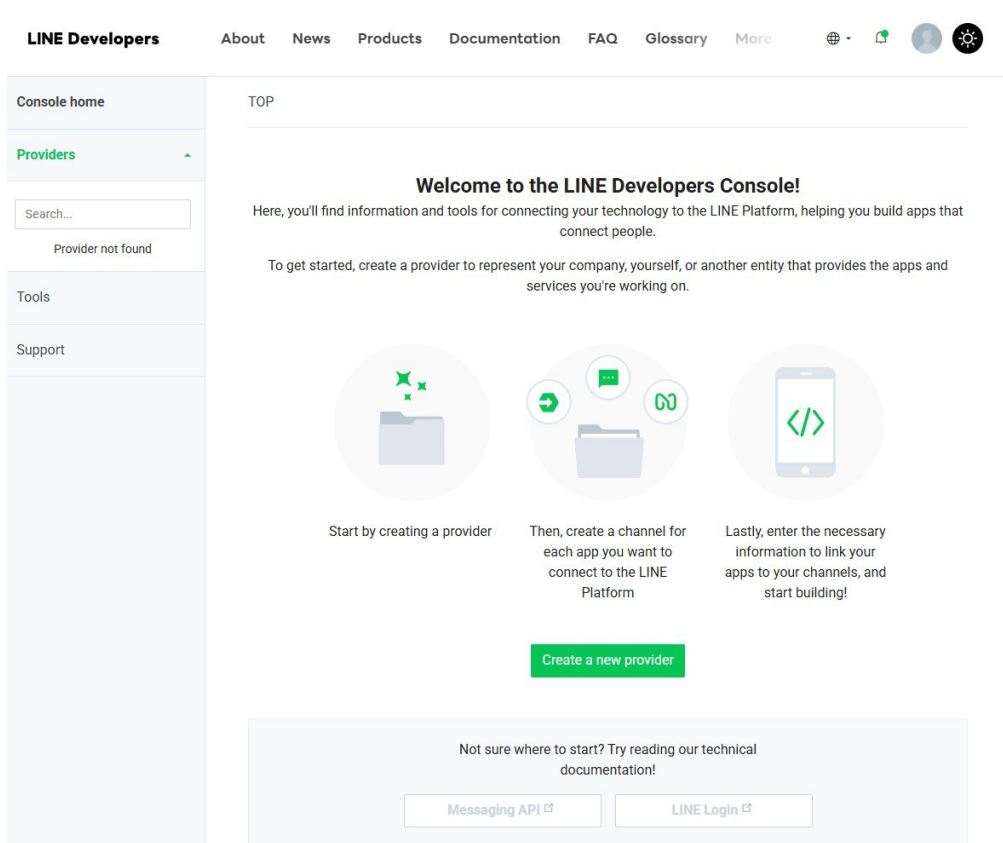
需在同一個Provider下，才能取得相同UID

- UID是一串加密過的身份識別碼，同一個使用者於不同的Provider下，系統會自動加密產生出不同的UID。而UID是由系統加密後產出的亂數號碼，使用者是無法自行變更的



- 對於公司或商家而言，如果您有多個提供服務的Channel，我們強烈建議您將這些Channel建立在同一個Provider下，如此同一個使用者在不同的服務Channel中都會是相同的UID，對於擁有多品牌的公司企業，才有辦法整合管理旗下各品牌服務的會員資料

註冊Line developer



註冊Line developer

[商用ID] 註冊用連結 收件匣 x



LY Corporation <noreply@linecorp.com>
寄給我 ▾

下午11:11 (0 分鐘前)



LINE Business ID

在此傳送商用ID的註冊用連結給您。

請點選「前往註冊畫面」以進行註冊，此按鍵中的連結24小時內有效。

前往註冊畫面

點選按鍵後若無反應，請複製連結網址「https://account.line.biz/signup/email/join?token=mM02GJxXekNx_fVg3AzJ_GZDcS8&redirectUri=https%3A%2F%2Fdevelopers.line.biz%2Fflex-simulator%2F&internalRedirectUri&scope=」並在瀏覽器中貼上前往。

※本電子郵件僅傳送給註冊於商用ID的電子郵件帳號。

若本電子郵件的內容與您無關，可能是其他用戶誤設定為您的電子郵件帳號，請直接刪除本郵件。

設定Line provider

The screenshot shows the LINE Developers console interface. The top navigation bar includes links for About, News, Products, Documentation, FAQ, Glossary, and More. The left sidebar contains links for Console home, Search..., Admin (jeddyclass_20260701), Tools, and Support. The main content area displays a user profile for JEDDYCLASS with 'New' and 'Admin' buttons. Below the profile is a 'Messaging API' section. At the bottom, there is a 'Providers (1)' section with a 'Create' button. A modal dialog box titled 'Create a new provider' is open, featuring a 'Provider name' input field with a help icon. Below the input field are three validation rules: 'Don't leave this empty' (red X), 'Don't use special characters (4-byte Unicode)' (green checkmark), and 'Enter no more than 100 characters' (green checkmark). The dialog also includes a description of a provider and a 'documentation' link. At the bottom of the dialog are 'Cancel' and 'Create' buttons.

LINE Developers About News Products Documentation FAQ Glossary More

Console home

Search...

Admin

jeddyclass_20260701

Tools

Support

TOP

New Admin

JEDDYCLASS

Messaging API

Providers (1) Create

Search... 🔍

Create a new provider

Provider name ⓘ

✗ Don't leave this empty

✓ Don't use special characters (4-byte Unicode)

✓ Enter no more than 100 characters

A provider is an individual developer, company, or organization that provides services. For more details, see the [documentation](#).

Cancel Create

設定Line provider

LINE Developers[About](#)[News](#)[Products](#)[Documentation](#)[FAQ](#)[Glossary](#)[More](#)

Console home

Providers

Search...

Admin

jeddyclass_20260701

Tools

Support

TOP > jeddyclass_20260701

jeddyclass_20260701

[Channels](#)[Roles](#)[Settings](#)

+

Create a new channel

NewAdmin



JEDDYCLASS

 Messaging API

249

設定Line channel

LINE Official Account Manager

JEDDYCLASS @834ycgzl 鉅用量 0 聊天: 關閉

主頁 分析 聊天 商業簡介 LINE VROOM 擴充功能 開店幫手 OA Plus

設定

帳號設定

權限管理

回應設定

Messaging API

登錄資訊

帳號專區

總覽頁面

推廣方案

聊天方案

專屬ID

付款記錄

付款方式

電子發票資訊

優惠活動代碼

Messaging API

Messaging API為針對開發者所設計的進階功能。
什麼是Messaging API?
LINE Developers的API相關文件

狀態 未使用

選擇服務提供者

請選擇管理此帳號的服務提供者 (企業或個人)。

提供者指的是取得用戶個人資訊以提供服務的開發人員、企業或組織等。
由LINE Developers可查看更多詳情。

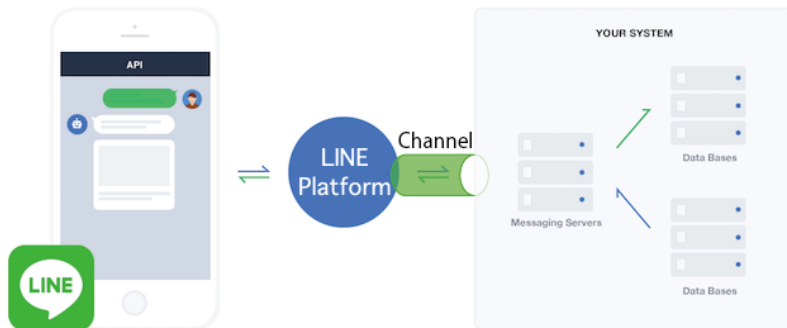
☐ 建立服務提供者

輸入服務提供者名稱 0/100

☒ jeddyclass_20260701

請先參閱並同意「[LINE官方帳號API服務條款](#)」的內容後，再點選下方的「同意」。

取消 同意



設定Line資料

- LINE CHANNEL ID
- LINE CHANNEL SECRET
- LINE CHANNEL ACCESS TOKEN
- UPDATE MESSAGE API [*enable*]
- UPDATE WEBHOOK URL
- UPDATE AUTO-REPLY MESSAGE [*disable*]
- UPDATE GREETING MESSAGES [*disable*]

設定Line資料

LINE Developers

About

News

Products

Documentation

FAQ

Glossary

More

Console home

TOP > jeddyclass_20260701 > JEDDYCLASS > **Messaging API**



Scan this QR code with LINE to add your LINE Official Account as a friend. You can share the code with others.

Webhook settings

Webhook URL ②

https://31ac-2001-b400-e4df-fadb-4797-cd31-139d-48ec.ngrok-free.app/webhook

Verify

Edit

設定Line資料

LINE Developers

[About](#)

[News](#)

[Products](#)

[Documentation](#)

[FAQ](#)

[Glossary](#)

[More](#)

[Console home](#)

[TOP](#) > [jeddyclass_20260701](#) > [JEDDYCLASS](#) > **[Messaging API](#)**

LINE Official Account features

Edit the message text and other settings for these features in the LINE Official Account Manager

**Allow bot to join
group chats** [?](#) Disabled

**Auto-reply
messages** [?](#) Disabled

Greeting messages [?](#) Disabled

Channel access token

Channel access token (long-lived) [?](#)

Ex. LINE 設定

- 示範重點：
 - Line 設定值

設定ngrok service

- 註冊ngrok
- 取用token

Ex. Ngrok 設定

- 示範重點：
 - ngrok service 驗證



ERR_NGROK_8012

Traffic successfully made it to the ngrok agent, but the agent failed to establish a connection to the upstream web service at <http://localhost:5001>. The error encountered was:

```
dial tcp 127.0.0.1:5001: connect: connection refused
```

[Get help with this error](#)

If you're the developer of this page

On the machine where the ngrok agent is running, make sure a service is running on <http://localhost:5001>. Try to cURL or open the address in a browser to see that you get the correct response.

Check out the docs to get [help with this error](#).

If you're a visitor of this page

Wait a few minutes and [refresh the page](#). If that still doesn't work, please contact the developer of this page for more information.

設定本地伺服器(1)

- 設定相關套件

```
sudo mkdir -p /etc/apt/trusted.gpg.d && curl -s https://ngrok-agent.s3.amazonaws.com/ngrok.asc | sudo tee /etc/apt/trusted.gpg.d/ngrok.asc >/dev/null && echo "deb [signed-by=/etc/apt/trusted.gpg.d/ngrok.asc] https://ngrok-agent.s3.amazonaws.com buster main" | sudo tee /etc/apt/sources.list.d/ngrok.list && sudo apt update && sudo apt install ngrok
```

- 設定ngrok

```
ngrok config add-authtoken [YOUR NGROK TOKEN]
```

設定本地伺服器(1)

- 設定LLM 及LINE 資料

```
mkdir -p ~/.config/rc-bot
```

```
cat > ~/.config/rc-bot/config.env << 'EOF'
```

```
LINE_CHANNEL_SECRET=[YOUR LINE CHANNEL SECRET]
```

```
LINE_CHANNEL_ACCESS_TOKEN=[YOUR LINE CHANNEL ACCESS TOKEN]
```

```
ALLOWED_UID=先留空_等Step5取得後再填入[未來限定為目前的開發者]
```

```
OPENWEBUI_API_KEY=sk-921dfd070d054fc786043f070f65a03f
```

```
OPENWEBUI_API_URL=http://127.0.0.1:8080/api/chat/completions
```

```
LLM_MODEL=gemma4:e4b
```

```
EOF
```

```
chmod 600 ~/.config/rc-bot/config.env
```

設定本地伺服器(1)

- 啟動ngrok

ngrok http 5050

- 查詢ngrok public URL

curl -s http://127.0.0.1:4040/api/tunnels | grep -o 'https://[^\"]*ngrok-free.app'

- 啟動本地伺服器(1)

python ex_llmservice4line_v1.py

- 啟動本地伺服器(2)

python ex_llmservice4line_v2.py

Ex. LINE Bot

- 示範重點：
 - 可與本地LLM 溝通



設定本地伺服器(2)

- 停止本地伺服器(1)
- 啟動Lab 2.10
- 啟動本地伺服器(2)

```
python ex_llmservice4line_v2.py
```

Ex. LINE Bot w/ Local RAG

- 示範重點：
 - 可與本地RAG 溝通

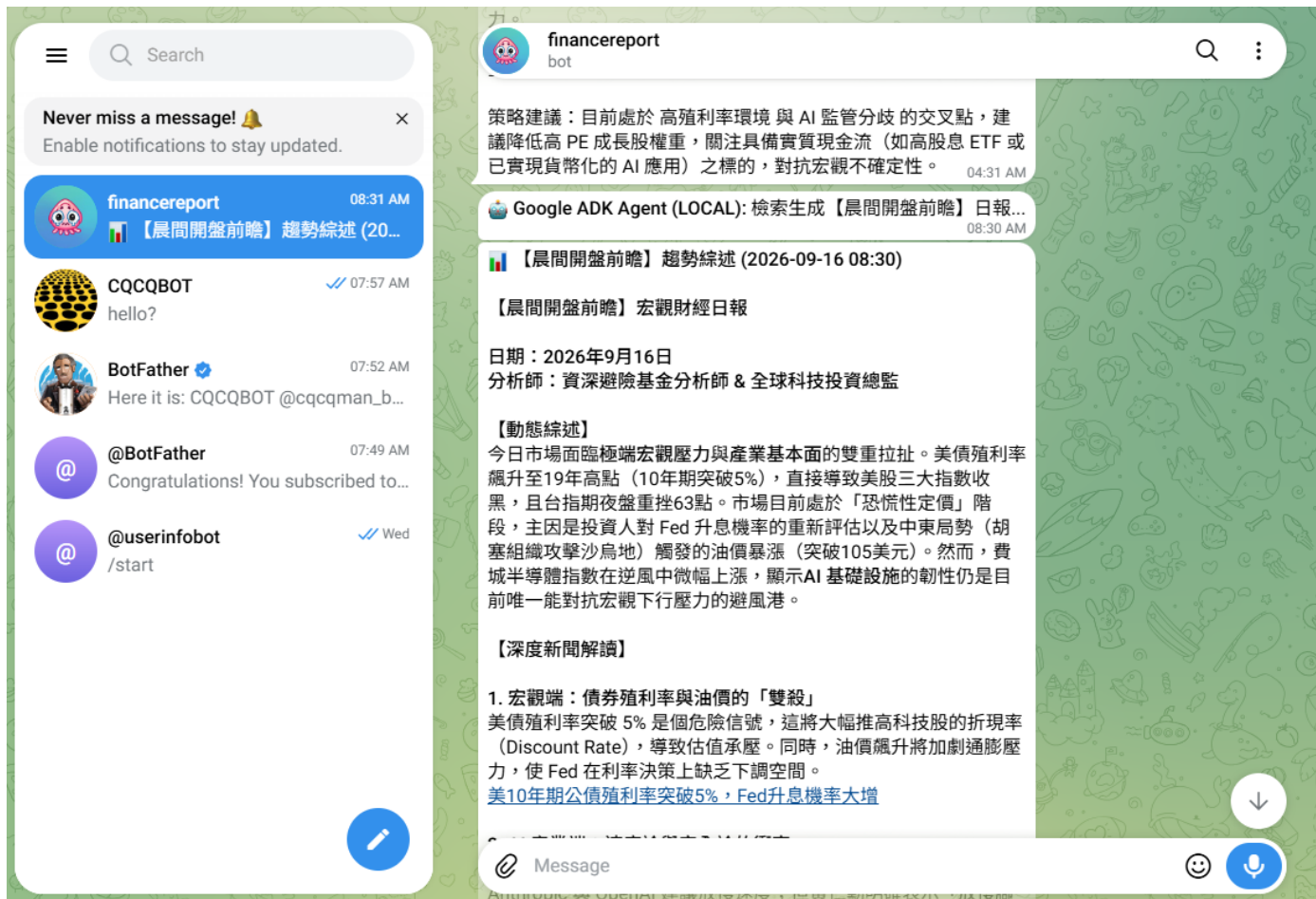
Telegram Bot 實踐

設定

- 啟動 @BotFather
- 設定自己的Bot Name
 - 取得自己Bot 的key
- 啟動自己的Bot
 - 取得自己Bot 的Chat ID

AI Agent 實例示範

真的例子



謝謝您的參與

